UNIVERSITE DE YAOUNDE I
****
ECOLE NATIONALE SUPERIEURE POLYTECHNIQUE DE YAOUNDE
****
DEPARTEMENT DE GENIE INFORMATIQUE



UNIVERSITY OF YAOUNDE I
****
NATIONAL ADVANCED SCHOOL OF ENGINEERING OF YAOUNDE
****
DEPARTMENT OF COMPUTER ENGINEERING

IMPLÉMENTATION ET ÉVALUATION PRATIQUE D'UN SCHÉMA DE SIGNATURE POST-QUANTIQUE

Mémoire de Master II

Présenté et soutenu par :

ANGOAH NGOAH MARCEL ERIC

En vue de l'obtention du :

Master II en Génie Informatique

Sous la supervision de :

HERVÉ TALÉ KALACHI, CHARGÉ DE COURS, UY1

Année académique 2025-2026

IMPLÉMENTATION ET ÉVALUATION PRATIQUE D'UN SCHÉMA DE SIGNATURE POST-QUANTIQUE

Mémoire de Master II

Présenté et soutenu par :

ANGOAH NGOAH MARCEL ERIC

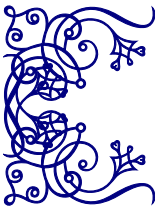
En vue de l'obtention du :

Master II en Génie Informatique

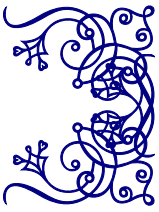
Année académique 2025-2026

Soutenu le [date]

Dédicace



(à compléter)



REMERCIEMENTS

« Une seule main ne peut nouer un fagot de bois. »
— Adage africain

Fort de cet adage, je prends la plume pour remercier celles et ceux dont le soutien m'a permis de mener à terme la rédaction de ce travail. Ce travail n'aurait pas été possible sans l'accompagnement de plusieurs personnes, à qui j'exprime ma sincère gratitude :

- **Dr Hervé Talé Kalachi**, mon encadrant, pour ses relectures exigeantes et les consignes méthodologiques qui ont structuré ce travail ;
- *(à compléter : membres du jury, enseignants du Département de Génie Informatique, collègues, famille, amis.)*

ABSTRACT

MAYO is an Oil-and-Vinegar multivariate signature scheme submitted to post-quantum standardisation. In the official AVX2 C implementation, products of the public matrix P_1 by the vectors of V are computed with `vpshufb` lookups on packed $\mathbb{F}_{16}$ vectors. When the vector length m is not a multiple of 64, the official kernel covers the tail with an overlapping 32-byte load that duplicates work. We study a local change, called A2, which handles the 8-byte tail of each P_1 entry in a single 128-bit register and one xmm `vpshufb` on the low lane of the multiplication table, reducing the number of shuffles per scalar pair from 4 to 3 (MAYO-1) and from 6 to 5 (MAYO-5). We formalise the counting argument and its limits, establish the properties on which correctness rests, and measure the effect on a physical laptop with a paired protocol. Kernel gains are 12–29 %; the full-signature gain is 10.3 % for MAYO-1 but only 1.3 % for MAYO-5. Static analysis shows that in the signature context GCC 16.2.0 emits extra 32-byte stores in several loops, asymmetrically between the two versions, consistently with the measured loss; causality is not established. GCC's dumps show that the extra stores are per-iteration stores of the accumulator arrays that are not moved out of the loop; SRA is ruled out, and why the compiler makes this choice remains open. A targeted comparison, with the two versions compiled separately and without probes (401 process pairs), gives 10.1 % for MAYO-1 and 1.5 % for MAYO-5. The compiler's analysis files (passes `lim2` and `pre`) show that the stores leave the A2 loops in the all-probes construction and not in the probe-free one; the cause of this difference remains open.

Keywords: MAYO, multivariate signatures, post-quantum cryptography, AVX2, vpshufb, finite field F16, optimisation, constant time.

RÉSUMÉ

MAYO est un schéma de signature numérique « multivarié » de type Oil-and-Vinegar. Il est candidat à la standardisation de la cryptographie post-quantique. Dans le code C officiel (MAYO-C) pour AVX2, le programme multiplie la matrice publique P_1 par les vecteurs de V avec l'instruction `vpshufb`. Les vecteurs de $\mathbb{F}_{16}$ y sont rangés à deux éléments par octet. Quand la longueur m n'est pas un multiple de 64, le code officiel traite la fin du vecteur en relisant 32 octets déjà traités. Cela fait des calculs en double : 48 éléments sur 128 pour MAYO-1 ($m = 80$) et 50 sur 192 pour MAYO-5 ($m = 142$).

Notre travail étudie une petite modification, appelée A2. Elle traite la fin de 8 octets de chaque entrée de P_1 dans un seul registre de 128 bits : un seul `vpshufb` xmm remplace deux `vpshufb` ymm. Le nombre de `vpshufb` par paire de nombres passe de 4 à 3 (MAYO-1) et de 6 à 5 (MAYO-5). Nous démontrons cette limite de comptage, nous établissons les propriétés qui rendent A2 correct, puis nous mesurons l'effet sur un ordinateur portable (Intel Core i9-9880H, GCC 16.2.0). Les mesures comparent les deux versions dans les mêmes conditions, et l'unité statistique est le processus (une exécution du programme).

Voici les résultats établis. Sur les noyaux pris seuls, A2 réduit le temps de 19 à 22 % (`P1_times_Vt`) et de 12 à 29 % (`P1t_times_Vt`). Sur la signature complète, le gain est de 10,3 % pour MAYO-1, mais de seulement 1,3 % pour MAYO-5. Pour MAYO-5, le résultat principal est donc 1,3 % ; les 7,2 % obtenus avec le binaire instrumenté (toutes les sondes actives) servent seulement à expliquer pourquoi le protocole de mesure compte. En lisant le code produit par GCC 16.2.0 (sans l'exécuter), nous voyons que, sans toutes les sondes, le compilateur ajoute dans plusieurs boucles une écriture mémoire par accumulateur (32 octets pour un registre ymm, 16 octets pour un xmm), et que ce n'est pas la même chose dans les deux versions. Les temps mesurés varient de la même façon, mais le lien de cause à effet n'est pas démontré, et un modèle à un seul paramètre exagère le gain d'environ 40 %. Ces écritures en plus sont les stockages des tableaux d'accumulateurs, refaits à chaque tour de boucle au lieu d'être faits une seule fois après la boucle. Nous écartons la passe SRA du compilateur comme cause. Pourquoi le compilateur fait ce choix reste une question ouverte. Une comparaison ciblée, avec les deux versions compilées séparément et sans sondes (401 paires de processus), donne 10,1 % pour MAYO-1 et 1,5 % pour MAYO-5. Les fichiers d'analyse du compilateur (passes `lim2` et `pre`) montrent que les écritures sortent des boucles de la version A2 dans la construction avec toutes les sondes, et pas dans celle sans sondes ; la cause de cette différence reste ouverte.

Mots-clés : MAYO, signatures multivariées, cryptographie post-quantique, AVX2, `vpshufb`**,** $\mathbb{F}_{16}$**, optimisation, mesure de performance.**

Liste des abréviations

- **AVX2 :** *Advanced Vector Extensions 2* : instructions des processeurs qui traitent 256 bits (32 octets) à la fois
- **CT :** *Constant Time* (temps constant)
- **DSE :** *Dead Store Elimination* (passe de GCC)
- **FRE :** *Full Redundancy Elimination* (passe de GCC)
- **GCC :** *GNU Compiler Collection*
- **GFNI :** *Galois Field New Instructions*
- **IC95 :** intervalle de confiance à 95 %
- **KAT :** *Known-Answer Test* : test de « réponses connues » (on compare à des valeurs officielles)
- **LIM :** *Loop Invariant Motion* (passe de GCC)
- **MAYO :** schéma de signature multivariée de type Oil-and-Vinegar
- **FIPS :** *Federal Information Processing Standards* : normes fédérales américaines de traitement de l'information
- **MQ :** *Multivariate Quadratic* : équations de degré 2 à plusieurs variables
- **NIST :** *National Institute of Standards and Technology* : Institut national américain des normes et de la technologie
- **OV, UOV :** *(Unbalanced) Oil and Vinegar*
- **PRE :** *Partial Redundancy Elimination* (passe de GCC)
- **SIMD :** *Single Instruction, Multiple Data*
- **SRA :** *Scalar Replacement of Aggregates* (passe de GCC)
- **TSC :** *Time Stamp Counter* : compteur de ticks du processeur

GLOSSAIRE

- **m-vec :** vecteur de m éléments de $\mathbb{F}_{16}$, empaqueté à deux éléments par octet.
- **Limbe :** mot de 64 bits d'un accumulateur.
- **Voie (*lane*) :** moitié de 128 bits d'un registre ymm.
- **Table de multiplication (multab) :** table de 16 octets donnant, pour une paire de nombres, les deux produits par un quartet.
- **Corps fini :** ensemble fini de nombres où l'on peut additionner, soustraire, multiplier et diviser (par exemple $\mathbb{F}_{16}$).
- **Forme quadratique :** expression de degré 2 en plusieurs variables, par exemple $x_1x_2 + x_3^2$.
- **Empreinte (hachage) :** courte suite de nombres calculée à partir d'un message ; deux messages différents ont des empreintes différentes.
- **Sel :** valeur aléatoire ajoutée au message avant le calcul de l'empreinte.
- **Compilateur, assembleur :** le compilateur transforme le code C en assembleur, la suite d'instructions que le processeur exécute.
- **Passe (du compilateur) :** une étape d'optimisation du code (par exemple SRA, PRE, DSE).
- **Bootstrap :** méthode qui rejoue le calcul sur des tirages au hasard des mesures pour estimer son incertitude.
- **Fin de vecteur :** fin d'un m-vec qui ne remplit pas un registre complet de 32 octets.
- **Recouvrement :** chargement reculé du dernier registre, qui retraite des octets déjà traités.
- **SAFE :** variante d'A2 qui ne lit rien au-delà de l'entrée de P_1.
- **Tick TSC :** unité du compteur d'horodatage (`rdtscp`) ; ce n'est pas un cycle cœur.
- **Processus :** unité statistique : une exécution du banc, 3 001 paires.

- **Doublon :** écritures de 32 octets supplémentaires par accumulateur dans le code généré, selon la construction.
- **E1, E1b :** analyses statiques du code généré (E1b : origine du doublon, exécutée le 25 octobre 2026 sous GCC 16.2.0).
- **Fait, inférence, hypothèse :** statuts des énoncés, notés **[F]**, **[I]**, **[H]**.

TABLE DES MATIÈRES

Dédicace i

Remerciements ii

Abstract iii

Résumé iv

Liste des abréviations vi

Glossaire vii

Table des matières ix

Liste des figures xiii

Liste des tableaux xiv

Introduction générale 1

Contexte et motivation . . . 1
Ce que notre travail étudie, en termes simples . . . 2
Problématique . . . 2
Objectifs de l'étude . . . 3
Méthode de recherche . . . 6
Intérêt de l'étude . . . 6
Plan du travail . . . 6

1 Généralités et état de l'art 8

1.1 Concepts mathématiques . . . 9
1.1.1 Corps fini . . . 9
1.1.2 Formes quadratiques et formes bilinéaires . . . 11
1.1.3 Systèmes linéaires . . . 12
1.2 Le problème difficile : MQ . . . 13

1.3 Le schéma MAYO . . . 13
1.3.1 Oil and Vinegar (OV) . . . 14
1.3.2 Unbalanced Oil and Vinegar (UOV) . . . 15
1.3.3 MAYO : une huile plus petite, puis le fouettage . . . 16
1.3.4 Évaluation par produits de matrices . . . 18
1.3.5 Algorithmes (version simplifiée de la spécification) . . . 19
1.3.6 Exemple complet : génération des clés, signature, vérification . . . 19
1.4 Du schéma au programme . . . 20
1.4.1 La signature dans le code de référence . . . 21
1.4.2 Représentation des données . . . 22
1.4.3 Arithmétique $\mathbb{F}_{16}$ par `vpshufb` . . . 22
1.5 Outils pour comprendre les analyses . . . 24
1.5.1 Mesurer le temps : le compteur TSC . . . 24
1.5.2 Comparer deux versions : mesure par paires . . . 25
1.5.3 Le compilateur et ses passes . . . 25
1.5.4 Lire l'assembleur sans exécuter . . . 25
1.5.5 Comment sont présentés les résultats . . . 26
1.5.6 Panorama des analyses menées . . . 26
1.6 Revue de littérature . . . 27
1.6.1 Implémentations existantes . . . 27
1.6.2 Fuites par canaux auxiliaires et temps constant . . . 27
1.7 Évaluation et positionnement . . . 28

2 Méthodologie **30**
2.1 Conception d'A2 : comptage, mise en œuvre, hypothèses et limites . . . 31
2.1.1 Modèle de comptage . . . 31
2.1.2 Énoncés . . . 31
2.1.3 Conception d'A2 . . . 33
2.1.4 La limite n'implique pas l'accélération . . . 34
2.2 Propriétés de correction et de sûreté . . . 34
2.2.1 Limites de lecture de $\mathbf{P}_1$ . . . 34
2.2.2 Limites d'écriture . . . 35
2.2.3 Invariant des deux voies des tables . . . 35
2.2.4 Influence des quartets de bourrage . . . 36
2.2.5 Temps constant : ce qui est établi et ce qui ne l'est pas . . . 36
2.2.6 Correction fonctionnelle (vérifiée par les KAT) . . . 37
2.3 Protocole expérimental . . . 37
2.3.1 Plateforme . . . 37
2.3.2 Code comparé . . . 38

2.3.3 Protocole de mesure apparié . . . 38
2.3.4 Unité statistique et intervalles . . . 38
2.3.5 Sessions . . . 38
2.3.6 Instrumentation de décomposition . . . 39
2.3.7 Analyse statique du code généré (E1) . . . 39
2.3.8 Protocole E1b (analyse statique de l'origine des écritures) . . . 39
2.3.9 Traçabilité et reproduction . . . 40
2.3.10 Menaces à la validité identifiées a priori . . . 40

3 Résultats et discussion 42
3.1 Correction . . . 43
3.2 Gains sur les noyaux et sur la signature (S1, S2) . . . 43
3.3 Temps de base du code officiel . . . 44
3.4 Comparaison ciblée . . . 46
3.5 Effet d'ordre . . . 46
3.6 Anomalie de MAYO-5 . . . 47
3.6.1 Toutes sondes actives (PH1) . . . 47
3.6.2 Le gain avec toutes les sondes n'est pas le gain sans sonde (CT1) . . . 47
3.6.3 Mesure directe, sans soustraction de médianes (config. `mcalc`) . . . 47
3.7 Analyse statique (E1) . . . 49
3.7.1 Comptage de base . . . 49
3.7.2 Écritures supplémentaires en contexte de signature . . . 49
3.8 Instructions et temps . . . 50
3.9 Analyse statique E1b . . . 50
3.9.1 Contrôles . . . 50
3.9.2 Désactivation de passes, une à la fois . . . 51
3.9.3 Ce que montrent les sorties de GCC . . . 51
3.9.4 Interprétation autorisée . . . 52
3.9.5 Les fichiers d'analyse de lim2 et de pre . . . 52
3.10 Comparaison avec la VM . . . 53
3.11 Discussion critique . . . 53
3.11.1 Bilan : établi, déduit, supposé . . . 53
3.11.2 Pourquoi le gain se dilue . . . 53
3.11.3 Ce que la concordance statique/dynamique prouve et ne prouve pas . . . 54
3.11.4 Pourquoi MAYO-1 et MAYO-5 se comportent différemment . . . 55
3.11.5 Pistes explorées puis écartées . . . 55
3.11.6 Limites de la validation . . . 56
3.11.7 Sur l'intégration dans MAYO-C . . . 56

Conclusion générale et perspectives **57**

Références **59**

A Inventaire des archives et reproduction **61**

A.1 Versions définitives à citer 61

A.2 Identification exacte du code 62

A.3 Reproduire les analyses 62

A.4 Refaire les mesures 62

B Identification des noyaux dans le code généré (GCC 16.2.0, MAYO-5) **64**

C Exemple jouet de MAYO : illustration simplifiée et contrôle **65**

D Comparaison ciblée : machine et commandes de compilation **67**

LISTE DES FIGURES

1.1 Une tentative de signature dans le code de référence. Les noyaux optimisés se trouvent dans l'étape en rouge, répétée à chaque tentative. 21
1.2 Chargements pour une entrée de $\mathbf{P}_1$ de MAYO-5 (octets). Le noyau officiel recouvre l'intervalle $[40, 64)$ et dépasse l'entrée d'un octet ; A2 SAFE lit exactement $[0, 71)$. . . . 24

3.1 Rapports A2/officiel mesurés (S1). Le gain se dilue de la couche noyau à la signature complète, fortement pour MAYO-5. 44
3.2 MAYO-5, rapport `mayo_sign` A2/officiel selon la configuration de sondes (CT1). Seule la configuration « toutes les sondes » change le résultat. 48

LISTE DES TABLEAUX

0.1 Statut des contributions. 5

1.1 Les 16 éléments de $\mathbb{F}_{16}$: chiffre hexadécimal, 4 bits et polynôme. 10
1.2 Paramètres de MAYO et tailles en octets (spécification du troisième tour, tableau 2.1). $q = 16$ pour tous. « Clé publique » : clé publique compacte ; « Clé secrète » : clé secrète compacte (une graine). 17
1.3 Paramètres des noyaux étudiés. m : longueur du m-vecteur ; $b = m/2$: octets par m-vecteur ; L : limbes de 64 bits d'un accumulateur ; K : nombre de vecteurs de V ; V : dimension de la partie vinaigre ; $N_{\text{ent}} = V(V+1)/2$: nombre d'entrées de $\mathbf{P}_1$. 22
1.4 Quelques abréviations utilisées dans les analyses. 26
1.5 Situation d'A2 par rapport aux sources consultées. 28

2.1 Application aux paramètres actuels. 32
2.2 Rapport attendu par le seul débit de `vpshufb` et rapport mesuré (S1, A2/officiel, noyaux isolés). 34
2.3 Sessions de mesure sur machine physique (dates UTC). 39
2.4 Issues possibles d'E1b et conclusions autorisées (fixées avant exécution). 40

3.1 Rapport A2/officiel, médiane des processus [IC95 bootstrap entre processus]. S1 : 20 processus ; S2 : 5 processus. Gain = 1 − rapport. 43
3.2 Temps de base du code officiel (AVX2), en ticks TSC 45
3.3 Comparaison ciblée : rapport A2 SAFE / officiel de la médiane des ticks par signature, médiane des 401 rapports par paire [IC95 bootstrap sur les paires]. 46
3.4 Rapport A2/officiel selon l'ordre d'exécution, et écart par processus [IC95]. 47
3.5 Cinq configurations du même programme (10 processus chacune, graines 41–50, un démarrage). Ticks par signature. 48
3.6 Décomposition directe de la signature MAYO-5 (ticks par signature). 48
3.7 Instructions de la boucle interne par entrée de $\mathbf{P}_1$ (MAYO-5) selon la construction. . . 49
3.8 E1b : boucles de travail selon l'option ajoutée (instructions / écritures de 32 ou 16 octets). « Dist. » : distance entre histogrammes de mnémoniques en dehors des boucles de travail, par rapport à `base`. 51
3.9 Ce qui est établi, déduit ou supposé. 54

B.1 Instructions / écritures (32 ou 16 octets) des boucles de travail selon la construction. . 64

C.1 Valeurs de l'exemple (hexadécimal, $\mathbb{F}_{16}$). 65

INTRODUCTION GÉNÉRALE

Contexte et motivation

À quoi sert une signature numérique ? Quand on télécharge une mise à jour d'un logiciel ou qu'on ouvre un site sécurisé, il faut s'assurer que le fichier vient bien de son auteur et qu'il n'a pas été modifié en route. C'est le rôle de la *signature numérique* : l'auteur signe avec une clé secrète, et tout le monde peut vérifier avec une clé publique. Les signatures utilisées aujourd'hui (RSA, ECDSA) reposent sur deux problèmes mathématiques réputés difficiles : la factorisation des grands nombres et le logarithme discret.

La menace quantique. En 1994, Shor [6] a montré qu'un grand ordinateur quantique résoudrait ces deux problèmes rapidement. Ces signatures deviendraient alors faciles à falsifier. Un tel ordinateur n'existe pas encore, mais des documents signés aujourd'hui doivent parfois rester sûrs pendant de longues années. La *cryptographie post-quantique* cherche donc des méthodes qui résistent aussi aux ordinateurs quantiques.

Normes et NIST. Une *norme* (ou « standard ») est un document officiel qui fixe la façon de réaliser quelque chose, pour que tous les systèmes soient compatibles entre eux. Pour la sécurité informatique, une institution américaine très suivie dans le monde publie ces normes : le NIST (*National Institute of Standards and Technology*, en français « Institut national américain des normes et de la technologie »). En 2024, le NIST a publié les trois premières normes post-quantiques, appelées FIPS 203, 204 et 205 [13] (FIPS signifie *Federal Information Processing Standards*, « normes fédérales de traitement de l'information »). Il continue de chercher de nouvelles signatures [14], plus petites ou plus rapides. Plusieurs candidats sont des schémas dits « multivariés », car leur sécurité repose sur la résolution d'équations à plusieurs variables.

Parmi eux, MAYO [11] est une signature de type Oil-and-Vinegar (OV) [7–9]. Son intérêt est que ses clés et ses signatures sont petites. Pour cela, MAYO utilise un petit espace secret (l'« huile »), puis le « fouette » (*whipping* en anglais) pour obtenir un système plus grand. Le calcul d'une signature consiste surtout à faire des opérations sur le corps $\mathbb{F}_{16}$, c'est-à-dire sur des nombres de 4 bits.

Certains schémas multivariés ont déjà été cassés [10]. Il faut donc rester prudent quand on parle de leur sécurité. Notre travail ne traite pas de la sécurité mathématique de MAYO. Il

traite de la *vitesse du programme* et de ses *qualités* (résultat correct, accès à la mémoire, temps d'exécution indépendant du secret). Il étudie une petite modification du code C officiel.

Déroulement du travail et recentrage. Le travail a commencé par une implémentation écrite par nous-même (arithmétique $\mathbb{F}_{16}$ par tables, étude d'une fuite d'information par le cache, élimination de Gauss–Jordan à temps constant). Cette étape était motivée par les attaques qui mesurent le temps d'exécution [15, 16]. Sur demande de l'encadrant, nous sommes ensuite passés au code C *officiel* (MAYO-C), afin d'obtenir des gains comparables à ce qui existe déjà. Notre travail ne porte que sur cette seconde phase. La première n'est pas reprise, sauf quand un résultat négatif éclaire une décision (section 3.11).

Ce que notre travail étudie, en termes simples

Un programme qui signe doit être *rapide*, *correct* et ne pas révéler d'information secrète par son temps d'exécution. Les concepteurs de MAYO ont écrit une *implémentation* (c'est-à-dire un programme en langage C) très optimisée, qui utilise les instructions *AVX2* des processeurs : ces instructions traitent 32 octets d'un seul coup. Dans ce programme, de petites fonctions appelées *noyaux* font l'essentiel du travail. Nous étudions deux de ces noyaux et nous cherchons une façon de les accélérer. Nous vérifions ensuite, avec précaution, si cette accélération se retrouve sur la signature complète.

Lien avec le sujet. Le sujet de notre travail est l'*implémentation et l'évaluation pratique d'un schéma de signature post-quantique*. Le schéma est MAYO, présenté au chapitre 1 avec ses paramètres et ses tailles. L'implémentation est notre modification A2 du code C officiel (MAYO-C), décrite au chapitre 2, et vérifiée par les tests officiels. L'évaluation pratique est la mesure du temps de signature sur une machine réelle, avec les deux versions comparées dans les mêmes conditions, et l'analyse du code produit par le compilateur (chapitre 3).

Notre travail est organisé pour que l'on sache toujours où l'on va : le chapitre 1 explique tout ce qu'il faut savoir pour comprendre MAYO, son programme et les outils d'analyse ; le chapitre 2 décrit notre proposition et la façon de la tester ; le chapitre 3 donne les résultats ; la conclusion résume ce qui est établi et ce qui reste ouvert.

Problématique

De quel écart part-on ? MAYO est intéressante parce que ses clés sont petites. Mais une signature ne sert que si elle est aussi *rapide* et *sûre* sur les vrais processeurs. Ses concepteurs ont déjà beaucoup optimisé le programme. Il reste pourtant un endroit où ce programme fait des calculs en trop : la fin des vecteurs. Il y a donc un écart entre ce que le programme fait et ce qu'il pourrait faire, et c'est cet écart qui motive notre travail.

Dans le code officiel pour AVX2, un m-vecteur de $\mathbb{F}_{16}$ (une suite de m éléments) est rangé à deux éléments par octet, soit $m/2$ octets. Pour le multiplier par un nombre, le code utilise l'instruction `vpshufb`. Cette instruction fait 32 recherches dans une table de 16 octets, et le code l'utilise pour obtenir, en une fois, les produits par *deux* nombres. Un registre de 256 bits traite donc 64 éléments à la fois. Quand m n'est pas un multiple de 64, il reste un petit morceau à la fin du vecteur. Le code officiel le traite en relisant les 32 derniers octets, ce qui *recouvre* (relit) une partie déjà traitée. L'article d'optimisation [12] décrit ce recouvrement pour $m = 96$ (MAYO-3) : il fait des calculs en double.

Pour les paramètres étudiés ici, le calcul en double est important : 48 éléments refaits sur 128 pour MAYO-1 ($m = 80$, soit 37,5 %) et 50 sur 192 pour MAYO-5 ($m = 142$, soit 26 %). La question centrale est :

> Peut-on réduire ce calcul en double en traitant la fin du vecteur de façon spéciale ? Quelle est la limite exacte, en nombre d'instructions `vpshufb` ? Et ce gain se retrouve-t-il, de façon sûre et démontrable, sur le temps de la signature complète ?

Cette question sépare trois choses que l'on confond souvent : (i) une *limite de comptage* (un résultat mathématique, dans un modèle précis), (ii) sa *mise en œuvre* par un algorithme (à montrer et à tester) et (iii) l'*effet sur le temps* (un résultat d'expérience, qui dépend du compilateur et de la machine).

Objectifs de l'étude

QR1 Quel est le plus petit nombre d'instructions `vpshufb` par paire de nombres pour un m-vecteur, avec des tables de 16 octets ? Pour quelles valeurs de m le code officiel est-il déjà optimal ?

QR2 Existe-t-il une mise en œuvre sûre de cette limite pour MAYO-1 et MAYO-5, c'est-à-dire sans lecture en dehors des données et sans adresse qui dépende du secret ?

QR3 Quel gain mesure-t-on sur les noyaux `P1_times_Vt` et `P1t_times_Vt`, puis sur `mayo_sign`, sur une machine physique, avec un protocole dont les limites sont chiffrées ?

QR4 Si le gain dans la signature complète est différent du gain sur le noyau seul, que montre l'analyse du code produit par le compilateur, et que reste-t-il à démontrer ?

Objectif général. Évaluer de façon rigoureuse et refaisable une modification des deux noyaux (A2), en séparant ce qui est démontré, ce qui est mesuré et ce qui reste ouvert.

Cet objectif se décline en cinq objectifs spécifiques. Chacun répond à une question de recherche et correspond à une partie de notre travail :

O1 *Formaliser le comptage* (limite, mise en œuvre, portée) : répond à QR1 ; chapitre 2, section 2.1.

O2 *Décrire la conception d'A2* et les propriétés dont dépend sa correction : répond à QR2 ; chapitre 2, section 2.2.

O3 *Mesurer avec un protocole que l'on peut refaire* : répond à QR3 ; chapitre 2, section 2.3, et chapitre 3.

O4 *Expliquer l'écart* entre le gain sur le noyau seul et le gain dans la signature : répond à QR4 ; chapitres 2 et 3.

O5 *Situer le travail* par rapport aux implémentations existantes, sans affirmer plus que ce que les sources consultées permettent : chapitre 1, section 1.6.

Notre travail est centré sur la variante **A2 SAFE**. Il faut dire clairement ce qui n'est pas nouveau : l'article *Nibbling MAYO* [12] (section 5.1) décrit déjà la représentation en quartets, l'utilisation de `vpshufb` et le recouvrement des registres. Notre apport est le traitement de la fin du vecteur par *un seul* `vpshufb` sur 128 bits, pour MAYO-1 et MAYO-5, avec un chargement qui reste dans l'entrée de $\mathbf{P}_1$. Nous présentons séparément deux choses : l'*économie d'instructions* dans notre modèle de comptage, et le *gain mesuré* sur la signature complète. Le premier résultat à retenir est la réduction de 10,3 % pour MAYO-1 sur notre configuration ; pour MAYO-5, le résultat principal est 1,3 %, et les 7,2 % obtenus avec le binaire instrumenté servent seulement à expliquer pourquoi le protocole de mesure compte.

Nous séparons ce qui est établi de ce qui reste ouvert. Le tableau 0.1 donne le statut de chaque contribution.

TABLEAU 0.1 : Statut des contributions.

	Contribution	Statut
C1	Comptage : nombre minimal d'instructions $\lceil m/32 \rceil$, schéma officiel $2\lceil m/64 \rceil$, écart $\Delta(m)$ (prop. 2.3–2.5)	établi (preuve)
C2	Réalisation d'A2 SAFE : fin de vecteur (≤ 16 éléments) traitée par un seul `vpshufb` sur 128 bits pour MAYO-1 et MAYO-5, sans lecture en dehors de $\mathbf{P}_1$ (le reste vient de [12])	établi (code, KAT, tests)
C3	Gain mesuré sur `mayo_sign`, présenté à part du modèle de comptage : MAYO-1 10,3 %, MAYO-5 1,3 % (résultats principaux) ; gains de noyaux isolés 12–29 % ; 7,2 % pour MAYO-5 avec le binaire instrumenté, cité seulement pour expliquer l'effet du protocole ; une machine, un compilateur	établi, avec limites
C4	Localisation de la perte de gain de MAYO-5 dans `m_calculate`, quand on mesure la signature complète	établi (mesure directe)
C5	Écritures mémoire en plus, et inégales entre les deux versions, dans le code produit par GCC 16.2.0 ; elles correspondent à la perte de C4 ; l'analyse E1b les identifie à des stockages de `temp[]`/`tt[]` dans la boucle ; la passe SRA est écartée	constat établi sur le code ; lien avec le temps *non* démontré
C6	Dépôt public (scripts, données brutes, empreintes) qui permet de refaire les 9 analyses	établi
Q1	Pourquoi GCC laisse ces écritures dans la boucle quand toutes les sondes de mesure ne sont pas actives : si les fichiers d'analyse `lim2` et `pre` ne permettent pas d'identifier la décision, la causalité reste une question ouverte	*ouvert* (fichiers `lim2`/`pre` à produire en compilation seule, MAYO-5)
Q2	Lien de cause à effet entre ces écritures et la perte de gain, avec un modèle qui n'exagère pas leur effet	*ouvert*
Q3	Variante pour une fin de vecteur de 17 à 32 éléments ; autres compilateurs et autres processeurs	*ouvert* (non implémenté)
Q4	Temps d'exécution indépendant du secret, démontré for-	*non fourni*

Méthode de recherche

Trois étapes structurent la démarche, et chacune a son chapitre : *concevoir et vérifier* (chapitre 2), *mesurer* (chapitres 2 et 3), puis *analyser le code produit* par le compilateur (chapitres 2 et 3). La démarche combine le raisonnement et l'expérience. Le comptage est démontré dans un modèle précis. La mise en œuvre est justifiée par le code et testée. Les gains sont mesurés sur une machine physique, en comparant les deux versions dans les mêmes conditions ; l'unité statistique est le processus (une exécution du programme). Le code produit par le compilateur est lu et analysé sans exécution (analyse « statique ») pour interpréter les écarts. Chaque résultat porte une étiquette qui dit s'il est un fait, une déduction ou une hypothèse. Les scripts, les données brutes et les empreintes sont publiés pour que l'on puisse tout refaire.

Le travail se limite aux noyaux `P1_times_Vt` et `P1t_times_Vt` pour MAYO-1 et MAYO-5. MAYO-2 ($m = 64$) et MAYO-3 ($m = 108$) servent de témoins : pour eux, aucun gain n'est possible avec les tables de 16 octets (proposition 2.3). Nous ne proposons aucune autre optimisation. Les noyaux optimisés ne sont pas modifiés pendant l'analyse, et la décision de les intégrer dans MAYO-C est volontairement repoussée. Le fait de ne pas trouver une technique dans les sources consultées ne prouve pas qu'elle n'existe pas ailleurs : nous ne prétendons pas à une nouveauté absolue.

Les résultats sont étiquetés ainsi : **[F]** *fait* (lu dans le code produit ou dans des mesures publiées), **[I]** *déduction* tirée des faits, **[H]** *hypothèse* non testée. Pour les propriétés de sûreté de la section 2.2, **[C]** veut dire « justifiée par le code » et **[E]** « vérifiée par des tests » ; une propriété **[E]** seule n'est pas une démonstration. Les durées sont mesurées en « ticks » du compteur TSC du processeur (instruction rdtscp), et non en cycles d'horloge du cœur.

Intérêt de l'étude

Pour ceux qui font évoluer ou utilisent MAYO-C, l'étude donne une limite précise de ce que l'on peut gagner en nombre d'instructions `vpshufb`, une mise en œuvre vérifiée pour deux jeux de paramètres, et surtout un avertissement : un gain mesuré sur un noyau seul ne se retrouve pas automatiquement sur la signature complète, parce que le code produit par le compilateur change selon le contexte. Pour la méthode, notre travail propose un protocole de mesure et d'analyse qui sépare les faits des hypothèses et que l'on peut refaire.

Plan du travail

Notre travail comprend trois chapitres, entourés par cette introduction et par une conclusion. Le chapitre 1 présente MAYO, le calcul par `vpshufb` et les implémentations existantes, puis situe A2. Le chapitre 2 formalise le comptage et décrit A2 (section 2.1), établit ses propriétés de correction et de sûreté (section 2.2) et détaille le protocole d'expérience (section 2.3). Le chapitre 3 présente

les résultats et les discute en séparant faits, déductions et hypothèses. La conclusion rappelle les contributions, les limites et les suites possibles.

CHAPITRE 1

GÉNÉRALITÉS ET ÉTAT DE L'ART

Ce chapitre fixe les concepts mathématiques, présente le schéma MAYO et ses ancêtres, son code de référence et les outils d'analyse, puis fait l'état de l'art.

1.1 Concepts mathématiques

Cette section donne les définitions mathématiques utilisées dans tout notre travail. Chaque définition est suivie d'un exemple calculé.

Notations. Les éléments d'un corps sont notés en italique (a, x_i). Les vecteurs sont notés par des lettres minuscules en gras ($\mathbf{x}$, $\mathbf{v}$) et les matrices par des lettres majuscules en gras ($\mathbf{A}$, $\mathbf{P}$). La transposée d'une matrice $\mathbf{A}$ est notée $\mathbf{A}^\top$. Les nombres de $\mathbb{F}_{16}$ sont écrits en hexadécimal, avec les lettres en minuscules (voir la section 1.1.1).

1.1.1 Corps fini

Définition 1.1 (Corps [2])**.** Un *corps* est un ensemble F muni de deux opérations, l'addition $+$ et la multiplication $\cdot$, tel que $(F,+)$ est un groupe commutatif d'élément neutre 0, $(F\setminus\{0\},\cdot)$ est un groupe commutatif d'élément neutre 1, et la multiplication est distributive par rapport à l'addition. Un corps est *fini* s'il a un nombre fini d'éléments ; ce nombre est son *ordre*.

On sait [2] que l'ordre d'un corps fini est de la forme p^n avec p premier et $n \geq 1$, et que, pour chaque tel nombre, il existe un seul corps fini à p^n éléments (à isomorphisme près), noté $\mathbb{F}_{p^n}$. Le nombre premier p est la *caractéristique* du corps : c'est le plus petit entier $k \geq 1$ tel que $k \cdot 1 = 0$.

Exemple : le corps $\mathbb{F}_2$. Prenons $p = 2$ et $n = 1$: $\mathbb{F}_2 = \{0,1\}$. L'addition est le « ou exclusif » (XOR) : $0+0=0$, $0+1=1$, $1+1=0$. La multiplication est le « et » logique : $1\cdot 1 = 1$ et tous les autres produits valent 0. Comme $1+1=0$, on a $a+a=0$ pour tout a : soustraire revient à additionner. La caractéristique est 2.

Pour construire un corps à p^n éléments avec $n > 1$, on utilise des polynômes.

Définition 1.2 (Polynôme [2])**.** Soit F un corps. Un *polynôme* sur F en l'indéterminée x est une expression $f = a_0 + a_1 x + \cdots + a_d x^d$ avec $a_0, \ldots, a_d \in F$. Si $a_d \neq 0$, l'entier d est le *degré* de f. L'ensemble des polynômes sur F est noté $F[x]$. On additionne deux polynômes coefficient par coefficient, et on les multiplie en distribuant et en utilisant $x^i \cdot x^j = x^{i+j}$.

Exemple. Sur $\mathbb{F}_2$, soit $f = x^3 + x + 1$ (de degré 3) et $g = x + 1$ (de degré 1). Alors $f + g = x^3 + (x + x) + (1 + 1) = x^3$, car $1 + 1 = 0$. Et

$$f \cdot g = (x^3 + x + 1)(x + 1) = x^4 + x^3 + x^2 + x + x + 1 = x^4 + x^3 + x^2 + 1.$$

Définition 1.3 (Polynôme irréductible [2])**.** Soit F un corps. Un polynôme $f \in F[x]$ de degré au moins 1 est *irréductible sur F* s'il ne peut pas s'écrire comme un produit de deux polynômes de $F[x]$ de degrés strictement plus petits que celui de f.

Exemple. Sur $\mathbb{F}_2$, le polynôme x^2+1 n'est pas irréductible : $x^2+1=(x+1)^2$. Le polynôme x^2+x+1 est irréductible : de degré 2, il ne pourrait être réductible qu'en un produit de deux polynômes de degré 1, donc s'il avait une racine ; or sa valeur en 0 et en 1 est 1.

Théorème 1.4 (Construction de $\mathbb{F}_{p^n}$ [2])**.** *Si f est un polynôme irréductible de degré n sur $\mathbb{F}_p$, alors l'ensemble des restes de la division par f des polynômes de $\mathbb{F}_p[x]$, muni de l'addition des polynômes et de la multiplication des polynômes suivie d'une réduction modulo f, est un corps à p^n éléments. On le note $\mathbb{F}_p[x]/(f)$.*

Le corps utilisé dans notre travail : $\mathbb{F}_{16}$. Nous utilisons $\mathbb{F}_{16}$ ($p=2$, $n=4$), construit avec $f(x)=x^4+x+1$:

$$\mathbb{F}_{16}=\mathbb{F}_2[x]/(x^4+x+1).$$

Ce polynôme est bien irréductible sur $\mathbb{F}_2$. Il n'a pas de racine, car $f(0)=1$ et $f(1)=1+1+1=1$. Il n'est pas non plus un produit de deux polynômes de degré 2 : le seul polynôme irréductible de degré 2 sur $\mathbb{F}_2$ est x^2+x+1, et son carré vaut $x^4+x^2+1\neq f$.

Un élément de $\mathbb{F}_{16}$ est un polynôme $a_0+a_1x+a_2x^2+a_3x^3$ à coefficients dans $\mathbb{F}_2$. On le note par la suite de ses quatre coefficients $a_3a_2a_1a_0$, donc par *4 bits*. Ces 4 bits se lisent comme un chiffre en base 16 : 0000 vaut 0, 1001 vaut 9, 1010 vaut 10, et ainsi de suite jusqu'à 1111 qui vaut 15. Comme il n'y a que dix chiffres (de 0 à 9), on utilise les lettres `a`, `b`, `c`, `d`, `e`, `f` pour les valeurs 10 à 15 ; nous les écrivons en minuscules (tableau 1.1).

TABLEAU 1.1 : Les 16 éléments de $\mathbb{F}_{16}$: chiffre hexadécimal, 4 bits et polynôme.

Chiffre	4 bits	Polynôme	Chiffre	4 bits	Polynôme
0	0000	0	8	1000	x^3
1	0001	1	9	1001	x^3+1
2	0010	x	a	1010	x^3+x
3	0011	$x+1$	b	1011	x^3+x+1
4	0100	x^2	c	1100	x^3+x^2
5	0101	x^2+1	d	1101	x^3+x^2+1
6	0110	x^2+x	e	1110	x^3+x^2+x
7	0111	x^2+x+1	f	1111	x^3+x^2+x+1

Pourquoi $\mathbb{F}_{16}$? C'est le corps choisi par les spécifications de MAYO [11]. Il a deux avantages pour un programme : un élément tient dans 4 bits, donc un octet contient deux éléments ; et il n'a que 16 éléments, donc toute table de multiplication par une constante n'a que 16 cases.

Les deux opérations. L'*addition* est le XOR des 4 bits (on additionne les coefficients modulo 2). La *multiplication* multiplie les polynômes puis remplace x^4 par $x+1$ (car $x^4+x+1=0$ dans le corps).

Exemple : calculs dans $\mathbb{F}_{16}$. Addition : 0110+0011 = 0101, soit 6+3 = 5. Pour la multiplication, on utilise $x^4 = x+1$, donc $x^5 = x^2+x$ et $x^6 = x^3+x^2$:

$$\begin{aligned}
\mathrm{b}\cdot 8 &= (x^3+x+1)x^3 = x^6+x^4+x^3 = (x^3+x^2)+(x+1)+x^3 = x^2+x+1 = 7,\\
9\cdot 8 &= (x^3+1)x^3 = x^6+x^3 = (x^3+x^2)+x^3 = x^2 = 4,\\
8\cdot 7 &= x^3(x^2+x+1) = x^5+x^4+x^3 = (x^2+x)+(x+1)+x^3 = x^3+x^2+1 = \mathrm{d}.
\end{aligned}$$

Pour l'inverse : $6\cdot 7 = (x^2+x)(x^2+x+1) = x^4+x = (x+1)+x = 1$. Donc 7 est l'inverse de 6 : diviser par 6 revient à multiplier par 7.

1.1.2 Formes quadratiques et formes bilinéaires

Dans cette section, F est un corps (par exemple $\mathbb{F}_{16}$), et n un entier ≥ 1. Rappelons [3] que, pour des matrices, la transposée $\mathbf{A}^\top$ a pour coefficient (i,j) le coefficient (j,i) de $\mathbf{A}$, que le produit $\mathbf{AB}$ a pour coefficient (i,j) la somme $\sum_k a_{ik}b_{kj}$, et qu'une matrice est *triangulaire supérieure* si ses coefficients sous la diagonale sont nuls.

Définition 1.5 (Forme bilinéaire [4])**.** Une application $B : F^n \times F^n \to F$ est une *forme bilinéaire* si elle est linéaire par rapport à chacune de ses deux variables : pour tous $\mathbf{x},\mathbf{x}',\mathbf{y},\mathbf{y}' \in F^n$ et tout $c \in F$,

$$\begin{aligned}
B(\mathbf{x}+\mathbf{x}',\mathbf{y}) &= B(\mathbf{x},\mathbf{y})+B(\mathbf{x}',\mathbf{y}), & B(c\,\mathbf{x},\mathbf{y}) &= c\,B(\mathbf{x},\mathbf{y}),\\
B(\mathbf{x},\mathbf{y}+\mathbf{y}') &= B(\mathbf{x},\mathbf{y})+B(\mathbf{x},\mathbf{y}'), & B(\mathbf{x},c\,\mathbf{y}) &= c\,B(\mathbf{x},\mathbf{y}).
\end{aligned}$$

Exemple : une forme bilinéaire. Sur $\mathbb{F}_{16}$, avec $n=2$, soit $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 0 & 3 \end{pmatrix}$ et

$$B(\mathbf{x},\mathbf{y}) = \mathbf{x}^\top\mathbf{A}\mathbf{y} = x_1y_1 + 2x_1y_2 + 3x_2y_2.$$

Preuve de la bilinéarité. Pour $\mathbf{y}$ fixé, $\mathbf{x} \mapsto \mathbf{x}^\top(\mathbf{A}\mathbf{y})$ est un produit scalaire par le vecteur fixe $\mathbf{A}\mathbf{y}$; il est donc additif et vérifie $B(c\mathbf{x},\mathbf{y}) = c\,B(\mathbf{x},\mathbf{y})$, par distributivité de la multiplication sur l'addition dans $\mathbb{F}_{16}$. De même, pour $\mathbf{x}$ fixé, $\mathbf{y} \mapsto (\mathbf{x}^\top\mathbf{A})\mathbf{y}$ est linéaire.

Vérification sur des nombres. Prenons $\mathbf{x} = (1,2)$, $\mathbf{x}' = (3,0)$, $\mathbf{y} = (2,1)$, $\mathbf{y}' = (1,1)$ et $c = 2$. On trouve $B(\mathbf{x},\mathbf{y}) = 6$ et $B(\mathbf{x}',\mathbf{y}) = 0$; comme $\mathbf{x}+\mathbf{x}' = (2,2)$, $B(\mathbf{x}+\mathbf{x}',\mathbf{y}) = 6 = 6+0$. De même $B(\mathbf{x},\mathbf{y}') = 5$ et $B(\mathbf{x},\mathbf{y}+\mathbf{y}') = B(\mathbf{x},(3,0)) = 3 = 6+5$. Enfin $B(c\mathbf{x},\mathbf{y}) = B((2,4),\mathbf{y}) = \mathrm{c} = 2\cdot 6 = c\,B(\mathbf{x},\mathbf{y})$.

Définition 1.6 (Forme quadratique [2])**.** Une *forme quadratique* en n variables sur F est un polynôme homogène de degré 2 en $x_1,\ldots,x_n$:

$$p(\mathbf{x}) = \sum_{1\leq i\leq j\leq n} a_{ij}x_ix_j, \qquad a_{ij} \in F.$$

Si $\mathbf{P}$ est la matrice triangulaire supérieure de coefficients a_{ij} ($a_{ij} = 0$ pour $i > j$), alors $p(\mathbf{x}) = \mathbf{x}^\top\mathbf{P}\mathbf{x}$. Elle vérifie $p(c\,\mathbf{x}) = c^2\,p(\mathbf{x})$ pour tout $c \in F$.

Exemple : une forme quadratique. Sur $\mathbb{F}_{16}$, soit $\mathbf{P} = \begin{pmatrix} 1 & 2 \\ 0 & 3 \end{pmatrix}$, donc $p(\mathbf{x}) = x_1^2 + 2x_1x_2 + 3x_2^2$: chaque terme est de degré 2, c'est bien une forme quadratique. Pour $\mathbf{x} = (1,2)$,

$$p(\mathbf{x}) = 1 \cdot 1^2 + 2 \cdot (1 \cdot 2) + 3 \cdot 2^2 = 1 + 4 + \mathrm{c} = 9.$$

Vérification de $p(c\,\mathbf{x}) = c^2 p(\mathbf{x})$. Avec $c = 2$, $c\,\mathbf{x} = (2,4)$ et $p(2,4) = 2$; or $c^2 = 4$ et $4 \cdot 9 = 2$. Les deux valeurs sont égales.

Proposition 1.7 (Forme polaire)**.** *Soit* $p(\mathbf{x}) = \mathbf{x}^\top \mathbf{P}\mathbf{x}$ *une forme quadratique. L'application* $p'(\mathbf{x},\mathbf{y}) = p(\mathbf{x}+\mathbf{y}) - p(\mathbf{x}) - p(\mathbf{y})$ *est une forme bilinéaire, et* $p'(\mathbf{x},\mathbf{y}) = \mathbf{x}^\top(\mathbf{P} + \mathbf{P}^\top)\mathbf{y}$ *lorsque le corps est de caractéristique 2.*

Démonstration. On développe $p(\mathbf{x}+\mathbf{y}) = (\mathbf{x}+\mathbf{y})^\top \mathbf{P}(\mathbf{x}+\mathbf{y}) = p(\mathbf{x}) + p(\mathbf{y}) + \mathbf{x}^\top\mathbf{P}\mathbf{y} + \mathbf{y}^\top\mathbf{P}\mathbf{x}$. Un nombre est égal à sa transposée, donc $\mathbf{y}^\top\mathbf{P}\mathbf{x} = \mathbf{x}^\top\mathbf{P}^\top\mathbf{y}$ et $p'(\mathbf{x},\mathbf{y}) = \mathbf{x}^\top(\mathbf{P} + \mathbf{P}^\top)\mathbf{y}$. Cette expression est linéaire en $\mathbf{x}$ et en $\mathbf{y}$ (voir l'exemple de forme bilinéaire). En caractéristique 2, « moins » est égal à « plus ». ■

Exemple : la forme polaire. Avec la même matrice $\mathbf{P}$, prenons $\mathbf{x} = (1,2)$ et $\mathbf{y} = (2,1)$. On a $p(\mathbf{x}) = 9$, $p(\mathbf{y}) = 1 \cdot 4 + 2 \cdot 2 + 3 \cdot 1 = 3$, $\mathbf{x}+\mathbf{y} = (3,3)$ et $p(\mathbf{x}+\mathbf{y}) = (1+2+3) \cdot (3 \cdot 3) = 0$. Donc $p'(\mathbf{x},\mathbf{y}) = 0 + 9 + 3 = \mathrm{a}$. Avec la formule : $\mathbf{P} + \mathbf{P}^\top = \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}$, donc $\mathbf{x}^\top(\mathbf{P} + \mathbf{P}^\top)\mathbf{y} = 2 \cdot (1 \cdot 1) + 2 \cdot (2 \cdot 2) = 2 + 8 = \mathrm{a}$. Les deux méthodes donnent le même résultat. Bilinéarité : avec $\mathbf{x}' = (3,0)$, $p'(\mathbf{x}',\mathbf{y}) = 2 \cdot (3 \cdot 1) = 6$ et $p'(\mathbf{x}+\mathbf{x}',\mathbf{y}) = p'((2,2),\mathbf{y}) = 2 \cdot (2 \cdot 1) + 2 \cdot (2 \cdot 2) = 4 + 8 = \mathrm{c} = \mathrm{a} + 6$.

Définition 1.8 (Application quadratique)**.** Soient $p_1, \ldots, p_m$ des formes quadratiques en n variables sur $\mathbb{F}_q$. L'application

$$\mathscr{P} : \mathbb{F}_q^n \longrightarrow \mathbb{F}_q^m, \qquad \mathscr{P}(\mathbf{x}) = \big(p_1(\mathbf{x}), \ldots, p_m(\mathbf{x})\big)$$

est une *application quadratique*. Son application polaire est $\mathscr{P}'(\mathbf{x},\mathbf{y}) = \mathscr{P}(\mathbf{x}+\mathbf{y}) - \mathscr{P}(\mathbf{x}) - \mathscr{P}(\mathbf{y})$; elle est bilinéaire, composante par composante.

Exemple. Avec $n = m = 2$ sur $\mathbb{F}_{16}$, prenons $p_1 = p$ (la forme de l'exemple précédent) et $p_2(\mathbf{x}) = x_1^2 + x_2^2$. Alors $\mathscr{P} : \mathbb{F}_{16}^2 \to \mathbb{F}_{16}^2$ et $\mathscr{P}(1,2) = (9,\ 1+4) = (9,5)$.

1.1.3 Systèmes linéaires

Définition 1.9 (Système linéaire [3])**.** Soit $\mathbf{A} \in F^{m \times n}$ une matrice et $\mathbf{b} \in F^m$ un vecteur. Le *système linéaire* $\mathbf{A}\mathbf{x} = \mathbf{b}$ a pour inconnue $\mathbf{x} \in F^n$; une *solution* est un vecteur $\mathbf{x}$ qui vérifie les m équations.

L'*élimination de Gauss* [3] résout un tel système : on peut échanger deux lignes, multiplier une ligne par un nombre non nul, ou ajouter à une ligne un multiple d'une autre, sans changer l'ensemble des solutions ; on se ramène ainsi à un système où les inconnues apparaissent une à une. Un système peut n'avoir aucune solution, une seule, ou plusieurs ; si $\mathbf{A}$ est carrée et inversible, la solution est unique.

Exemple avec une solution. Sur $\mathbb{F}_{16}$, soit $x_1 + x_2 = 3$ et $x_1 + 2x_2 = 5$. On additionne les deux lignes : $(2+1)x_2 = 5+3$, soit $3x_2 = 6$. L'inverse de 3 est e, donc $x_2 = 6 \cdot \mathrm{e} = 2$. Puis $x_1 = 3 + 2 = 1$. Contrôle : $1 + 2 \cdot 2 = 1 + 4 = 5$.

Exemple sans solution. Soit $x_1 + x_2 = 1$ et $x_1 + x_2 = 2$. Les deux lignes ont le même membre de gauche mais deux résultats différents : aucune valeur ne convient.

1.2 Le problème difficile : MQ

Cette section présente le problème mathématique sur lequel repose la sécurité de MAYO. La notion de signature est introduite plus loin, au moment où on en a besoin, pour chacun des trois schémas : Oil and Vinegar, UOV, puis MAYO.

Définition 1.10 (Problème MQ [11])**.** Soient q une puissance d'un nombre premier et n, m des entiers. Étant donnés une application quadratique $\mathscr{P} : \mathbb{F}_q^n \to \mathbb{F}_q^m$ et une cible $\mathbf{t} \in \mathbb{F}_q^m$, le problème *MQ* (*Multivariate Quadratic*) consiste à trouver $\mathbf{s} \in \mathbb{F}_q^n$ tel que $\mathscr{P}(\mathbf{s}) = \mathbf{t}$.

Pour des paramètres bien choisis, on ne connaît pas de méthode rapide pour résoudre ce problème. Par contre, *vérifier* qu'un point $\mathbf{s}$ est une solution est facile : il suffit de calculer $\mathscr{P}(\mathbf{s})$.

Exemple : un tout petit problème MQ. Sur $\mathbb{F}_2$, avec $n = 3$ variables et $m = 2$ polynômes :

$$p_1(\mathbf{x}) = x_1 x_2 + x_3^2, \qquad p_2(\mathbf{x}) = x_1^2 + x_2 x_3, \qquad \mathbf{t} = (1, 0).$$

Avec 3 variables, on peut tester les 8 points : $\mathbf{s} = (0,0,1)$ convient, car $p_1(\mathbf{s}) = 0 \cdot 0 + 1 \cdot 1 = 1$ et $p_2(\mathbf{s}) = 0 \cdot 0 + 0 \cdot 1 = 0$. Avec 100 variables, tout tester serait impossible. Les schémas qui suivent utilisent cette différence de difficulté : résoudre est dur, vérifier est facile, *sauf pour celui qui connaît un secret*.

1.3 Le schéma MAYO

Cette section construit MAYO en trois étapes : Oil and Vinegar (OV), puis UOV, puis MAYO. Pour chaque étape, nous donnons la définition, nous précisons ce que signifie « signer » dans ce schéma, puis nous déroulons un *exemple complet* : génération des clés, signature, vérification. Les paramètres des exemples sont minuscules pour que l'on puisse tout vérifier à la main.

1.3.1 Oil and Vinegar (OV)

Avant de définir OV, rappelons ce qu'est une signature numérique.

Définition 1.11 (Schéma de signature [5])**.** Un *schéma de signature numérique* est formé de trois algorithmes :

- KeyGen produit une paire (clé publique, clé secrète) ;
- Sign prend la clé secrète et un message M et produit une signature ;
- Verify prend la clé publique, un message et une signature, et répond « accepté » ou « refusé ».

Un schéma est *correct* si toute signature produite par Sign avec la clé secrète est acceptée par Verify avec la clé publique correspondante.

Un message peut être très long : on le transforme d'abord en une courte liste de nombres, son *empreinte* $\mathbf{t}$, avec une *fonction de hachage* H [5] (dans MAYO, on ajoute aussi au message un *sel*, valeur aléatoire : $\mathbf{t} = H(M \,\|\, \text{sel})$).

Définition 1.12 (Oil and Vinegar [7, 9, 11])**.** Soient v et o des entiers, $n = v + o$ et $m = o$. La *clé secrète* est une matrice $\mathbf{O} \in \mathbb{F}_q^{v \times o}$; elle définit l'*espace d'huile*

$$\mathscr{O} = \left\{ \begin{pmatrix} \mathbf{O}\mathbf{x} \\ \mathbf{x} \end{pmatrix} : \ \mathbf{x} \in \mathbb{F}_q^{o} \right\} \subset \mathbb{F}_q^{n}.$$

La *clé publique* est une application quadratique $\mathscr{P} : \mathbb{F}_q^n \to \mathbb{F}_q^m$ qui vaut zéro sur $\mathscr{O}$. Les v premières variables sont dites « vinaigre », les o dernières « huile ». Dans la version d'origine, dite *équilibrée*, $v = o$.

Chaque polynôme s'écrit $p_\ell(\mathbf{y}) = \mathbf{y}^\top \mathbf{P}_\ell \mathbf{y}$, où $\mathbf{P}_\ell = \begin{pmatrix} \mathbf{P}_\ell^{(1)} & \mathbf{P}_\ell^{(2)} \\ \mathbf{0} & \mathbf{P}_\ell^{(3)} \end{pmatrix}$ est triangulaire supérieure, découpée en blocs (vinaigre–vinaigre, vinaigre–huile, huile–huile). Pour que p_ℓ s'annule sur $\mathscr{O}$, on pose $\mathbf{P}_\ell^{(3)} = \mathrm{Upper}\big(\mathbf{O}^\top(\mathbf{P}_\ell^{(1)}\mathbf{O} + \mathbf{P}_\ell^{(2)})\big)$, où Upper ajoute à chaque case au-dessus de la diagonale la case symétrique et met à zéro les cases en dessous.

Ce que « signer » veut dire ici. *Signer* M revient à trouver $\mathbf{s}$ tel que $\mathscr{P}(\mathbf{s}) = \mathbf{t}$, c'est-à-dire à résoudre un problème MQ *grâce au secret*. *Vérifier* revient à calculer $\mathscr{P}(\mathbf{s})$ et à comparer avec $\mathbf{t}$.

Comment le secret aide. On tire d'abord les variables vinaigre $\mathbf{v}$ au hasard. On cherche alors $\mathbf{s} = (\mathbf{v} + \mathbf{O}\mathbf{x},\, \mathbf{x})$. Comme p_ℓ vaut zéro sur l'huile, on obtient (calcul fait dans la partie MAYO)

$$p_\ell(\mathbf{v} + \mathbf{O}\mathbf{x}, \mathbf{x}) = \mathbf{v}^\top \mathbf{P}_\ell^{(1)} \mathbf{v} + \mathbf{v}^\top \mathbf{L}_\ell \mathbf{x}, \qquad \mathbf{L}_\ell = (\mathbf{P}_\ell^{(1)} + \mathbf{P}_\ell^{(1)\top})\mathbf{O} + \mathbf{P}_\ell^{(2)},$$

qui est *linéaire* en $\mathbf{x}$. Il reste un système linéaire de m équations à o inconnues (section 1.1.3).

Exemple complet OV. Corps $\mathbb{F}_{16}$, $v = o = m = 2$ donc $n = 4$. Le secret est $\mathbf{O} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$.

Génération des clés. On tire au hasard

$$\mathbf{P}_1^{(1)} = \begin{pmatrix} 1 & 2 \\ 0 & 3 \end{pmatrix}, \mathbf{P}_1^{(2)} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}; \qquad \mathbf{P}_2^{(1)} = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}, \mathbf{P}_2^{(2)} = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix}.$$

La formule ci-dessus donne

$$\mathbf{P}_1^{(3)} = \begin{pmatrix} 3 & 8 \\ 0 & \mathrm{b} \end{pmatrix}, \qquad \mathbf{P}_2^{(3)} = \begin{pmatrix} 9 & 6 \\ 0 & 3 \end{pmatrix}.$$

La clé publique est formée des deux matrices $\mathbf{P}_1, \mathbf{P}_2$ complètes ; la clé secrète est $\mathbf{O}$.

Signature. L'empreinte est $\mathbf{t} = (3,7)$. On tire les vinaigres $\mathbf{v} = (1,2)$. On calcule

$$\mathbf{L}_1 = \begin{pmatrix} 3 & 6 \\ 6 & 3 \end{pmatrix}, \mathbf{L}_2 = \begin{pmatrix} 0 & 2 \\ 2 & 3 \end{pmatrix}; \qquad \mathbf{v}^\top \mathbf{L}_1 = (\mathrm{f}, 0),\ \mathbf{v}^\top \mathbf{L}_2 = (4,4); \qquad \left(\mathbf{v}^\top \mathbf{P}_\ell^{(1)} \mathbf{v}\right)_\ell = (9,4).$$

Les inconnues $\mathbf{x} = (x_1, x_2)$ vérifient donc

$$\mathrm{f} x_1 = 3 + 9 = \mathrm{a}, \qquad 4x_1 + 4x_2 = 7 + 4 = 3.$$

La première équation donne $x_1 = \mathrm{f}$ (car $\mathrm{f} \cdot \mathrm{f} = \mathrm{a}$). La seconde donne $4x_2 = 3 + 4 \cdot \mathrm{f} = 3 + 9 = \mathrm{a}$, donc $x_2 = \mathrm{b}$. On a alors $\mathbf{Ox} = (6,1)$ et la signature est

$$\mathbf{s} = (\mathbf{v} + \mathbf{Ox}, \mathbf{x}) = (7, 3, \mathrm{f}, \mathrm{b}).$$

Vérification. On évalue la clé publique complète en $\mathbf{s}$: $p_1(\mathbf{s}) = 3$ et $p_2(\mathbf{s}) = 7$. On retrouve $\mathbf{t} = (3,7)$: la signature est *acceptée*. Si l'on change un seul élément (par exemple $7 \to 6$ au début de $\mathbf{s}$), on obtient $(\mathrm{b}, 2) \neq \mathbf{t}$: la signature est *refusée*.

La fin d'OV. La version équilibrée ($v = o$) a été cassée en 1998 par Kipnis et Shamir [8]. Il a donc fallu la corriger : c'est UOV.

1.3.2 Unbalanced Oil and Vinegar (UOV)

Définition 1.13 (UOV [9])**.** Le schéma *Unbalanced Oil and Vinegar* (UOV), proposé en 1999 par Kipnis, Patarin et Goubin, est le schéma OV avec $v > o$ (beaucoup plus de variables vinaigre que de variables huile) et $m = o$.

L'attaque contre la version équilibrée ne marche plus, et UOV n'a pas été cassé depuis. Son défaut est la taille de la clé publique : $n = v + o$ est grand, et la clé contient m matrices triangulaires de taille $n \times n$.

Ce que « signer » veut dire ici. Rien ne change dans les trois algorithmes ni dans le rôle de l'empreinte $\mathbf{t}$: signer, c'est trouver $\mathbf{s}$ avec $\mathscr{P}(\mathbf{s}) = \mathbf{t}$; vérifier, c'est calculer $\mathscr{P}(\mathbf{s})$. Seuls les paramètres changent ($v > o$).

Exemple complet UOV (notre exemple fil rouge). Corps $\mathbb{F}_{16}$, $v = 2$, $o = 1$, $m = 1$, donc $n = 3$. Le secret est $\mathbf{O} = (\mathrm{b}, 8)^\top$: l'espace d'huile est formé des points $(\mathrm{b}x,\ 8x,\ x)$.

Génération des clés. Le polynôme public est $p_1(\mathbf{y}) = \mathbf{y}^\top \mathbf{P}_1 \mathbf{y}$ avec

$$\mathbf{P}_1^{(1)} = \begin{pmatrix} 8 & 9 \\ 0 & 6 \end{pmatrix}, \quad \mathbf{P}_1^{(2)} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \quad \mathbf{P}_1^{(3)} = \mathrm{Upper}\big(\mathbf{O}^\top(\mathbf{P}_1^{(1)}\mathbf{O} + \mathbf{P}_1^{(2)})\big) = (6),$$

c'est-à-dire $p_1(\mathbf{y}) = 8y_1^2 + 9y_1y_2 + 2y_1y_3 + 6y_2^2 + 2y_2y_3 + 6y_3^2$.

Propriété de l'huile. Pour $x = 1$, les six termes de $p_1(\mathrm{b}, 8, 1)$ valent $4, \mathrm{a}, 5, \mathrm{e}, 3, 6$; leur somme (XOR) vaut 0. Donc p_1 s'annule sur l'espace d'huile.

Signature. On vise la valeur $t = \mathrm{f}$. On tire les vinaigres $\mathbf{v} = (2, 3)$. Alors $c = p_1(2, 3, 0) = 8$, et il reste à atteindre $\mathrm{f} + 8 = 7$. On a $\mathbf{L}_1 = (6, \mathrm{e})^\top$ et $\mathbf{v}^\top \mathbf{L}_1 = \mathrm{d}$. On résout $\mathrm{d} \cdot x = 7$, ce qui donne $x = 7 \cdot 4 = \mathrm{f}$ (l'inverse de d est 4). La signature est

$$\mathbf{s} = (\mathbf{v} + \mathbf{O}x,\ x) = (2 + \mathrm{b} \cdot \mathrm{f},\ 3 + 8 \cdot \mathrm{f},\ \mathrm{f}) = (1, 2, \mathrm{f}).$$

Vérification. On calcule $p_1(1, 2, \mathrm{f}) = \mathrm{f} = t$: la signature est acceptée.

1.3.3 MAYO : une huile plus petite, puis le fouettage

Définition 1.14 (MAYO [11])**.** Soient $q = 16$ et des entiers n, m, o, k avec $o < m$ et $v = n - o$. Un schéma *MAYO* est un schéma OV dont l'espace d'huile est *trop petit* ($o < m$), muni de la fonction publique étendue « fouettée » $\mathscr{P}^\star : \mathbb{F}_q^{kn} \to \mathbb{F}_q^m$ et d'un terme linéaire L, définis ci-dessous (spécification du troisième tour [1]). Une signature est $\mathbf{S} = (\mathbf{s}_0, \ldots, \mathbf{s}_{k-1})$ avec $\mathbf{s}_i \in \mathbb{F}_q^n$, accompagnée d'un sel.

Un espace d'huile plus petit réduit beaucoup la taille des clés, mais on ne peut plus résoudre directement : il y a m équations pour seulement o inconnues.

Exemple : pourquoi $k = 1$ ne suffit pas. Ajoutons à l'exemple UOV une seconde équation ($m = 2$ pour $o = 1$) :

$$p_2(\mathbf{y}) = 8y_1^2 + \mathrm{a}y_1y_2 + \mathrm{b}y_1y_3 + 8y_2^2 + \mathrm{c}y_2y_3 + \mathrm{e}y_3^2 \quad \big(\mathbf{P}_2^{(1)} = \begin{pmatrix} 8 & \mathrm{a} \\ 0 & 8 \end{pmatrix},\ \mathbf{P}_2^{(2)} = \begin{pmatrix} \mathrm{b} \\ \mathrm{c} \end{pmatrix},\ \mathbf{P}_2^{(3)} = (\mathrm{e})\big).$$

Prenons $\mathbf{t} = (\mathrm{f}, 9)$ et les mêmes vinaigres $\mathbf{v} = (2, 3)$. On a $p_2(\mathbf{v}, 0) = 1$ et, pour p_2, le coefficient $\mathbf{v}^\top \mathbf{L}_2 = 9$. Avec la seule inconnue x, on doit résoudre

$$p_1 :\ \mathrm{d} \cdot x = \mathrm{f} + 8 = 7 \qquad \text{et} \qquad p_2 :\ 9 \cdot x = 9 + 1 = 8.$$

La première équation donne $x = \mathrm{f}$, la seconde donne $x = 3$. Les deux valeurs sont différentes : *pas de solution*. C'est le problème d'une huile trop petite : deux équations pour une seule inconnue.

Le fouettage et le terme linéaire. La spécification du troisième tour [1] définit la fonction publique de MAYO à partir de $\mathscr{P}$ et de deux ingrédients supplémentaires. Dans cette section, les indices des k vecteurs vont de 0 à $k-1$, comme dans la spécification.

- Le *fouettage* (*whipping*) : $\mathscr{P}^\star(\mathbf{s}_0,\dots,\mathbf{s}_{k-1}) = \sum_{i=0}^{k-1} \mathbf{E}_{ii}\,\mathscr{P}(\mathbf{s}_i) + \sum_{i=0}^{k-1}\sum_{j=i+1}^{k-1} \mathbf{E}_{ij}\,\mathscr{P}'(\mathbf{s}_i,\mathbf{s}_j)$, où $\mathscr{P}'$ est l'application polaire (section 1.1). La matrice publique $\mathbf{E}_{ij} \in \mathbb{F}_q^{m\times m}$ représente la multiplication par $Z[i,j]$ modulo un polynôme irréductible $f(z)$ de degré m, où Z est une matrice symétrique $k\times k$ dont les coefficients sont des puissances de z.
- Le *terme linéaire dérivé de* $\boldsymbol{\Lambda}$: $L(\boldsymbol{\Lambda},\mathbf{s}_0,\dots,\mathbf{s}_{k-1}) = \sum_{i=0}^{k-1} \mathbf{E}_{i0}\,\boldsymbol{\Lambda}\,\mathbf{s}_i$, où $\boldsymbol{\Lambda} \in \mathbb{F}_q^{m\times n}$ est une matrice publique, calculée à partir du message et du sel (voir ci-dessous).

Une *signature* d'un message M est un couple $(\mathbf{S},\text{sel})$, avec $\mathbf{S} = (\mathbf{s}_0,\dots,\mathbf{s}_{k-1})$, tel que

$$\mathscr{P}^\star(\mathbf{S}) + L(\boldsymbol{\Lambda},\mathbf{S}) = \mathbf{t},$$

où $\mathbf{t}$ et $\boldsymbol{\Lambda}$ sont tirés d'un même haché : SHAKE256(SHAKE256(M) ‖ sel) [1]. Les paramètres m,k,o,n définissent quatre jeux (MAYO-1, 2, 3 et 5), selon le niveau de sécurité demandé par le NIST [14]. Les inconnues sont maintenant $k\cdot o$, ce qui donne assez d'inconnues pour les m équations.

Les quatre jeux de paramètres et leurs tailles. Le tableau 1.2 donne les paramètres et les tailles des clés et des signatures, tels que la spécification du troisième tour les fixe (tableau 2.1 de [1]). Ce sont des chiffres de la spécification : nous ne les avons pas mesurés. Ils montrent l'intérêt de MAYO : une clé publique de 1 456 à 5 554 octets et une signature de 239 à 964 octets, avec une clé secrète réduite à une graine de 24 à 40 octets.

TABLEAU 1.2 : Paramètres de MAYO et tailles en octets (spécification du troisième tour, tableau 2.1). $q = 16$ pour tous. « Clé publique » : clé publique compacte ; « Clé secrète » : clé secrète compacte (une graine).

Jeu	Niveau NIST	n	m	o	k	Sel	Clé publique	Clé secrète	Signature
MAYO-1	1	88	80	8	10	24	1 456	24	464
MAYO-2	1	86	64	13	5	24	2 928	24	239
MAYO-3	3	118	108	10	11	32	2 986	32	681
MAYO-5	5	154	142	12	12	40	5 554	40	964

Contrôle des tailles. Une signature contient k vecteurs de n éléments de $\mathbb{F}_{16}$ (deux éléments par octet), plus le sel : $\lceil nk/2\rceil$ + sel. Pour MAYO-1, 440 + 24 = 464 ; pour MAYO-5, 924 + 40 = 964. La clé publique compacte contient une graine de 16 octets et la matrice $\mathbf{P}^{(3)}$, soit $m\,o(o+1)/2$ éléments, donc $m\,o(o+1)/4$ octets : pour MAYO-1, 16 + 1 440 = 1 456 ; pour MAYO-5, 16 + 5 538 = 5 554. Les quatre lignes du tableau vérifient ces formules.

Ce que « signer » veut dire ici. Signer M, c'est trouver $\mathbf{S}$ tel que $\mathscr{P}^\star(\mathbf{S}) + L(\mathbf{\Lambda}, \mathbf{S}) = \mathbf{t}$ (algorithme 7 de la spécification [1]). Vérifier, c'est recalculer $\mathbf{t}$ et $\mathbf{\Lambda}$ à partir du message et du sel, calculer le membre de gauche avec la clé publique et comparer à $\mathbf{t}$ (algorithme 8). Les trois algorithmes ont la même forme que dans OV et UOV ; la différence est que l'on résout pour k vecteurs à la fois, et que le terme linéaire s'ajoute à la fois dans la construction du système et dans la vérification.

Où intervient le terme linéaire. Avec $\mathbf{s}_i = (\mathbf{v}_i + \mathbf{O}\mathbf{x}_i,\ \mathbf{x}_i)$ et $\mathbf{\Lambda} = (\mathbf{\Lambda}_v\ \mathbf{\Lambda}_o)$ (les $n-o$ premières colonnes, puis les o dernières), on a $\mathbf{\Lambda}\mathbf{s}_i = \mathbf{\Lambda}_v\mathbf{v}_i + (\mathbf{\Lambda}_v\mathbf{O} + \mathbf{\Lambda}_o)\mathbf{x}_i$. Le terme linéaire a donc deux effets pour la signature :

- il ajoute au second membre le nombre connu $\mathbf{E}_{i0}\mathbf{\Lambda}_v\mathbf{v}_i$ (on le passe de l'autre côté : $\mathbf{y} \leftarrow \mathbf{y} - \mathbf{E}_{i0}\mathbf{\Lambda}_v\mathbf{v}_i$) ;
- il ajoute au coefficient des inconnues le terme $\mathbf{E}_{i0}(\mathbf{\Lambda}_v\mathbf{O} + \mathbf{\Lambda}_o)$, que l'algorithme 7 obtient en remplaçant $\mathbf{M}_0$ par $\mathbf{M}_0 + \mathbf{\Lambda}_v\mathbf{O} + \mathbf{\Lambda}_o$.

À la vérification, le terme linéaire s'ajoute simplement : $\mathbf{y} \leftarrow \mathbf{y} + \mathbf{E}_{i0}\mathbf{\Lambda}\mathbf{s}_i$ pour chaque i (algorithme 8).

Exemple : le fouettage pour $k = 2$. On prend $k = 2$ vecteurs, s_0 et s_1, donc deux inconnues x_0 et x_1 pour les deux équations. Une liste de $m = 2$ éléments (a_0, a_1) représente le polynôme $a_0 + a_1 z$, et l'on calcule modulo $f(z) = z^2 + z + 8$ (ce polynôme n'a aucune racine dans $\mathbb{F}_{16}$: on l'a vérifié en testant les 16 valeurs ; il est donc irréductible, car il est de degré 2). Multiplier par z s'écrit

$$z \cdot (a_0, a_1) = (8 \cdot a_1,\ a_0 + a_1).$$

Par exemple $z \cdot (\mathrm{d}, 4) = (8 \cdot 4,\ \mathrm{d} + 4) = (6, 9)$. Pour $k = 2$ nous prenons, comme illustration simplifiée, $Z = \begin{pmatrix} 1 & z \\ z & z^2 \end{pmatrix}$: c'est une matrice symétrique dont les coefficients sont des puissances de z. Les vraies matrices Z de la spécification sont plus grandes et choisies de façon plus soignée. On a donc $\mathbf{E}_{00} = \mathbf{I}$ (multiplication par 1), $\mathbf{E}_{01} = \mathbf{E}_{10}$ (multiplication par z) et $\mathbf{E}_{11}$ (multiplication par z^2), et

$$\mathscr{P}^\star(\mathbf{s}_0, \mathbf{s}_1) = \mathscr{P}(\mathbf{s}_0) + z \cdot \mathscr{P}'(\mathbf{s}_0, \mathbf{s}_1) + z^2 \cdot \mathscr{P}(\mathbf{s}_1), \qquad L(\mathbf{\Lambda}, \mathbf{S}) = \mathbf{\Lambda}\mathbf{s}_0 + z \cdot \mathbf{\Lambda}\mathbf{s}_1.$$

Chaque terme est une liste de 2 éléments (un par équation).

1.3.4 Évaluation par produits de matrices

Les m polynômes s'écrivent $p_\ell(\mathbf{y}) = \mathbf{y}^\top \mathbf{P}_\ell \mathbf{y}$ avec $\mathbf{P}_\ell = \begin{pmatrix} \mathbf{P}_\ell^{(1)} & \mathbf{P}_\ell^{(2)} \\ \mathbf{0} & \mathbf{P}_\ell^{(3)} \end{pmatrix}$; $\mathbf{P}_\ell^{(1)}$ est de taille $v \times v$, $\mathbf{P}_\ell^{(2)}$ de taille $v \times o$ et $\mathbf{P}_\ell^{(3)}$ de taille $o \times o$. Les blocs $\mathbf{P}^{(1)}$ et $\mathbf{P}^{(2)}$ sont recalculés à partir d'une graine par une fonction à base d'AES ; seul $\mathbf{P}^{(3)}$ est stocké dans la clé publique [12]. Soit $\mathbf{S}$ la matrice dont

les lignes sont $\mathbf{s}_0, \ldots, \mathbf{s}_{k-1}$. Tout se déduit de $\mathbf{S}\mathbf{P}_\ell\mathbf{S}^\top$: $\mathscr{P}(\mathbf{s}_i)_\ell$ est l'élément (i,i) de cette matrice, et $\mathscr{P}'(\mathbf{s}_i,\mathbf{s}_j)_\ell$ est la somme des éléments (i,j) et (j,i). *Ce sont donc les produits de matrices qui coûtent le plus cher*, d'où l'intérêt de les optimiser [12]. Le terme linéaire, lui, ne demande que k produits d'une matrice $m \times n$ par un vecteur.

1.3.5 Algorithmes (version simplifiée de la spécification)

KeyGen. Tirer `seed_sk` ; en dériver `seed_pk` et $\mathbf{O} \in \mathbb{F}_{16}^{v\times o}$; développer $\{\mathbf{P}_\ell^{(1)}, \mathbf{P}_\ell^{(2)}\}$ depuis `seed_pk` ; calculer $\mathbf{P}_\ell^{(3)} = \mathrm{Upper}\big(\mathbf{O}^\top(\mathbf{P}_\ell^{(1)}\mathbf{O} + \mathbf{P}_\ell^{(2)})\big)$ et $\mathbf{L}_\ell = (\mathbf{P}_\ell^{(1)} + \mathbf{P}_\ell^{(1)\top})\mathbf{O} + \mathbf{P}_\ell^{(2)}$. Clé publique : `seed_pk` et $\{\mathbf{P}_\ell^{(3)}\}$; clé secrète : `seed_sk`.

Sign(M) (algorithme 7 [1]). (1) Calculer sel puis SHAKE256(SHAKE256$(M)\|$sel) ; en tirer $\mathbf{t} \in \mathbb{F}_{16}^m$ et développer la matrice $\mathbf{\Lambda} = (\mathbf{\Lambda}_v \ \mathbf{\Lambda}_o)$ avec AES-128-CTR. (2) Pour ctr $= 0, \ldots, 255$: tirer les vinaigres $\mathbf{v}_0, \ldots, \mathbf{v}_{k-1} \in \mathbb{F}_{16}^v$ et une valeur $\mathbf{r}$. (3) Pour chaque i : $\mathbf{M}_i$ est la matrice $m \times o$ dont la ligne ℓ est $\mathbf{v}_i^\top \mathbf{L}_\ell$; puis $\mathbf{M}_0 \leftarrow \mathbf{M}_0 + \mathbf{\Lambda}_v\mathbf{O} + \mathbf{\Lambda}_o$. (4) Poser $\mathbf{y} \leftarrow \mathbf{t}$ et $\mathbf{A} \leftarrow \mathbf{0}$; pour chaque i : $\mathbf{y} \leftarrow \mathbf{y} - \mathbf{E}_{i0}\mathbf{\Lambda}_v\mathbf{v}_i$; pour chaque $j \geq i$: calculer $\mathbf{u}$ ($= \mathbf{v}_i^\top\mathbf{P}_\ell^{(1)}\mathbf{v}_i$ si $i = j$, sinon $\mathbf{v}_i^\top\mathbf{P}_\ell^{(1)}\mathbf{v}_j + \mathbf{v}_j^\top\mathbf{P}_\ell^{(1)}\mathbf{v}_i$) ; $\mathbf{y} \leftarrow \mathbf{y} - \mathbf{E}_{ij}\mathbf{u}$; ajouter $\mathbf{E}_{ij}\mathbf{M}_j$ au bloc de colonnes i de $\mathbf{A}$, et, si $i \neq j$, ajouter $\mathbf{E}_{ij}\mathbf{M}_i$ au bloc de colonnes j. (5) Résoudre $\mathbf{A}\mathbf{x} = \mathbf{y}$; si le système n'a pas de solution, passer au ctr suivant ; sinon choisir une solution à l'aide de $\mathbf{r}$. (6) Signature : $\mathbf{s}_i = (\mathbf{v}_i + \mathbf{O}\mathbf{x}_i,\ \mathbf{x}_i)$ pour chaque i, et sel.

Verify (algorithme 8 [1]). Redévelopper $\mathbf{P}^{(1)}, \mathbf{P}^{(2)}$; recalculer $\mathbf{t}$ et $\mathbf{\Lambda}$; poser $\mathbf{y} \leftarrow \mathbf{0}$; pour chaque i : $\mathbf{y} \leftarrow \mathbf{y} + \mathbf{E}_{i0}\mathbf{\Lambda}\mathbf{s}_i$, et pour chaque $j \geq i$: $\mathbf{y} \leftarrow \mathbf{y} + \mathbf{E}_{ij}\mathbf{u}$ avec $\mathbf{u}$ calculé comme à la signature mais avec la matrice complète $\mathbf{P}_\ell$; accepter si $\mathbf{y} = \mathbf{t}$.

La signature fait donc intervenir trois choses : des produits de matrices (pour $\mathbf{M}_i$ et $\mathbf{u}$, à partir de la clé publique développée $\mathbf{P}^{(1)}$, $\mathbf{P}^{(2)}$), la résolution d'un système linéaire par élimination de Gauss en temps constant, et des reprises quand le système n'a pas de solution. D'après [12], ce sont le calcul des évaluations partielles de $\mathscr{P}$ et l'élimination de Gauss qui coûtent le plus ; l'assemblage de $\mathbf{A}$ et de $\mathbf{y}$ ne coûte presque rien. Cette présentation est une version simplifiée ; la spécification [1] fait foi.

1.3.6 Exemple complet : génération des clés, signature, vérification

Cet exemple est une *illustration simplifiée* : les paramètres sont minuscules et la matrice Z est celle de la section précédente, plus simple que celle de la spécification. Il suit les étapes des algorithmes 7 et 8, et chaque résultat a été recalculé (par un programme indépendant, qui évalue directement la clé publique). Paramètres : $q = 16$, $v = 2$, $o = 1$, $n = 3$, $m = 2$, $k = 2$ (donc $k \cdot o = m$).

Génération des clés. Le secret est $\mathbf{O} = (\mathrm{b}, 8)^\top$. Les blocs développés à partir de la graine sont

$$\mathbf{P}_1^{(1)} = \begin{pmatrix} 8 & 9 \\ 0 & 6 \end{pmatrix},\ \mathbf{P}_1^{(2)} = \begin{pmatrix} 2 \\ 2 \end{pmatrix};\qquad \mathbf{P}_2^{(1)} = \begin{pmatrix} 8 & \mathrm{a} \\ 0 & 8 \end{pmatrix},\ \mathbf{P}_2^{(2)} = \begin{pmatrix} \mathrm{b} \\ \mathrm{c} \end{pmatrix}.$$

Le bloc $\mathbf{P}_\ell^{(3)} = \mathrm{Upper}\big(\mathbf{O}^\top(\mathbf{P}_\ell^{(1)}\mathbf{O} + \mathbf{P}_\ell^{(2)})\big)$ se calcule avec le secret : $\mathbf{P}_1^{(3)} = (6)$ et $\mathbf{P}_2^{(3)} = (\mathrm{e})$. On calcule aussi $\mathbf{L}_1 = (6, \mathrm{e})^\top$ et $\mathbf{L}_2 = (4, \mathrm{e})^\top$. La clé publique contient la graine et les deux blocs $\mathbf{P}^{(3)}$.

Données du message. On suppose que le haché salé donne $\mathbf{t} = (\mathrm{f}, 9)$ et, pour la matrice du terme linéaire,

$$\mathbf{\Lambda} = \begin{pmatrix} 3 & 1 & 2 \\ 5 & 7 & 1 \end{pmatrix}, \qquad \mathbf{\Lambda}_v = \begin{pmatrix} 3 & 1 \\ 5 & 7 \end{pmatrix}, \quad \mathbf{\Lambda}_o = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$$

Signature. On tire les vinaigres $\mathbf{v}_0 = (2,3)$ et $\mathbf{v}_1 = (3,\mathrm{a})$.

1. *Matrices* $\mathbf{M}_i$. Les lignes de $\mathbf{M}_i$ sont $\mathbf{v}_i^\top \mathbf{L}_\ell$: $\mathbf{M}_0 = (\mathrm{d}, 9)^\top$ et $\mathbf{M}_1 = (\mathrm{c}, \mathrm{a})^\top$. Le terme linéaire s'ajoute à $\mathbf{M}_0$: $\mathbf{\Lambda}_v \mathbf{O} + \mathbf{\Lambda}_o = (4, \mathrm{d})^\top$, donc $\mathbf{M}_0 \leftarrow (\mathrm{d}+4,\ 9+\mathrm{d}) = (9,4)$.
2. *Second membre.* Les produits $\mathbf{\Lambda}_v \mathbf{v}_i$ valent $(5,3)$ pour $i = 0$ et (f, c) pour $i = 1$; avec $\mathbf{E}_{00} = \mathbf{I}$ et $\mathbf{E}_{10} = z$, le terme linéaire retire $(5,3) + z \cdot (\mathrm{f}, \mathrm{c}) = (5,3) + (\mathrm{a}, 3) = (\mathrm{f}, 0)$. Les nombres $\mathbf{u}$ valent $\mathbf{u}_{00} = (8,1)$, $\mathbf{u}_{01} = (1,7)$ et $\mathbf{u}_{11} = (4,9)$; leur contribution est $\mathbf{u}_{00} + z\,\mathbf{u}_{01} + z^2 \mathbf{u}_{11} = (8,1) + (\mathrm{d}, 6) + (2,9) = (7, \mathrm{e})$. Donc $\mathbf{y} = \mathbf{t} + (\mathrm{f}, 0) + (7, \mathrm{e}) = (7,7)$.
3. *Matrice du système.* Le bloc de colonnes 0 est $\mathbf{M}_0 + z\,\mathbf{M}_1 = (9,4) + (\mathrm{f}, 6) = (6,2)$; le bloc 1 est $z\,\mathbf{M}_0 + z^2 \mathbf{M}_1 = (6, \mathrm{d}) + (5,9) = (3,4)$. Donc $\mathbf{A} = \begin{pmatrix} 6 & 3 \\ 2 & 4 \end{pmatrix}$, de déterminant $\mathrm{d} \neq 0$.
4. *Résolution.* $\mathbf{A}\mathbf{x} = \mathbf{y}$ a la solution unique $\mathbf{x} = (x_0, x_1) = (\mathrm{b}, 9)$. Contrôle : $6 \cdot \mathrm{b} + 3 \cdot 9 = \mathrm{f} + 8 = 7$ et $2 \cdot \mathrm{b} + 4 \cdot 9 = 5 + 2 = 7$.
5. *Signature.* $\mathbf{s}_i = (\mathbf{v}_i + \mathbf{O} x_i,\ x_i)$, soit

$$\mathbf{s}_0 = (\mathrm{b}, 4, \mathrm{b}), \qquad \mathbf{s}_1 = (\mathrm{f}, \mathrm{e}, 9).$$

Vérification. On utilise seulement la clé publique complète (avec $\mathbf{P}^{(3)}$), en évaluant directement : $\mathscr{P}(\mathbf{s}_0) = (\mathrm{e}, \mathrm{d})$, $\mathscr{P}(\mathbf{s}_1) = (2, \mathrm{c})$ et $\mathscr{P}'(\mathbf{s}_0, \mathbf{s}_1) = (3,8)$. Alors

$$\mathscr{P}^\star(\mathbf{S}) = (\mathrm{e}, \mathrm{d}) + z \cdot (3,8) + z^2 \cdot (2, \mathrm{c}) = (\mathrm{e}, \mathrm{d}) + (\mathrm{c}, \mathrm{b}) + (9,4) = (\mathrm{b}, 2).$$

Pour le terme linéaire, $\mathbf{\Lambda}\mathbf{s}_0 = (\mathrm{f}, 5)$ et $\mathbf{\Lambda}\mathbf{s}_1 = (\mathrm{d}, 3)$, donc

$$L(\mathbf{\Lambda}, \mathbf{S}) = (\mathrm{f}, 5) + z \cdot (\mathrm{d}, 3) = (\mathrm{f}, 5) + (\mathrm{b}, \mathrm{e}) = (4, \mathrm{b}).$$

On obtient $\mathrm{P}^\star(\mathbf{S}) + L(\mathbf{\Lambda}, \mathbf{S}) = (\mathrm{b}, 2) + (4, \mathrm{b}) = (\mathrm{f}, 9) = \mathbf{t}$: la signature est acceptée. Sans le terme linéaire, on trouverait $(\mathrm{b}, 2) \neq \mathbf{t}$: c'est lui qui rend l'égalité vraie. Si l'on change un seul élément de $\mathbf{s}_0$ (par exemple $\mathrm{b} \to \mathrm{a}$ au début), le résultat devient $(\mathrm{c}, \mathrm{f}) \neq \mathbf{t}$ et la signature est refusée.

1.4 Du schéma au programme

Jusqu'ici, MAYO était une suite de formules. Cette section montre comment ces formules deviennent un programme en langage C : le déroulement d'une tentative de signature (section 1.4.1), la façon dont les nombres de $\mathbb{F}_{16}$ sont rangés en mémoire (section 1.4.2), et le calcul des produits avec l'instruction `vpshufb` (section 1.4.3). Un mot sur le vocabulaire : *AVX2* est un ensemble d'instructions des processeurs Intel et AMD qui traitent 256 bits (32 octets) à la fois ; un *octet* est un groupe de 8 bits ; un *quartet* est un groupe de 4 bits, c'est-à-dire un élément de $\mathbb{F}_{16}$.

1.4.1 La signature dans le code de référence

1.4.1.1 Le déroulement d'une tentative

Dans le code de référence (annexe A), la fonction `mayo_sign_signature` calcule le sel et $\mathbf{t}$, puis fait une boucle d'au plus 256 tentatives (un octet sert de compteur). Chaque tentative enchaîne les étapes suivantes (figure 1.1) :

1. tirer et décoder les vinaigres $\mathbf{V}$ (avec SHAKE256) ;
2. `m_calculate_M_and_VPV` : calculer les matrices $\mathbf{M}_\ell$ et tous les produits $\mathbf{v}_i^\top \mathbf{P}^{(1)} \mathbf{v}_j$ (c'est ici que se trouvent les noyaux étudiés) ;
3. `compute_rhs` (second membre $\mathbf{y}$) puis `compute_A` (matrice $\mathbf{A}$) ;
4. `sample_solution` : résoudre $\mathbf{Ax} = \mathbf{y}$; en cas d'échec, nouvelle tentative ;
5. `m_compute_s` : assembler la signature $\mathbf{s}_i = [\,\mathbf{v}_i + \mathbf{O}\mathbf{x}_i \mid \mathbf{x}_i\,]$.

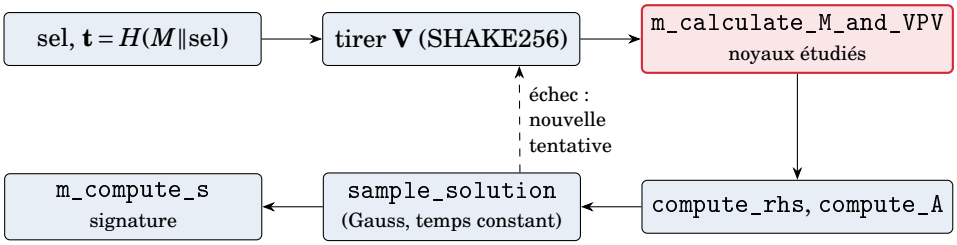


FIGURE 1.1 : Une tentative de signature dans le code de référence. Les noyaux optimisés se trouvent dans l'étape en rouge, répétée à chaque tentative.

1.4.1.2 Où sont les noyaux étudiés

Dans `m_calculate_M_and_VPV` (chemin à L implicite, celui où les deux noyaux sont utilisés), les opérations se font dans cet ordre : construction des tables de multiplication de $\mathbf{V}$ (`mayo_V_multabs`), puis :

- `P1_times_Vt` : accumule $\mathbf{P}^{(1)} \cdot \mathbf{V}^\top$ dans un tampon ; `Vt_times_Pv` en déduit $\mathbf{V}\mathbf{P}^{(1)}\mathbf{V}^\top$, c'est-à-dire tous les $\mathbf{v}_i^\top \mathbf{P}^{(1)} \mathbf{v}_j$;
- `P1t_times_Vt` : accumule la contribution transposée $\mathbf{P}^{(1)\top}\mathbf{V}^\top$ sur le même tampon, qui devient $(\mathbf{P}^{(1)} + \mathbf{P}^{(1)\top})\mathbf{V}^\top$ (la diagonale s'annule en caractéristique 2) ; le calcul de $\mathbf{M}_\ell$ réutilise ce produit, puis `Ut_times_O` et `Vt_times_L` complètent M_ℓ.

Les noyaux `P1_times_Vt` et `P1t_times_Vt` calculent donc des produits de la *partie vinaigre de la clé publique* $\mathbf{P}^{(1)}$ par les multiples de $\mathbf{V}$. Ce sont les seuls que notre travail optimise (variante A2), pour MAYO-1 et MAYO-5.

1.4.1.3 Rôle de $\mathbf{P}_1$ et paramètres

$\mathbf{P}_1$ est une matrice triangulaire supérieure dont chaque case est un m-vecteur de $\mathbb{F}_{16}^m$. Elle est rangée à plat, une entrée après l'autre : l'entrée numéro e commence à la position $e \cdot b$. Chaque noyau calcule, pour chaque entrée de $\mathbf{P}_1$ et chaque vecteur de V, le produit d'un nombre de $\mathbb{F}_{16}$ par le m-vecteur de l'entrée, et l'ajoute à un accumulateur de L « limbes ». Les bornes des boucles (V, K) et les adresses de lecture $(e \cdot b)$ ne dépendent que de paramètres publics. Le tableau 1.3 donne les valeurs utiles.

TABLEAU 1.3 : Paramètres des noyaux étudiés. m : longueur du m-vecteur ; $b = m/2$: octets par m-vecteur ; L : limbes de 64 bits d'un accumulateur ; K : nombre de vecteurs de V ; V : dimension de la partie vinaigre ; $N_{\text{ent}} = V(V+1)/2$: nombre d'entrées de $\mathbf{P}_1$.

Paramètre	m	b	L	K	V	N_{ent}	$m \bmod 64$
MAYO-1	80	40	5	10	80	3 240	16
MAYO-2	64	32	4	–	73	–	0
MAYO-3	108	54	–	–	–	–	44
MAYO-5	142	71	9	12	142	10 153	14

1.4.2 Représentation des données

1.4.2.1 Le corps $\mathbb{F}_{16}$ et les m-vecteurs

$\mathbb{F}_{16}$ est construit comme $\mathbb{F}_2[x]/(x^4 + x + 1)$ (commentaire du code, `simple_arithmetic.h`). Un élément tient dans un *quartet* (4 bits) et l'addition est le OU exclusif. L'unité de calcul de MAYO est le *m-vecteur* : la liste des éléments qui occupent la même case (i, j) dans les m matrices P_ℓ. On calcule ainsi sur les m composantes en parallèle, ce qui convient bien à la vectorisation [12].

1.4.2.2 Empaquetage en quartets

Un m-vecteur occupe $b = m/2$ octets : deux éléments par octet, l'élément d'indice pair dans les 4 bits de poids faible. Les accumulateurs sont découpés en *limbes* de 64 bits, soit 16 éléments par limbe. Il faut donc $L = \lceil m/16 \rceil$ limbes, avec éventuellement des quartets de bourrage (remplissage) dans le dernier limbe ($m = 142 : L = 9$, 144 positions, dont deux de bourrage). Cette représentation par quartets vient de [12], qui l'a préférée à la représentation « bitsliced » (par plans de bits) de la première version. Elle permet d'utiliser des instructions qui permutent les octets pour multiplier, au prix d'un changement du format de la clé.

1.4.3 Arithmétique $\mathbb{F}_{16}$ par `vpshufb`

1.4.3.1 Rappel : le registre AVX2 et `vpshufb`

Un registre AVX2 `ymm` contient 256 bits, soit deux *voies* de 128 bits (`xmm` désigne la moitié basse). La plupart des instructions, dont `vpshufb`, travaillent séparément sur chaque voie. `vpshufb`

reçoit une table de 16 octets (une par voie) et un registre d'index. Pour chaque octet d'index, elle écrit l'élément de la table désigné par les 4 bits de poids faible de l'index, ou 0 si le bit de poids fort de l'index vaut 1. Un registre `ymm` fait donc 32 recherches avec une seule instruction.

1.4.3.2 Principe

Avec des éléments de 4 bits, multiplier un vecteur par un nombre c revient à chercher chaque élément x dans une table de 16 cases qui contient $c \cdot x$. L'instruction `vpshufb` (`_mm256_shuffle_epi8`) fait cette recherche pour chaque octet d'un registre d'index. C'est la méthode des « quatre Russes » [17] : on remplace des multiplications par des lectures dans une table. Pour calculer $A \cdot B$, on découpe A en bandes de t colonnes et B en bandes de t lignes. Pour chaque bande de B, on prépare toutes les combinaisons de ses lignes (16^t entrées), puis chaque ligne de A sert d'indice dans cette table. Ici la table tient dans un registre, donc $t = 1$ (voir aussi [18] pour $\mathbb{F}_2$). Les tables dépendent de V (qui est secret), mais elles ne servent que de données pour `vpshufb` : elles ne servent jamais d'adresse mémoire.

Exemple : multiplier par 8 avec une table. La table T de $c = 8$ contient $T[x] = 8 \cdot x$:

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
$8 \cdot x$	0	8	3	b	6	e	5	d	c	4	f	7	a	2	9	1

Pour le vecteur $(1, 2, 3, \mathrm{b})$, chaque élément sert d'indice : $T[1] = 8$, $T[2] = 3$, $T[3] = \mathrm{b}$, $T[\mathrm{b}] = 7$. Donc $8 \cdot (1, 2, 3, \mathrm{b}) = (8, 3, \mathrm{b}, 7)$. Vérification pour $T[3]$: $8 \cdot 3 = x^3(x+1) = x^4 + x^3 = (x+1) + x^3 = \mathrm{b}$. L'instruction `vpshufb` lit ces 16 cases pour 32 éléments à la fois.

1.4.3.3 Deux produits par recherche

Pour une paire de nombres (c_{2p}, c_{2p+1}), la table $T_p[x] = (c_{2p}x) \oplus \big((c_{2p+1}x) \ll 4\big)$ donne *les deux* produits dans un seul octet. Le code range T_p dans un registre de 256 bits dont les deux voies sont identiques (lemme 2.7). Un registre de données de 32 octets contient 64 éléments. Il faut séparer les quartets impairs et pairs (ET avec `0x0f`, puis décalage de 4 bits), donc deux `vpshufb` par registre et par paire. Les accumulateurs « impairs » et « pairs » restent séparés jusqu'à la fin de la ligne, où on les remet ensemble. Cette technique est décrite dans l'article *Nibbling MAYO* [12].

Exemple : deux produits par recherche. Prenons la paire $(c_0, c_1) = (2, 3)$. La table $T_p[x]$ range $2 \cdot x$ dans le quartet bas et $3 \cdot x$ dans le quartet haut. Pour $x = 5$: $2 \cdot 5 = \mathrm{a}$ et $3 \cdot 5 = \mathrm{f}$, donc l'octet `fa`. Pour $x = \mathrm{b}$: $2 \cdot \mathrm{b} = 5$ et $3 \cdot \mathrm{b} = \mathrm{e}$, donc l'octet `e5`. Soit maintenant l'octet de données `b5`, qui contient les deux éléments 5 (quartet bas) et b (quartet haut). Une première recherche, avec l'indice 5, donne `fa` ; une seconde, avec l'indice b (après décalage de 4 bits et masquage), donne `e5`. Deux instructions `vpshufb` ont donc fourni les 4 produits $(2 \cdot 5,\ 3 \cdot 5,\ 2 \cdot \mathrm{b},\ 3 \cdot \mathrm{b})$.

1.4.3.4 Recouvrement en fin de vecteur

Pour $m \leq 64$, $64 < m \leq 128$ et $128 < m \leq 192$, le code officiel utilise respectivement 1, 2 et 3 chargements de 32 octets. Le dernier est *reculé* pour se terminer à l'octet b (adresse `vec + M_VEC_LIMBS_MAX - 4` limbes). Les octets communs sont donc traités deux fois, et les écritures finales se recouvrent avec les mêmes valeurs. Pour MAYO-5, on a les chargements $[0,32)$, $[32,64)$ et $[40,72)$: 192 éléments sont calculés pour 142 utiles, et le dernier chargement lit un octet de trop après l'entrée de $\mathbf{P}_1$ (figure 1.2).

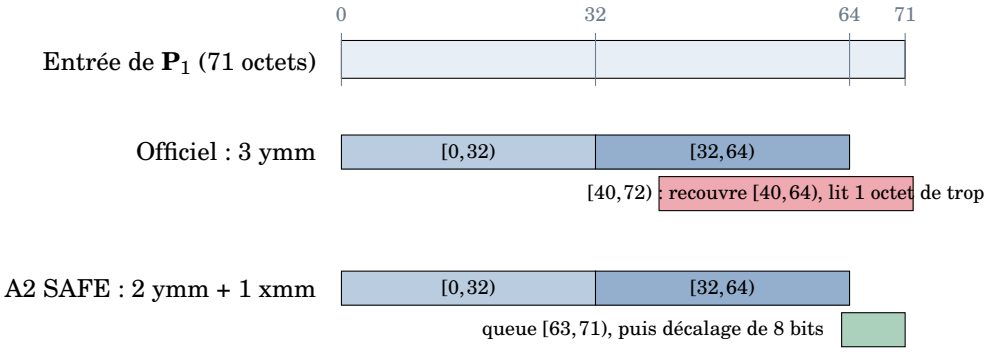


FIGURE 1.2 : Chargements pour une entrée de $\mathbf{P}_1$ de MAYO-5 (octets). Le noyau officiel recouvre l'intervalle $[40,64)$ et dépasse l'entrée d'un octet ; A2 SAFE lit exactement $[0,71)$.

Lien avec notre étude. la fin des vecteurs est traitée par recouvrement : c'est le point de départ de la question centrale.

1.5 Outils pour comprendre les analyses

Les chapitres suivants utilisent quatre outils : un compteur de temps, une méthode de comparaison par paires, un compilateur, et la lecture de l'assembleur. Cette section les explique sans supposer de connaissance préalable. Elle précise aussi ce que chaque outil permet de conclure, et ce qu'il ne permet pas de conclure.

1.5.1 Mesurer le temps : le compteur TSC

Le processeur contient un *compteur de ticks*, appelé TSC (*Time Stamp Counter*). On lit ce compteur avant et après le code à mesurer (instruction `rdtscp`). Le temps mesuré est la différence des deux lectures, exprimée en *ticks*.

Attention : un tick TSC n'est pas un cycle du processeur. Le compteur avance à vitesse constante, même quand le processeur ralentit ou accélère. On ne peut donc pas dire « tant de cycles ». On peut seulement comparer deux versions d'un programme *dans les mêmes conditions*, sur la même machine.

1.5.2 Comparer deux versions : mesure par paires

Pour savoir si la version A2 est plus rapide que la version officielle, on ne peut pas mesurer l'une un matin et l'autre le soir : la machine change (température, autres programmes, fréquence). On utilise donc une *mesure par paires* : à chaque tour, on exécute les deux versions avec les mêmes données, dans un ordre *tiré au hasard*. L'ordre au hasard évite qu'une version soit toujours favorisée (par exemple parce que l'autre « réchauffe » la mémoire cache).

Chaque tour donne un *rapport* temps A2/temps officiel, et le *gain* est 1 – rapport. Un gain de 10 % veut dire que A2 prend 90 % du temps de la version officielle.

Un même *processus* (une exécution du programme de mesure) enchaîne des milliers de tours (3 001). Ces tours ne sont pas indépendants : ils partagent le même état de la machine. Si on les comptait comme des mesures indépendantes, on croirait être beaucoup plus précis qu'on ne l'est. L'*unité statistique* est donc le processus, pas le tour. On calcule la médiane des rapports par processus, puis un intervalle de confiance à 95 % par *bootstrap* : on retire au hasard, avec remise, des processus parmi ceux qu'on a mesurés, des milliers de fois, et l'on regarde de combien la médiane varie.

1.5.3 Le compilateur et ses passes

Le code est écrit en langage C. Le processeur ne comprend pas le C : un programme appelé *compilateur* (ici GCC, *GNU Compiler Collection*) le transforme en *assembleur*, c'est-à-dire en une suite d'instructions du processeur. Pour que le programme aille vite, GCC applique des « passes » d'optimisation. Trois d'entre elles reviendront dans notre travail :

- **SRA** (*Scalar Replacement of Aggregates*) : découper un tableau ou une structure en variables séparées, gardées dans des registres du processeur plutôt qu'en mémoire ;
- **PRE** (*Partial Redundancy Elimination*) : éviter de refaire un calcul déjà fait ;
- **DSE** (*Dead Store Elimination*) : supprimer une écriture en mémoire qui ne sert à rien.

Le même code C peut donner des assembleurs différents selon la version du compilateur, ses options et le contexte. Un changement dans le C (par exemple une modification d'A2) peut donc changer le code produit *ailleurs* que là où on l'a fait. **Règle de notre travail :** on n'attribue jamais une différence à une passe précise (par exemple SRA) sans avoir d'abord écarté les autres changements.

1.5.4 Lire l'assembleur sans exécuter

On peut recompiler le code, le désassembler (par exemple avec `objdump`) et *compter* les instructions : combien de `vpshufb`, combien d'écritures de 32 octets, combien de rechargements depuis la pile (zone de mémoire où le programme met temporairement ses valeurs). C'est l'*analyse statique*, notée E1 puis E1b dans la suite : aucune mesure de temps n'est lancée.

- **Ce qu'on peut conclure :** le nombre exact d'instructions d'un type donné, et ce qui change entre deux versions du code ou entre deux options du compilateur.
- **Ce qu'on ne peut pas conclure :** que le programme est plus rapide ou plus lent. Le temps dépend aussi de la machine (débit des unités de calcul, mémoire cache, dépendances entre instructions).

Pour parler de vitesse, il faut mesurer (section 1.5.2) ; pour parler du code, il faut le lire. Les deux se complètent, mais ne se remplacent pas.

1.5.5 Comment sont présentés les résultats

Pour que le lecteur sache toujours ce qui est établi, chaque résultat porte une étiquette : **[F]** *fait* (lu dans le code produit ou dans des mesures publiées), **[I]** *déduction* tirée des faits, **[H]** *hypothèse* non testée. Pour la sûreté du code, **[C]** veut dire « justifiée par le code » et **[E]** « vérifiée par des tests ».

TABLEAU 1.4 : Quelques abréviations utilisées dans les analyses.

Abréviation	Sens
KAT	*Known-Answer Test* : test de « réponses connues » ; on compare la sortie du programme à des valeurs de référence officielles
TSC	*Time Stamp Counter* : compteur de ticks du processeur
E1, E1b	analyses statiques du code produit par le compilateur (E1b : avec une option du compilateur en plus)
SAFE	variante d'A2 qui ne lit rien en dehors de l'entrée
A2	l'optimisation étudiée : traiter la fin du vecteur avec un seul registre de 128 bits

1.5.6 Panorama des analyses menées

Pour s'y retrouver dans la suite, voici les analyses et ce qu'elles établissent (détails aux chapitres 2 et 3) :

- **Comptage** (théorique, dans un modèle précis) : au moins $\lceil m/32 \rceil$ `vpshufb` par paire de nombres et par m-vecteur ; A2 atteint cette limite pour $m \in \{80, 142\}$;
- **Correction et sûreté** : tests de réponses connues (KAT), tests unitaires, limites de lecture et d'écriture, propriétés des tables, indépendance vis-à-vis du bourrage ;
- **Mesures de temps** sur une machine physique (compteur TSC, unité statistique : le processus) : gains sur les noyaux seuls, puis sur `mayo_sign`, avec contrôle de l'effet d'ordre et des sondes de mesure ;
- **Analyse du code produit sans l'exécuter** (E1, E1b) : on compte les instructions et les écritures dans les boucles produites par GCC 16.2.0, et on désactive des passes du compilateur une à une ;

- **Chaque résultat porte une étiquette** : [F] fait établi, [I] déduction, [H] hypothèse non testée.

Lien avec notre étude. ces outils ne disent pas la même chose (le temps d'un côté, le code de l'autre) ; les chapitres suivants les utilisent ensemble, en distinguant toujours ce qui est établi de ce qui est supposé.

1.6 Revue de littérature

1.6.1 Implémentations existantes

Référence C. Arithmétique ordinaire, sans `vpshufb`.

AVX2 officiel (MAYO-C). Fichier `src/AVX2/shuffle_arithmetic.h` : macros `MULTIPLY_ACCUMULATE_M_VEC` et `DISENTANGLE_AND_ACCUMULATE_M_VECS`, avec le recouvrement décrit plus haut. C'est la base de ce travail : version amont `74de637` [20].

***Nibbling MAYO* [12].** Implémentation AVX2 et Cortex-M4 optimisée, avec la représentation en quartets. Au paragraphe 5.1, un `vpshufb` donne 64 multiplications, les accumulateurs impairs et pairs sont séparés et, pour $m = 96$ (MAYO-3 de l'époque), un recouvrement « duplique 32 opérations ». Le débit est d'un `vpshufb` par cycle sur Skylake et de deux sur Ice Lake. Nous avons lu l'article en entier. Il utilise les paramètres du tour 1 ($m = 64$ pour MAYO-1 et MAYO-2, 96 pour MAYO-3, 128 pour MAYO-5) : MAYO-1 et MAYO-5 n'y ont donc *pas* de fin de vecteur à traiter. Les mesures ont été faites sur Skylake (Xeon E3-1245 v5) et Ice Lake (Xeon Gold 6338), avec `clang-14`, Turbo Boost désactivé, médiane de 10 000 répétitions. Les tables de multiplication sont construites par la méthode de [19], appliquée deux fois puis entrelacée. L'approche par `vpshufb` se distingue de [19] en cherchant *deux* produits par recherche. L'article évoque le recouvrement pour m non multiple de 64 (cas MAYO-3), mais sans traitement spécial de la fin du vecteur. Le code publié avec l'article (`shuffle_arithmetic_\{64,96,128\}.h`) charge aux positions de limbes 0 et 2 (octets $[0,32)$ et $[16,48)$) et utilise quatre `vpshufb` par paire pour $m = 96$.

Version GFNI. Elle utilise l'instruction `vgf2p8affineqb` et lit sur toute la largeur dans une marge de sécurité (`MAYO_MVEC_SLACK`) ; elle est hors du cadre de notre travail.

1.6.2 Fuites par canaux auxiliaires et temps constant

Les attaques par mesure du temps [15] et par le cache [16] exigent que les adresses mémoire et les branches du programme ne dépendent pas de données secrètes. Dans les noyaux, les tables de multiplication calculées à partir du vinaigre ne sont que des données de `vpshufb` : elles ne

servent ni d'adresses ni de conditions. A2 doit garder cette propriété ; la section 2.2 dit où en est la preuve.

Lien avec notre étude. les implémentations existantes montrent où se situe le recouvrement et ce qui a déjà été étudié.

1.7 Évaluation et positionnement

Le tableau 1.5 sépare ce qui était connu, ce qui est venu après l'article, et ce qu'A2 fait de différent. Il se fonde sur les sources consultées : le paragraphe 5.1 de l'article (lu en entier), les parties AVX2 du code, et tout l'historique du fichier `shuffle_arithmetic.h` dans le dépôt officiel (versions `3c9be9e`, `d81b689`, `11ff0f0`, `4b7cd94`, `fef05b5` et la dernière version de `main`, `74de637`).

TABLEAU 1.5 : Situation d'A2 par rapport aux sources consultées.

Catégorie	Élément	Source
Connu avant A2	`vpshufb` avec $t = 1$, deux produits par octet ; séparation impair/pair ; remise en ordre à la fin	[12]
Connu avant A2	Recouvrement de registres quand m n'est pas multiple de 64 (MAYO-3, $m = 96$)	[12]
Venu après l'article	Généralisation par chargements reculés pour tout $m \leq 192$	code officiel (tour 2)
Différent dans A2	Fin de vecteur ≤ 16 éléments regroupée dans *un* registre xmm (quartets [bas|haut]), avec *un* `vpshufb` xmm sur la voie basse de la table	ce travail
Différent dans A2	Variante SAFE pour $m = 142$: on charge $[63, 71)$ puis on décale, sans aucune lecture en dehors de l'entrée	ce travail
Différent dans A2	Propriétés vérifiées pour la fin du vecteur (section 2.2)	ce travail
Non revendiqué	Recouvrement, séparation impair/pair, remise en ordre tardive, schéma $t = 1$, nouveauté absolue	–

Ce qui manque dans l'existant. À partir des sources consultées, on peut résumer les limites ainsi :

- **Des calculs en double.** Quand m n'est pas un multiple de 64, le recouvrement refait une partie du travail (37,5 % des éléments pour MAYO-1, 26 % pour MAYO-5).
- **Une lecture en trop.** Pour MAYO-5, le dernier chargement du noyau officiel lit un octet après l'entrée de $\mathbf{P}_1$ (deux pour MAYO-3).

- **Une limite inconnue.** Nous n'avons pas trouvé, dans les sources consultées, de calcul du nombre minimal de `vpshufb` avec des tables de 16 octets.
- **Un effet sur la signature complète non étudié.** Un gain sur un noyau seul ne se retrouve pas forcément sur toute la signature, et nous n'avons pas trouvé d'étude de cet écart.

Ce que nous proposons, et pourquoi. A2 traite la fin du vecteur avec un seul registre de 128 bits. Nous l'avons retenue pour trois raisons : (i) elle s'attaque exactement au calcul en double ; (ii) sa variante SAFE ne lit rien en dehors de l'entrée ; (iii) son gain en nombre d'instructions peut être démontré par un comptage, puis testé par des mesures. Ses limites, et ce qu'elle ne traite pas, sont discutées aux chapitres 2 et 3.

Nous n'avons trouvé aucun `vpshufb` xmm sur une fin de vecteur, ni dans le code publié avec l'article, ni dans l'historique officiel jusqu'à la dernière version de `main`. Les `__m128i` qui apparaissent à partir de `fef05b5` (lignes 548–606) servent à autre chose : transposer des m-vecteurs par décalage de bits, et charger une table de 16 octets répétée sur 256 bits. **Limite :** nous n'avons lu l'article en entier que pour sa section 5.1 et ses parties AVX2 utiles. Ne pas avoir trouvé la technique dans ces sources ne prouve pas qu'elle n'existe pas ailleurs.

Ce qu'il faut retenir de ce chapitre

1. MAYO signe en tirant des « vinaigres » au hasard, puis en résolvant un système linéaire ; le « fouettage » (k copies) permet une clé petite.
2. Le coût d'une signature est surtout celui de **produits de matrices** sur $\mathbb{F}_{16}$, refaits à chaque tentative de signature.
3. Dans le code AVX2, on calcule avec `vpshufb` : des tables de 16 entrées, deux produits par recherche, 64 multiplications par instruction.
4. Quand m n'est pas un multiple de 64, la fin du vecteur est traitée par un **recouvrement** : des éléments sont calculés deux fois. A2 cherche à l'éviter (chapitre 2).
5. Pour juger d'un gain, on dispose de deux familles d'outils qui ne disent pas la même chose : des **mesures de temps** (mesure par paires, unité statistique = processus) et la **lecture du code produit** (qui ne donne pas la vitesse).

Suite : le chapitre 2 formalise le comptage, décrit A2 et le protocole d'expérience ; le chapitre 3 donne les résultats.

CHAPITRE 2

MÉTHODOLOGIE

Ce chapitre répond à QR1 et à QR2, puis pose le protocole qui permettra de répondre à QR3 et à QR4. Il présente d'abord la conception d'A2 et son modèle de comptage (section 2.1), ensuite les propriétés qui assurent sa correction et sa sûreté (section 2.2), enfin le protocole d'expérience et d'analyse du code produit (section 2.3).

Comment lire ce chapitre. Les énoncés mathématiques valent *dans un modèle* précis ; ils ne disent rien de la vitesse. Les propriétés de sûreté portent une étiquette : **[C]** (justifiée par le code), **[E]** (vérifiée par des tests), **[H]** (hypothèse sur la machine ou le compilateur). Les outils de mesure et d'analyse ont été présentés à la section 1.5.

2.1 Conception d'A2 : comptage, mise en œuvre, hypothèses et limites

Cette section répond à QR1 et décrit la mise en œuvre (QR2). Les propositions 2.3 à 2.5 sont des énoncés mathématiques *dans un modèle* ; on n'en tire aucune conclusion sur la vitesse (section 2.1.4).

2.1.1 Modèle de comptage

Définition 2.1 (Données)**.** Un *m-vec* est un vecteur $v \in \mathbb{F}_{16}^{m}$, avec m pair, rangé sur $b = m/2$ octets : v_{2i} est le quartet bas de l'octet i et v_{2i+1} le quartet haut. Les K nombres $c_0, \dots, c_{K-1}$ sont regroupés en $P = \lceil K/2 \rceil$ paires (complétées par 0 si K est impair). Pour chaque $k < K$, on calcule $\mathrm{acc}_k \leftarrow \mathrm{acc}_k \oplus c_k v$.

Pour la paire p, on pose $T_p[x] = (c_{2p}x) \oplus ((c_{2p+1}x) \ll 4)$. Un `vpshufb` de largeur $W \in \{16, 32\}$ octets fait W recherches indépendantes. Chacune donne, pour *un* élément de v, les deux produits de la paire.

Définition 2.2 (Classe $\mathscr{C}$)**.** Algorithmes qui (i) obtiennent chaque produit $c_k v_j$ par une recherche dans une table de 16 octets ; (ii) donnent à chaque recherche un quartet seul dans un octet (bit 7 nul) ; (iii) ne font pas deux recherches pour le même couple (élément, paire), sauf pour le recouvrement entre registres ; (iv) utilisent librement XOR, ET, décalages, permutations et chargements. Sont *hors* de $\mathscr{C}$: GFNI, les représentations *bitsliced*, les tables de 256 octets.

La quantité comptée est $N(\text{algo}, m)$: le nombre de `vpshufb` exécutés *par paire de nombres et par m-vec*.

2.1.2 Énoncés

Proposition 2.3 (Limite)**.** *Tout algorithme de $\mathscr{C}$ vérifie $N \geq \lceil m/32 \rceil$.*

Démonstration. Chaque élément v_j ($j < m$) demande au moins une recherche par paire, et une instruction en fait au plus $W \leq 32$. Donc $32N \geq m$. ∎

Proposition 2.4 (Schéma officiel avec recouvrement)**.** *Le schéma qui couvre les b octets avec $\lceil b/32 \rceil$ chargements de 32 octets (le dernier est reculé) et qui traite chaque registre avec deux `vpshufb` (impairs, pairs) a $N_{\text{off}}(m) = 2\lceil m/64 \rceil$.*

Démonstration. Il y a $\lceil b/32 \rceil = \lceil m/64 \rceil$ chargements. Chacun donne 32 indices impairs et 32 indices pairs, donc deux `vpshufb`. ■

Proposition 2.5 (Mise en œuvre de la limite si $0 < r \leq 32$)**.** *Écrivons $m = 64q + r$ avec $0 \leq r < 64$, et $t = r/2$ le nombre d'octets de la fin du vecteur. Si $0 < r \leq 32$, l'algorithme suivant est dans $\mathscr{C}$ et a $N = 2q + 1 = \lceil m/32 \rceil$:*

1. *pour $g < q$: charger l'intervalle $[32g, 32g + 32)$ et le traiter avec deux `vpshufb` ymm ;*
2. *pour la fin du vecteur : charger les t octets de $[32q, 32q + t)$, construire X = [quartets bas des t octets | quartets hauts des t octets] (cela met les $r \leq 32$ indices dans un seul registre), puis appliquer* un seul *`vpshufb` ;*
3. *remettre en ordre les sorties de la fin du vecteur (même formule que pour un registre complet, limitée aux r positions).*

Démonstration. On compte $2q + 1$ instructions, et $\lceil m/32 \rceil = 2q + \lceil r/32 \rceil = 2q + 1$ quand $0 < r \leq 32$. ■

Conditions de mise en œuvre (le code doit les respecter ; le modèle ne les garantit pas) : (a) le chargement de la fin du vecteur ne lit que des octets de l'entrée ; (b) si $r \leq 16$ ($t \leq 8$), X tient dans un registre de 128 bits et on utilise la *voie basse* de T_p ; si $16 < r \leq 32$, X occupe un registre de 256 bits (voie basse = impairs, voie haute = pairs), ce qui demande que les deux voies soient égales ; (c) le coût de préparation de la fin du vecteur est payé une fois par entrée, et celui de la remise en ordre une fois par paire.

Corollaire 2.6 (Limite exacte)**.** *Pour $m \leq 192$, $\Delta(m) := N_{\text{off}}(m) - \lceil m/32 \rceil = 1$ si $0 < (m \bmod 64) \leq 32$, et 0 sinon. Dans le second cas, le code officiel atteint déjà la limite : aucun algorithme de $\mathscr{C}$ ne peut faire mieux en nombre de `vpshufb`.*

TABLEAU 2.1 : Application aux paramètres actuels.

	m	q	r	N_{off}	$\lceil m/32 \rceil$	Δ	Statut
MAYO-1	80	1	16	4	3	1	implémentée (A2), fin de 8 octets
MAYO-2	64	1	0	2	2	0	sans objet
MAYO-3	108	1	44	4	4	0	impossible dans $\mathscr{C}$
MAYO-5	142	2	14	6	5	1	implémentée (A2), fin de 7 octets + 1 de bourrage

Ce qui est fait et ce qui ne l'est pas. Le prototype traite les fins de vecteur d'au plus 16 éléments ($t \leq 8$, $r \leq 16$). La variante pour $16 < r \leq 32$ **n'est ni réalisée ni testée** : ce n'est pas un résultat acquis. Aucun des quatre paramètres actuels n'a $r \in (16, 32]$.

2.1.3 Conception d'A2

2.1.3.1 Structure du noyau

Pour une entrée de $\mathbf{P}_1$ et une ligne d'accumulateurs de K vecteurs, A2 garde tels quels les $q = \lfloor m/64 \rfloor$ registres ymm complets sur les $32q$ premiers octets. Il traite ensuite la fin du vecteur ainsi :

1. **Préparation (une fois par entrée)** : chargement de 8 octets (`_mm_loadl_epi64`), décalage de 4 bits (`vpsrlw`), entrelacement `vpunpcklqdq` (quartets bas dans les octets 0 à 7, quartets hauts dans les octets 8 à 15), ET avec `0x0f`. Cela fait quatre instructions (cinq dans la variante SAFE, qui ajoute un décalage d'octets).
2. **Produits** : un `vpshufb` xmm avec la *moitié basse* de la table de multiplication déjà chargée, et un `vpxor` xmm dans un accumulateur xmm.
3. **Remise en ordre (fin de ligne)** : séparation impair/pair comme pour la partie ymm, puis lecture, modification et écriture d'*un seul* limbe de 64 bits (le limbe d'indice $4q = L - 1$, soit 4 pour MAYO-1 et 8 pour MAYO-5).

Dans l'assembleur produit par GCC 16.2.0, la boucle interne passe donc, par entrée, de 20 à 15 `vpshufb` pour MAYO-1 (10 ymm + 5 xmm) et de 36 à 30 pour MAYO-5 (24 ymm + 6 xmm).

2.1.3.2 Variante par défaut et variante SAFE (MAYO-5)

Pour MAYO-1, $b = 40 = 32 + 8$: la fin du vecteur est exactement la fin de l'entrée, $[32, 40)$. Pour MAYO-5, $b = 71$ et $64 + 8 = 72$:

- *par défaut* : lit $[64, 72)$, donc **un octet après** l'entrée (comme le noyau officiel, dont le dernier chargement couvre $[40, 72)$) ;
- *SAFE* : lit $[63, 71)$ puis décale le registre de 8 bits vers les poids faibles ; l'octet 63 est éliminé, les octets 64 à 70 passent aux positions 0 à 6, et la position 7 reçoit 0. **Toutes les lectures restent dans l'entrée.**

Si l'on intègre un jour A2, on ne retiendrait que la variante SAFE. Les quartets de bourrage (positions 142 et 143 de MAYO-5, car $16L = 144$) n'ont pas d'effet sur les positions utiles (section 2.2.4).

2.1.3.3 Préconditions

(P1) $\mathbf{P}_1$ est rangé à plat, sans bourrage entre les entrées, à l'adresse $P_1 + e \cdot b$; (P2) les accumulateurs ont L limbes, avec du bourrage de quartets seulement à la fin du dernier limbe ; (P3) les tables de multiplication sont construites par la routine officielle (voies égales, lemme 2.7) ; (P4) $m \in \{80, 142\}$: les noyaux A2 ne sont pas actifs pour les autres paramètres (condition de compilation `MAYO_TAIL_XMM_ACTIVE`) ; (P5) compilation sans `ENABLE_PARAMS_DYNAMIC`.

2.1.4 La limite n'implique pas l'accélération

Faire moins de `vpshufb` ne garantit pas d'aller plus vite : il faut ajouter le coût de préparation de la fin du vecteur, les dépendances entre instructions, les rechargements depuis la pile, les chargements non alignés et le débit des unités de calcul. Si seul le débit du `vpshufb` (un par cycle sur Skylake [12]) limitait la boucle, le rapport de temps attendu serait N_{A2}/N_{off} : 15/20 = 0,750 (MAYO-1) et 30/36 = 0,833 (MAYO-5).

TABLEAU 2.2 : Rapport attendu par le seul débit de `vpshufb` et rapport mesuré (S1, A2/officiel, noyaux isolés).

Noyau	Limite « débit seul »	Mesuré (S1)	Lecture
MAYO-1 `P1_times_Vt`	0,750	0,780	plus lent que la limite
MAYO-1 `P1t_times_Vt`	0,750	0,710	plus rapide que la limite
MAYO-5 `P1_times_Vt`	0,833	0,807	plus rapide que la limite
MAYO-5 `P1t_times_Vt`	0,833	0,880	plus lent que la limite

Deux cas sur quatre sont plus rapides que ce que permettrait le seul comptage des `vpshufb` : d'autres effets jouent (par exemple moins de rechargements depuis la pile : de 11 à 5 pour MAYO-1 `P1_times_Vt`, de 30 à 24 pour MAYO-5), mais ce n'est pas établi. Le modèle fondé sur le débit n'explique donc pas à lui seul les gains. Une hypothèse de performance H-P (« sur un processeur où le débit des permutations est le facteur limitant, le temps est au moins $N \cdot P$ cycles par entrée ») ne peut se tester qu'avec des cycles d'horloge, pas avec des ticks TSC. Elle vient après ce travail et reste ouverte.

2.2 Propriétés de correction et de sûreté

Cette section répond à la partie « sûreté » de QR2. Chaque propriété porte une étiquette : **[C]** (déduite du code et de ses invariants, par relecture et par des assertions à la compilation), **[E]** (vérifiée par des tests) ou **[H]** (hypothèse sur la machine ou le compilateur). Une propriété **[E]** seule n'est pas une démonstration. Notation : $n_y = \lfloor m/64 \rfloor$ (1 pour MAYO-1, 2 pour MAYO-5), $T_{\text{OFF}} = 32n_y$ (32 ; 64), $T_{\text{LIMB}} = 4n_y$ (4 ; 8).

Cette section distingue deux sortes d'énoncés. La *correction fonctionnelle* (bornes de lecture et d'écriture, invariant des tables, bourrage) est justifiée par le code et vérifiée par les KAT et des tests. L'*analyse des branchements et des adresses qui pourraient dépendre d'un secret* est un autre sujet : les KAT n'en disent rien, et elle est traitée à part (section 2.2.5).

2.2.1 Limites de lecture de $\mathbf{P}_1$

L'entrée e occupe les octets $[eb, (e+1)b)$.

- **[C]** Les n_y chargements ymm lisent $[eb + 32g, eb + 32g + 32)$ pour $g < n_y$; comme $32n_y \le b$ ($32 \le 40$; $64 \le 71$), ils restent dans l'entrée.

- **[C]** MAYO-1 : la fin du vecteur lit $[eb+32, eb+40)$, exactement la fin de l'entrée.
- **[C]** MAYO-5, SAFE : la fin du vecteur lit $[eb+63, eb+71)$ puis décale de 8 bits ; toutes les lectures restent dans l'entrée.
- **[C]** (variante par défaut, non retenue) MAYO-5 lit $[eb+64, eb+72)$: un octet après l'entrée. Pour $e < N_{\text{ent}} - 1$, c'est le premier octet de l'entrée suivante. Pour la dernière entrée, c'est l'octet 720 863 du tampon, qui est alloué (zone de $8 \cdot$`P1_LIMBS_MAX`$= 731\,016$ octets). Des `_Static_assert` imposent $N_{\text{ent}} b + \text{OVERREAD} \leq 8 \cdot$`P1_LIMBS_MAX` et $\text{OVERREAD} \leq 1$.
- **[E]** Test « garde-page » (la fin de $\mathbf{P}_1$ est collée à une page interdite, sans marge) : le code officiel et A2 par défaut provoquent une erreur SIGSEGV pour MAYO-5, A2 SAFE non ; pour MAYO-1, aucun des trois. Ce test contrôle la propriété ci-dessus sans la démontrer.

Le noyau officiel de MAYO-5 lit lui aussi 1 octet de trop (son dernier ymm couvre les octets 40 à 71), et celui de MAYO-3 en lit 2 (son second ymm couvre $[24, 56)$ pour $b = 54$). A2 SAFE n'a besoin d'aucune marge.

2.2.2 Limites d'écriture

Pour la ligne r, l'accumulateur est à $\text{acc} + rKL$ (en limbes) ; le vecteur k occupe $[kL, (k+1)L)$.

- **[C]** Les écritures ymm touchent les limbes $[kL + 4g, kL + 4g + 4)$ pour $g < n_y$, et jamais au-delà du limbe $4n_y - 1 \leq L - 1$.
- **[C]** La fin du vecteur est traitée par lecture, modification puis écriture d'un seul limbe de 64 bits, d'indice $T_{\text{LIMB}} = L - 1$, pour $k < K$ (et $k + 1 < K$ pour le second, avec une garde explicite). Toutes les écritures sont donc dans les VKL limbes de `acc`. Le noyau officiel écrit un ymm à $L - 4$: même étendue.
- **[E]** Un « canari » placé après `acc` n'est pas modifié ; $\mathbf{P}_1$, passé en `const`, n'est pas modifié non plus (la copie de l'entrée est inchangée).

2.2.3 Invariant des deux voies des tables

A2 lit la *voie basse* de chaque table et suppose qu'elle est égale à la voie haute.

Lemme 2.7. ***[C]*** *Pour tout i, `V_multabs[i]` a ses deux voies de 128 bits égales.*

Justification. (i) Chacune des quatre lignes de 32 octets de `__gf16_mulbase` répète deux fois la même suite de 16 octets (constantes lues dans `arithmetic_common.h`, lignes 12 à 17). (ii) `tbl32_gf16_multab2(b)` est un OU exclusif de ET entre ces lignes et des masques obtenus par `_mm256_set1_epi16(b & 0xf)` décalé puis comparé ; ces opérations agissent par élément de 16 bits, donc voie par voie, avec des masques identiques dans les deux voies. (iii) `mayo_V_multabs`

combine deux telles tables par `^` et `_mm256_slli_epi16(.,4)`, aussi voie par voie. Par récurrence, l'égalité est conservée.

[E] Nous avons vérifié 272 tables avec le mode `MAYO_TAIL_XMM_CHECK` (arrêt en cas d'écart), sur les tables données aux noyaux. C'est une *condition préalable* d'A2 : si les tables étaient construites autrement (autre routine, autre version du code), il faudrait revérifier cet invariant.

2.2.4 Influence des quartets de bourrage

Seul MAYO-5 a des quartets de bourrage (positions 142 et 143 ; pour MAYO-1, $80 = 5 \cdot 16$).

- **[C]** *Chaque position est indépendante.* La sortie à la position j ne dépend que des entrées à la position j de chaque m-vec : une recherche lit un index et une table, jamais un autre index. Les décalages `srli/slli_epi16` mélangent deux octets voisins, mais chaque résultat est suivi d'un ET avec `0x0f` (ou vient d'un registre déjà masqué).

- **[C]** Les deux seules différences entre le code officiel et A2 aux positions de bourrage sont : pour l'officiel, les produits de l'octet 71 (premier octet de l'entrée suivante) ; pour A2 SAFE, 0.

- **[C]** Les noyaux alimentent `Pv`, puis `Vt_times_Pv` produit `VP1V`, qui est utilisé par `compute_rhs`. C'est le premier endroit qui mélange des positions, et il met à zéro les quartets de bourrage avant de les utiliser (`src/mayo.c`, lignes 50 à 60, masque $(1 \ll ((m \bmod 16) \cdot 4)) - 1$). Entre les noyaux et `compute_rhs`, toutes les opérations travaillent position par position.

- **[E]** Tests unitaires (20 000 cas par taille, 8 familles d'entrées) : 0 écart sur les $M/2$ octets utiles et sur les limbes complets après masquage. Sur 30 010 signatures appariées de MAYO-5 (15 005 pour MAYO-1 et pour MAYO-2), aucune signature n'est différente.

Limite. Cette propriété suppose que `Pv` n'est utilisé que par les chemins listés, ce que nous avons établi en cherchant dans le code source de la version AVX2. Si une modification future lisait le bourrage de `Pv`, il faudrait réexaminer la question.

2.2.5 Temps constant : ce qui est établi et ce qui ne l'est pas

- **[C]** Les bornes des boucles (V, K) et les adresses de chargement (eb) ne dépendent que de paramètres publics. Les tables calculées à partir du vinaigre ne sont que des données pour `vpshufb` : ce ne sont jamais des adresses ni des conditions. Le code officiel a la même structure.

- **[C]**→à vérifier sur le programme compilé : à la remise en ordre de la fin du vecteur, A2 ajoute deux `vmovq` d'un registre vectoriel vers un registre général (`ar[...] ^= (uint64_t)_mm_cvtsi128_si64(...)`). Des données liées au secret passent donc par un

registre général, mais seulement comme donnée d'un XOR vers la mémoire. Nous avons inspecté l'assembleur (script `inspect_asm.py`) et n'avons trouvé aucun usage non conforme (adresse, branche, drapeau). Ce résultat vaut pour *une* version de GCC, pas en général.

- **[E]** (indice) Valgrind en mode temps constant (données secrètes marquées comme indéfinies), dans la VM seulement : il repère les branches et les adresses qui dépendent de données marquées, sur les exécutions testées.
- **[H]** Les instructions `vpshufb`, `vpxor`, `vpand`, les décalages, les chargements et les écritures prennent un temps qui ne dépend pas des données sur les processeurs visés (d'après la documentation du constructeur, non vérifiée ici).

Nous ne fournissons aucune preuve formelle de temps constant. Que les KAT réussissent et qu'il n'y ait pas de différence de résultat ne démontre pas le temps constant.

2.2.6 Correction fonctionnelle (vérifiée par les KAT)

Les KAT officiels réussissent pour MAYO-1, 2, 3 et 5 (toutes les sessions), ainsi que les tests du schéma et les tests unitaires précédents. Il y a zéro signature différente dans toutes les paires A/B, et le nombre de tentatives de signature est le même. La correction est donc établie *sur les entrées testées* ; l'argument général repose sur les propriétés **[C]** ci-dessus.

2.3 Protocole expérimental

L'objectif est qu'un lecteur qui a le dépôt, les archives et une machine comparable puisse refaire chaque analyse et, au bruit près, chaque mesure. Les commandes sont à l'annexe A.

2.3.1 Plateforme

Machine physique (toutes les mesures citées au chapitre 3). Portable Intel Core i9-9880H (8 cœurs, 16 threads, microcode `0x104`) ; `systemd-detect-virt = none`, drapeau hyperviseur = 0 ; Kali 7.1.5 (démarrage sur disque, double amorçage) ; GCC 16.2.0 (Debian 16.2.0-3), cmake 4.3.4, Python 3.14.8, numpy 2.4.6. Gouverneur `performance`, turbo actif, SMT actif, secteur branché. Processus épinglé sur le cœur 1 (`taskset`). La fréquence instantanée du cœur n'est enregistrée qu'au lancement (de 800 MHz au repos à 4 540 MHz selon les sessions) ; elle n'est pas une fréquence soutenue. Horloge du portable à UTC+2 ; les journaux `machine.txt` sont en UTC.

Environnement virtuel (historique, non utilisé pour les résultats de performance). VM KVM à 2 cœurs, GCC 13.3.0, cmake 3.28.3. Les mesures de cette VM (qui ont orienté les décisions de conception) ne sont jamais présentées comme performances matérielles.

Compilation. Options de la bibliothèque : `-O3 -funroll-loops -march=haswell -maes`, identiques pour les deux versions comparées. Pour l'analyse statique, la commande exacte de chaque objet est lue dans `compile_commands.json`.

2.3.2 Code comparé

Base amont : dépôt officiel `PQCMayo/MAYO-C`, commit `74de637a7c4a7f9fafa888183bc05f5cf920cd2d`. Les modifications sont appliquées par `git am` dans l'ordre : `0001` (prototype A2), `0002` (sondes de décomposition, inertes par défaut), `0003` (contrôle de l'effet des sondes), `0004` (analyse statique E1, aucun noyau modifié). Les identifiants de commit changent avec `git am` ; les arbres, non (annexe A). L'arbre des mesures S1/S2 est celui après `0001` ; celui de PH1/CT1 après `0003`. Les noyaux optimisés n'ont pas été modifiés pendant la phase d'analyse.

2.3.3 Protocole de mesure apparié

Pour chaque niveau mesuré (`P1_times_Vt`, `P1t_times_Vt`, `m_calculate`, `mayo_sign`), un processus exécute des paires (officiel, A2) dans un ordre *aléatoire*, avec la même graine et le même message pour les deux versions. Le temps est lu par `rdtscp` (ticks TSC). L'ordre d'exécution de chaque paire est enregistré (`xmm_first`). Un processus comprend 3 001 paires. Les signatures des deux versions sont comparées octet par octet, et le nombre de tentatives de signature est vérifié.

2.3.4 Unité statistique et intervalles

Les 3 001 paires d'un même processus ne sont pas des répétitions indépendantes (même état de la machine, même fréquence). L'intervalle à 95 % « naïf », calculé sur les paires, est d'environ ±0,0001 pour `mayo_sign`, alors que la dispersion entre processus est 5 à 15 fois plus grande. **L'unité statistique est le processus.** Pour chaque niveau et chaque processus, on calcule la médiane du rapport A2/officiel. La valeur donnée est la médiane de ces médianes, avec un intervalle à 95 % obtenu par *bootstrap sur les processus*. Nous testons une seule différence entre sessions (S2−S1) : avec deux démarrages, on ne peut pas estimer à part la variation « entre sessions » ; nous donnons donc l'écart et son intervalle bootstrap.

Effet d'ordre. Pour chaque processus, on calcule la médiane du rapport sur les paires « A2 en premier » et sur les paires « officiel en premier ». L'effet d'ordre est la différence, avec un intervalle bootstrap sur les processus ; on le juge par rapport au gain.

2.3.5 Sessions

Les scripts de lancement refusent d'écraser un dossier ou une archive existants : aucune campagne n'écrase la précédente. S1 et S2 comparent deux démarrages différents avec le même code,

TABLEAU 2.3 : Sessions de mesure sur machine physique (dates UTC).

Session	Date, démarrage	Plan	Objet
S1	19 oct. 21 :58–22 :43 ; un démarrage	4 campagnes × 5 processus × 3 001 paires (graines 1–20)	gains
S2	20 oct. 12 :10 ; nouveau démarrage	5 processus (graines 21–25)	reproductibilité
PH1	21 oct. 19 :13 ; nouveau démarrage	MAYO-5 : 10 processus ; MAYO-1, MAYO-2 : 5	décomposition instrumentée
CT1	21 oct. 19 :28 ; même démarrage que PH1	5 configurations × 10 processus (graines 41–50)	effet des sondes

la même machine et le même compilateur. Pour les temps absolus, S2 est environ 7 % plus lent que S1 pour tous les niveaux, témoins compris ; nous n'avons pas mesuré la cause, et les rapports ne sont pas touchés.

2.3.6 Instrumentation de décomposition

Des sondes (`rdtscp` précédé d'un `lfence`) mesurent, pour chaque signature et chaque version, le temps total, le nombre de tentatives et, pour chaque phase, le temps et le nombre d'appels. Elles sont inactives par défaut : sans la macro correspondante, le code objet de `mayo.c` et de `arithmetic.c` est identique à l'original (vérifié par `cmp` et `objdump`). Le patch `0003` ajoute un masque pour activer chaque sonde séparément. Les cinq configurations de CT1 sont : `plain` (bibliothèque d'origine, aucune sonde), `none` (sondes compilées mais toutes désactivées), `esk` (sonde sur `expand_sk_internal` seule), `mcalc` (sonde sur `m_calculate_M_and_VPV` seule) et `all` (toutes les sondes).

2.3.7 Analyse statique du code généré (E1)

Pour chaque construction, nous recompilons l'objet avec la commande enregistrée, nous le désassemblons (`objdump -d -M intel`) et nous repérons les boucles de travail par leur nombre de `vpshufb` (36 ymm pour l'officiel de MAYO-5 ; 24 ymm + 6 xmm pour A2). Les comptes sont ramenés au facteur de déroulage de boucle (ici $u = 1$). Nous identifions le noyau par le *pas d'adresse sur* $\mathbf{P}_1$: `add reg,0x47` (entrées de 71 octets, le pas d'une entrée) pour `P1_times_Vt` et `imul ...,0x47` pour `P1t_times_Vt`. Les repères de débogage de GCC ne sont pas fiables (les corps identiques sont fusionnés et étiquetés `P1_times_O`). Pour attribuer une différence de code à une cause, il faut d'abord avoir écarté les autres modifications du code (section 2.3.8).

2.3.8 Protocole E1b (analyse statique de l'origine des écritures)

E1b recompile `src/mayo.c` avec *la commande exacte* de E1 plus une seule option en plus, sans aucune mesure de temps. Constructions : `P_none` (sans sondes), `P_all` (toutes les sondes), `TAB` (construction des mesures S1/S2). Variantes :

`base` (contrôle : doit redonner exactement le code de E1); `dump` (fichiers de sortie `-fdump-tree-optimized`, `-fdump-tree-sra-details`, `-fdump-tree-esra-details`, `-fdump-rtl-expand`, `-fdump-rtl-ira`, `-fopt-info-inline-optimized`; contrôle : le code doit être identique); `nosra` (`-fno-tree-sra`); `noipasra` (`-fno-ipa-sra`); `nodse`; `nopre`; `nofre`.

Pour chaque variante, le rapport donne : les boucles de travail avec leur nombre d'instructions et d'écritures de 32 octets ; le nombre d'instructions de `sign_signature_internal` *en dehors des boucles de travail* ; la distance entre les histogrammes de mnémoniques, hors boucles, par rapport à `base` ; et le nombre de lignes GIMPLE qui mentionnent `temp`. **Règle de lecture :** on ne lit jamais seule la disparition des écritures dans une variante. On ne l'attribue à une passe que si la distance hors boucles est faible, c'est-à-dire si rien d'autre n'a changé.

TABLEAU 2.4 : Issues possibles d'E1b et conclusions autorisées (fixées avant exécution).

	Observation (GCC 16.2.0)	Conclusion autorisée / non autorisée
1	Doublon absent avec `-fno-tree-sra` *et* distance en dehors des boucles faible	la passe conditionne le doublon dans cette compilation / non : que `temp[]` en soit la cause sans les fichiers d'analyse
2	Doublon absent mais distance élevée	l'attribution est *non établie*
3	Doublon présent dans toutes les variantes	les passes testées ne le suppriment pas, hypothèse non confirmée / non : que le doublon soit sans cause compilateur
4	`base` ≠ code de E1	E1 non reproduit : arrêter l'interprétation

2.3.9 Traçabilité et reproduction

Les données brutes (S1, S2, PH1, CT1) sont publiées avec leurs empreintes SHA-256 (annexe A). Le script de reproduction réextrait les archives, vérifie les empreintes, relance les neuf analyses et compare les sorties : **9 analyses sur 9 identiques** lors de la dernière exécution. On retrouve *exactement* les analyses ; pour les *mesures*, on obtiendra des valeurs proches mais pas identiques (ticks TSC, fréquence, bruit).

2.3.10 Menaces à la validité identifiées a priori

- **Une machine, un compilateur sur machine physique**, trois démarrages ; S2 sur 5 processus.
- **Ticks TSC, non cycles cœur** : la fréquence d'exécution réelle est inconnue ; aucune conclusion sur le débit des unités d'exécution n'est possible.
- **Effet d'ordre** documenté (section 3.5).
- **Les sondes modifient le code** : mesures instrumentées ≠ mesures sans sonde.
- **Pas de compteurs matériels** disponibles dans l'environnement d'analyse.

- **Première exécution de la campagne s3 de S1** : CSV bruts non archivés (seule la seconde exécution l'est).
- **Comparaison VM/physique** : le gain de MAYO-5 sur `mayo_sign` était de 7,1 % dans la VM (GCC 13.3.0) ; l'écart avec la machine physique mêle machine et compilateur et n'est pas expliqué.

CHAPITRE 3

RÉSULTATS, INTERPRÉTATION ET DISCUSSION

Ce chapitre présente et interprète les résultats qui répondent à QR3 et à QR4 : d'abord la correction, puis les gains mesurés sur les noyaux et sur la signature complète, l'effet d'ordre, l'anomalie observée pour MAYO-5, et l'analyse du code produit (E1 et E1b). Il se termine par une discussion critique de ce qui est établi et de ce qui reste ouvert.

Sauf indication contraire, les valeurs sont des rapports A2 SAFE / officiel (médiane des processus), en ticks TSC, sur la machine physique décrite au chapitre 2. Les étiquettes [F], [I], [H] sont celles de l'introduction générale : [F] fait, [I] déduction, [H] hypothèse.

3.1 Correction

[F] Les tests officiels (KAT) de MAYO-1, 2, 3 et 5 réussissent dans toutes les sessions. Il y a 0 signature différente sur les 30 010 paires de MAYO-5 et sur les 15 005 paires de MAYO-1 et de MAYO-2, et le nombre de tentatives est le même dans les deux versions. Les propriétés de sûreté sont dans la section 2.2.

3.2 Gains sur les noyaux et sur la signature (S1, S2)

Deux résultats à ne pas confondre. Le premier est une *économie d'instructions* : dans notre modèle de comptage et dans l'assembleur, A2 réduit le nombre de `vpshufb` par entrée de $\mathbf{P}_1$ (20→15 pour MAYO-1, 36→30 pour MAYO-5). Le second est un *gain mesuré* sur la signature complète, sur notre configuration : **10,3 % pour MAYO-1** (premier résultat à retenir) et **1,3 % pour MAYO-5** (résultat principal pour ce paramètre). Le 7,2 % de MAYO-5 avec le binaire instrumenté n'est pas un gain de signature à retenir : il sert à montrer pourquoi le protocole de mesure compte (section 3.6).

TABLEAU 3.1 : Rapport A2/officiel, médiane des processus [IC95 bootstrap entre processus]. S1 : 20 processus ; S2 : 5 processus. Gain = 1 − rapport.

Paramètre	Niveau	S1	S2	Gain S1 / S2
MAYO-1	`P1_times_Vt`	0,7803 [0,7793 ; 0,7817]	0,7814 [0,7794 ; 0,7832]	22,0 / 21,9 %
	`P1t_times_Vt`	0,7099 [0,7060 ; 0,7124]	0,7080 [0,7024 ; 0,7126]	29,0 / 29,2 %
	`m_calculate`	0,8110 [0,8096 ; 0,8118]	0,8102 [0,8080 ; 0,8123]	18,9 / 19,0 %
	`mayo_sign`	**0,8975** [0,8970 ; 0,8979]	**0,8974** [0,8968 ; 0,8990]	**10,3 / 10,3 %**
MAYO-5	`P1_times_Vt`	0,8067 [0,8060 ; 0,8071]	0,8049 [0,8043 ; 0,8076]	19,3 / 19,5 %
	`P1t_times_Vt`	0,8798 [0,8783 ; 0,8818]	0,8792 [0,8770 ; 0,8858]	12,0 / 12,1 %
	`m_calculate`	0,8851 [0,8845 ; 0,8863]	0,8869 [0,8847 ; 0,8874]	11,5 / 11,3 %
	`mayo_sign`	**0,9873** [0,9871 ; 0,9876]	**0,9871** [0,9863 ; 0,9876]	**1,3 / 1,3 %**

[F] Pour les témoins (aucun noyau A2), le rapport vaut 1 : MAYO-2 `mayo_sign` 1,0000 / 1,0000 et `m_calculate` 1,0001 / 0,9996 ; MAYO-3 `mayo_sign` 1,0000 / 1,0001. Les écarts entre S2 et S1 sur les huit lignes de gains sont au plus de 0,0019 en valeur absolue, et l'intervalle à 95 % contient 0 (par exemple pour `mayo_sign` : −0,0001 [−0,0009 ; +0,0016] pour MAYO-1 et −0,0003 [−0,0011 ; +0,0003] pour MAYO-5).

Variabilité à quatre niveaux. Écart-type robuste entre les paires d'un même processus : 0,0015 (S1) et 0,0013 (S2) pour MAYO-1, 0,0114 et 0,0039 pour MAYO-5. Entre les processus

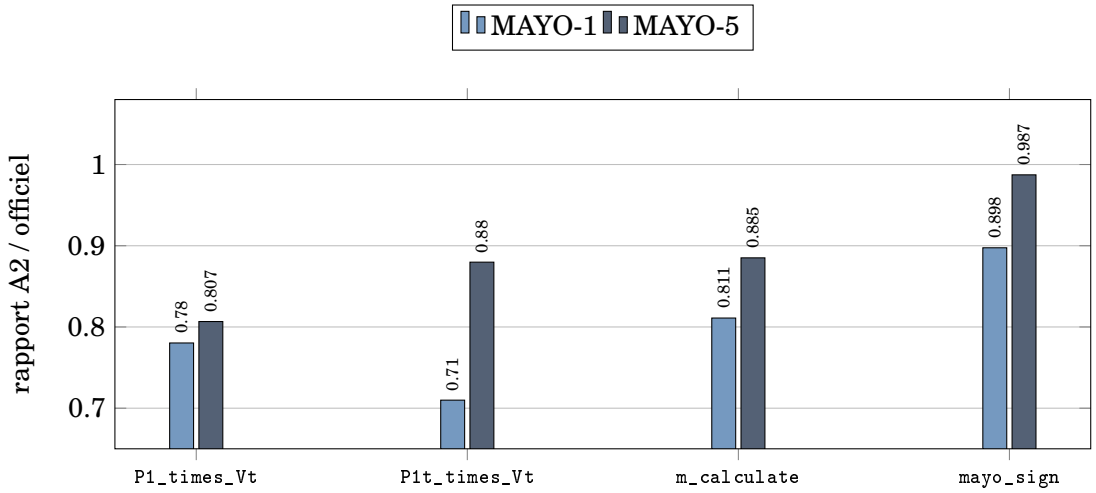


FIGURE 3.1 : Rapports A2/officiel mesurés (S1). Le gain se dilue de la couche noyau à la signature complète, fortement pour MAYO-5.

d'une même session : 0,0009 (20 processus) et 0,0010 (5 processus) pour MAYO-1, 0,0008 et 0,0005 pour MAYO-5. Entre les campagnes de S1 : 0,0001 et 0,0002. C'est donc la dispersion entre processus, et non entre paires, qui fixe l'incertitude.

Gain de MAYO-5 sur la signature. [F] Il est de 1,3 % dans S1, S2, PH1/CT1 et dans la mesure de référence, proche du critère d'environ 1 % fixé avant les mesures. C'est le seul chiffre de gain de signature de MAYO-5 à retenir (section 3.6).

3.3 Temps de base du code officiel

Pour situer l'évaluation, nous avons mesuré le code C *officiel* (AVX2, sans aucune de nos modifications) avec le programme de mesure fourni par MAYO-C, pour les quatre jeux de paramètres. Chaque programme fait 1 000 essais par opération et on répète cinq fois, sur le cœur 1 de la machine de la section suivante (portable i9-9880H, GCC 16.2.0), après la compilation du code officiel seul. Le tableau 3.2 donne, pour chaque opération, la médiane des cinq médianes. Les durées sont en ticks du compteur TSC (instruction `rdtsc`), pas en cycles d'horloge du cœur. [F] Ce sont des valeurs mesurées, sur une machine et une session ; elles servent de point de repère, pas de comparaison entre machines. [F] La signature est l'opération la plus coûteuse pour les quatre jeux, et c'est elle que touche notre modification pour MAYO-1 et MAYO-5. Pour MAYO-1, les cinq médianes de signature vont de 276 406 à 282 932 ; pour MAYO-5, de 1 480 710 à 1 491 080. La première répétition de MAYO-1 est plus basse que les autres sur plusieurs opérations (par exemple 91 192 contre environ 100 000 pour la génération des clés) : nous ne l'expliquons pas, et la médiane des cinq limite son effet. Ces valeurs ne se comparent pas directement à celles de la

TABLEAU 3.2 : Temps de base du code officiel (AVX2), en ticks TSC : médiane des cinq médianes de 1 000 essais. Clés : génération des clés ; Exp. sk et Exp. pk : expansion de la clé secrète et de la clé publique ; Signature : `mayo_sign`, à partir de la clé secrète compacte ; Vérif. : `mayo_verify`, avec la clé publique compacte.

Jeu	Clés	Exp. sk	Exp. pk	Signature	Vérif.
MAYO-1	100 322	127 914	51 276	281 450	124 566
MAYO-2	80 838	92 934	38 918	130 280	60 096
MAYO-3	241 412	323 188	124 516	608 630	279 252
MAYO-5	646 886	932 800	277 568	1 489 860	678 022

section suivante, qui mesure un autre programme (signature suivie d'une vérification, avec des messages de 32 octets).

3.4 Comparaison ciblée : compilations séparées, sans sondes

Cette comparaison a été demandée par l'encadrant pour savoir si le gain se maintient quand chaque version est compilée seule.

Protocole. MAYO-C officiel et A2 SAFE sont compilés *séparément* (deux exécutables, même fichier de test), sans sondes de mesure et sans `MAYO_TAIL_AB`. Une seule machine, GCC 16.2.0 (Debian 16.2.0-3) : portable Intel Core i9-9880H, installation directe de Kali (pas de machine virtuelle), noyau 7.1.5, gouverneur `powersave`, `no_turbo = 0`, processus fixé sur le cœur 1. Un processus = une paire de clés (graine) + 200 signatures complètes de messages de 32 octets, après 20 signatures d'échauffement ; il donne la médiane des ticks TSC par signature. L'unité statistique est le *processus*. À chaque paire, les deux versions reçoivent la même graine et donc les mêmes entrées, et l'ordre d'exécution alterne d'une paire à l'autre : 401 paires, pour MAYO-1 et MAYO-5. Avant la mesure, les KAT des deux versions réussissent. Pendant la mesure, pour chaque paire, les empreintes de toutes les signatures sont identiques dans les deux versions, et chaque signature est vérifiée. Les commandes de compilation sont en annexe D. [F] Le rapport

TABLEAU 3.3 : Comparaison ciblée : rapport A2 SAFE / officiel de la médiane des ticks par signature, médiane des 401 rapports par paire [IC95 bootstrap sur les paires].

Paramètre	Officiel	A2 SAFE	Rapport	Gain
MAYO-1	291 740	262 176	0,8988 [0,8979 ; 0,8994]	10,1 %
MAYO-5	1 471 920	1 450 402	0,9854 [0,9851 ; 0,9858]	1,5 %

Les colonnes « Officiel » et « A2 SAFE » donnent la médiane, sur les 401 processus, des médianes de ticks.

est inférieur à 1 pour les 401 paires de MAYO-1 et pour 398 des 401 paires de MAYO-5. L'ordre d'exécution ne change presque rien : pour MAYO-1, 0,8990 (A2 d'abord) et 0,8988 (officiel d'abord) ; pour MAYO-5, 0,9855 et 0,9852. Aucune signature ne diffère entre les deux versions et aucune vérification n'échoue.

Lecture. [I] Ces valeurs (10,1 % et 1,5 %) sont proches de celles de S1 (10,3 % et 1,3 %). Elles vont dans le même sens, avec les compilations séparées et sans sondes. L'intervalle est étroit parce qu'il ne tient compte que de la variation entre paires de processus dans *une* session, sur *une* machine : il ne couvre pas la variation entre sessions, entre machines ou entre versions du compilateur. Nous ne généralisons pas au-delà de cette configuration. Les autres tailles de fin de vecteur et les campagnes E2 à E5 sont réservées aux perspectives.

3.5 Effet d'ordre

[F] Les ordres sont équilibrés : A2 passe en premier dans 49,97 % des paires de `P1t_times_Vt` de MAYO-1 en S1 (29 993/60 020) et dans 50,5 % en S2 ; la probabilité que A2 passe en premier,

sachant que A2 était le précédent, est de 0,5003.

TABLEAU 3.4 : Rapport A2/officiel selon l'ordre d'exécution, et écart par processus [IC95].

Niveau	Session	A2 en 1er	officiel en 1er	écart [IC95]	proc. > 0
MAYO-1 `P1t_times_Vt`	S1	0,7142	0,6985	+0,0151 [+0,0101 ; +0,0206]	20/20
	S2	0,7095	0,7009	+0,0043 [−0,0070 ; +0,0161]	4/5
MAYO-1 `mayo_sign`	S1	0,8975	0,8975	−0,0001 [−0,0002 ; 0,0000]	8/20
MAYO-5 `mayo_sign`	S1	0,9870	0,9876	−0,0006 [−0,0008 ; −0,0005]	0/20
	S2	0,9871	0,9871	0,0000 [−0,0002 ; 0,0000]	2/5
MAYO-5 `P1_times_Vt`	S1 / S2	0,8071 / 0,8053	0,8059 / 0,8043	+0,0010 / +0,0010	20/20 ; 5/5
MAYO-2 `m_calculate` (témoin)	S1 / S2	1,0005 / 1,0004	0,9997 / 0,9994	+0,0010 / +0,0006	20/20 ; 4/5

[F] Le protocole produit lui-même un effet général d'environ 0,1 % (visible sur MAYO-2 `m_calculate`, où les deux versions sont le même code). L'effet de MAYO-1 `P1t_times_Vt` est environ quinze fois plus grand et propre à ce noyau. Il n'a pas d'effet sur `mayo_sign` de MAYO-1 (≤ 0,0001). Pour `mayo_sign` de MAYO-5, il vaut au plus 0,0006, soit moins de 5 % du gain de 0,0127. La version exécutée en second est plus rapide (MAYO-1 `P1t_times_Vt`, S1 : officiel −0,9 %, A2 −1,6 %), et l'effet est plus fort pour A2, d'où l'écart sur le rapport. **[H]** La cause n'est pas démontrée et nous n'avons fait aucune expérience pour la chercher. Pour ce noyau, nous donnons aussi la moyenne des deux ordres (0,706) en plus de la médiane de l'ensemble (0,710).

3.6 L'anomalie de MAYO-5 : décomposition instrumentée et contrôle

3.6.1 Toutes sondes actives (PH1)

[F] MAYO-5 : il y a une tentative de signature par signature dans les deux versions ; chaque phase est appelée une fois par signature, dont `P1_times_Vt` et `P1t_times_Vt` une fois par tentative. Le temps total est de 1 374 501 ticks (officiel) et 1 274 140 (A2) : rapport 0,9270 [0,9253 ; 0,9283], soit un gain de 7,3 %. La différence moyenne de −100 967 ticks se répartit ainsi : −100 529 pour `m_calculate` (99,6 %), dont −54 584 pour `P1_times_Vt` et −46 166 pour `P1t_times_Vt`. Les autres phases (dont `expand_sk_internal`, `sample_solution`, `compute_A`) sont identiques à ±0,1 % près. MAYO-1 : rapport 0,8983 ; MAYO-2 (témoin) : 0,9999.

3.6.2 Le gain avec toutes les sondes n'est pas le gain sans sonde (CT1)

3.6.3 Mesure directe, sans soustraction de médianes (config. `mcalc`)

[F] Le reste de la signature est identique dans les deux versions. Toute la différence est dans `m_calculate` *quand il est dans la signature et sans sonde interne* : il coûte 931 000 ticks (officiel) et 912 000 (A2), contre environ 801 000 et 707 000 quand il est isolé ou avec toutes les sondes. Le surcoût dû au contexte est d'environ 130 000 ticks pour l'officiel et 205 000 pour A2. La différence

TABLEAU 3.5 : Cinq configurations du même programme (10 processus chacune, graines 41–50, un démarrage). Ticks par signature.

Configuration	Officiel	A2	Rapport [min–max]	Gain
`plain` (aucune sonde)	1 504 741	1 484 504	0,9871 [0,9862–0,9879]	1,29 %
`none` (sondes compilées, désactivées)	1 504 802	1 484 611	0,9868 [0,9860–0,9880]	1,32 %
`esk` (`expand_sk_internal`)	1 504 167	1 484 146	0,9871 [0,9861–0,9879]	1,29 %
`mcalc` (`m_calculate_M_and_VPV`)	1 502 948	1 483 529	0,9877 [0,9865–0,9885]	1,23 %
`all` (toutes les sondes)	1 372 740	1 273 970	0,9276 [0,9261–0,9291]	7,24 %

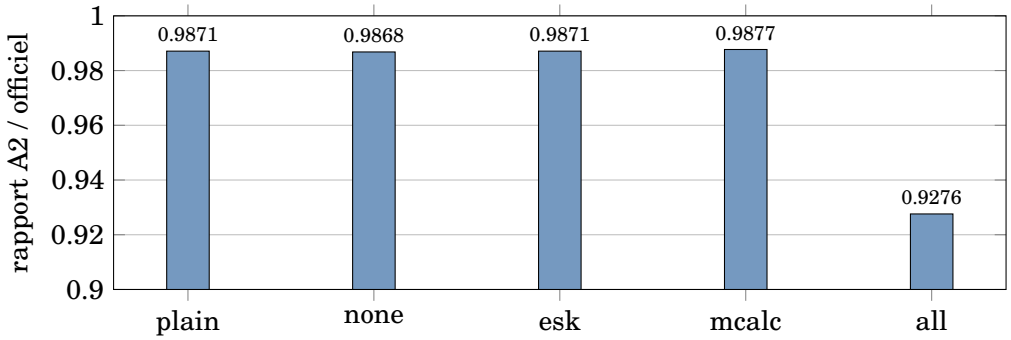


FIGURE 3.2 : MAYO-5, rapport `mayo_sign` A2/officiel selon la configuration de sondes (CT1). Seule la configuration « toutes les sondes » change le résultat.

TABLEAU 3.6 : Décomposition directe de la signature MAYO-5 (ticks par signature).

	Officiel	A2	Différence
`m_calculate` dans la signature	931 453	912 476	−18 406 [−19 396 ; −17 558]
hors `m_calculate` (total − sonde)	566 917	567 323	+406 (0,07 %)
`expand_sk_internal` (config. `esk`)	322 655	322 661	+6
`m_calculate` isolé (banc, tampons réutilisés)	801 324	707 314	≈ −94 000 (11,7 %)
`m_calculate`, toutes sondes, dans la signature	801 073	699 775	≈ −101 000

de 75 000 ticks entre les deux versions absorbe donc l'essentiel du gain mesuré sur le noyau isolé. Ce surcoût disparaît avec toutes les sondes (donc avec des `lfence` entre les sous-étapes de `m_calculate`), mais pas avec une seule sonde autour de `m_calculate`.

Correction d'une interprétation antérieure. Nous avions d'abord lu : « le reste de la signature ralentit avec A2 ». C'était une erreur, venue d'une soustraction de médianes prises à des niveaux de mesure différents. Cette lecture est *retirée*.

3.7 Analyse statique du code généré (E1, GCC 16.2.0)

3.7.1 Comptage de base

[F] Par entrée de $\mathbf{P}_1$, avec le déroulage de boucle ramené à 1, le code officiel a 20 `vpshufb` (MAYO-1) et 36 (MAYO-5), et A2 en a 15 (10 ymm + 5 xmm) et 30 (24 ymm + 6 xmm), comme le prévoit le modèle de la section 2.1. Dans le banc de test, la boucle interne passe de 124 à 106 instructions pour MAYO-5 `P1_times_Vt` et de 126 à 110 pour `P1t_times_Vt` ; pour MAYO-1, de 68 à 51 et de 71 à 56. Les rechargements depuis la pile passent de 30 à 24 (MAYO-5 `P1_times_Vt`) et de 11 à 5 (MAYO-1). Cette réduction est une explication appuyée sur l'assembleur ; ce n'est pas une preuve que ces instructions expliquent à elles seules le gain.

3.7.2 Écritures supplémentaires en contexte de signature

TABLEAU 3.7 : Instructions de la boucle interne par entrée de $\mathbf{P}_1$ (MAYO-5) selon la construction.

Construction	`P1` off.	`P1` A2	`P1t` off.	`P1t` A2
banc isolé	124	106	126	110
contexte, toutes sondes	123	105	126	109
contexte, sans toutes les sondes (`ctx_ab`, `esk`, `mcalc`, `none`)	123	135	162	139

[F] La différence entre « sans toutes les sondes » et « toutes les sondes » est *exactement* une écriture de plus par accumulateur et par entrée, de la taille de l'accumulateur (`vmovdqa [rbp-...], ymm` pour 32 octets, version xmm pour 16 octets). Pour `P1t_times_Vt` officiel : +36 écritures de 32 octets (36 accumulateurs ymm). Pour la boucle `P1_times_Vt` d'A2 : +30 écritures, qui comprennent 24 écritures de 32 octets et 6 de 16 octets (section 3.9) ; le total est aussi de +30 pour `P1t_times_Vt` d'A2 (30 accumulateurs). Le reste de la boucle est identique, instruction pour instruction. La boucle officielle `P1_times_Vt` n'a pas d'écriture en double. Ces écritures vont vers une deuxième série d'emplacements consécutifs (pas de `0x20`), situés à `0xbfc0` octets de la première, dont l'adresse est prise ailleurs dans la fonction (`lea`). Deux boucles de 160 instructions (`Ut_times_O`, `m_lambda_times_O`) ont aussi une écriture en double (de 123 à 160) dans *les deux* versions. Le cadre de pile dépasse 220 Ko : on ne peut donc pas distinguer, par le seul décalage, un débordement de registres d'un tableau local. Avec GCC 13.3 (dans la VM), aucune écriture en double n'apparaît. Pour MAYO-1 (GCC 16.2.0), elle ne touche que `P1t_times_Vt` (officiel de 71 à 91 ; A2 de 56 à 71).

[F] Nous repérons la boucle officielle `P1_times_Vt` dans le contexte par son pas d'adresse `add reg,0x47`, et nous avons contrôlé ce repérage dans six constructions. Elle compte 123 instructions et 29 écritures dans les contextes officiel, sans sondes et avec toutes les sondes (à des adresses distinctes), sans écriture en double. Le tableau 3.7 résume ces comptes.

3.8 Lien quantitatif entre instructions et temps : modèle à un paramètre

[F] Gain calculé sur le code (P1+P1t) : 13,6 % sur le banc, 14,1 % dans le contexte avec toutes les sondes, et **3,9** % dans le contexte sans sondes (250 → 216 ; 249 → 214 ; 285 → 274 instructions). Gain mesuré de `m_calculate` : 11,5 à 11,7 % isolé, 12,4 % avec toutes les sondes, **2,1** % sans sondes (936 031 → 916 206). Dans les trois cas, le gain calculé sur le code et le gain mesuré évoluent dans le même sens.

Modèle. On suppose que le temps d'une boucle est proportionnel à son nombre d'instructions par entrée, avec un même coefficient c pour les deux versions ($N_{\text{ent}} = 10\,153$). Calé sur « toutes les sondes », on obtient $c = 635\,681/(249 \cdot 10\,153) = 0{,}251$ tick par instruction. **Prévision** sans sondes : un gain de 28 082 ticks pour P1+P1t ; **mesure** : 19 825. Le modèle *exagère* donc le gain d'environ 40 %.

Coût d'une écriture en plus. À partir de la différence entre « sans sondes » et « toutes sondes », on trouve 130 895/365 508 = 0,358 tick par écriture pour l'officiel et 210 808/609 180 = 0,346 pour A2.

Portée de ce rapprochement. Les valeurs 0,358 et 0,346 sont proches, mais c'est seulement un test de cohérence (un paramètre, deux observations) ; *ce n'est pas une validation du modèle*. Le modèle exagère le gain : il n'explique donc pas l'écart de façon chiffrée. Le fait que les écritures en plus et la perte de gain évoluent ensemble ne prouve pas que l'un cause l'autre.

3.9 Analyse statique E1b : résultats (GCC 16.2.0)

Nous avons exécuté E1b sur la machine de mesure (GCC 16.2.0, binutils 2.47). L'archive des sorties (SHA-256 `49452120b1b787ad2aed0f30cbcece4f7584a6063768757d92214e264dcd3faa`) est publiée avec le script `e1b_dumps_analyse.py`, qui produit les comptes cités ici. Aucune mesure de temps n'est faite.

3.9.1 Contrôles

[F] Pour les trois constructions (`P_none`, `P_all`, `TAB`), le code de `sign_signature_internal` recompilé en variante `base` est identique à celui d'E1. La variante `dump` (qui ajoute seulement des fichiers de sortie) donne aussi un code identique (distance 0 en dehors des boucles). Les constructions `TAB` et `P_none` donnent les mêmes comptes dans toutes les variantes.

TABLEAU 3.8 : E1b : boucles de travail selon l'option ajoutée (instructions / écritures de 32 ou 16 octets). « Dist. » : distance entre histogrammes de mnémoniques en dehors des boucles de travail, par rapport à `base`.

Constr.	Variante	Dist.	`P1` A2	`P1t` A2	`P1t` officiel
`P_none`	`base`	0	135 / 52	139 / 52	162 / 65
	`nosra`	1	135 / 52	139 / 52	162 / 65
	`noipasra`	1	135 / 52	139 / 52	162 / 65
	`nodse`	412	135 / 52	139 / 52	162 / 65
	`nopre`	17	135 / 52	139 / 52	162 / 65
	`nofre`	357	135 / 52	139 / 52	185 / 58 (autres boucles modifiées)
`P_all`	`base`	0	105 / 22	109 / 22	126 / 29
	`nosra`	3	105 / 22	109 / 22	126 / 29
	`nodse`	45	105 / 22	109 / 22	126 / 29
	`nopre`	5	105 / 22	109 / 22	126 / 29
	`nofre`	876	**135 / 52**	**139 / 52**	185 / 58
	`noipasra`	1181	aucune boucle de travail reconnue (fonction de 2 892 instr.)		

3.9.2 Désactivation de passes, une à la fois

[F] Aucune des variantes `nosra`, `noipasra`, `nodse`, `nopre` (qui désactivent SRA, IPA-SRA, DSE et PRE) ne supprime l'écriture en double dans `P_none` ni dans `TAB` : les comptes des boucles ne changent pas (ligne 3 du tableau 2.4). **[F]** Avec la variante `nofre`, les boucles A2 gardent leur écriture en double, mais le code change beaucoup ailleurs (distance 357) et les boucles officielles prennent une autre forme. Dans `P_all`, la même option *fait apparaître* l'écriture en double dans les boucles A2 (135/52, 139/52), avec une distance de 876. **[I]** Ces deux dernières observations ne permettent *aucune attribution* à la passe FRE, puisque d'autres parties du code changent en même temps. **[F]** Avec la variante `noipasra` sur `P_all`, la structure de la fonction change tellement qu'aucune boucle de travail n'est reconnue : on ne peut rien lire.

3.9.3 Ce que montrent les sorties de GCC

SRA ne touche pas à `temp`, dans aucune des deux constructions. **[F]** Dans le fichier d'analyse `sra`, `temp` est écarté 12 fois dans chacune (8 fois pour « adresse prise hors d'un argument d'appel » et 4 fois pour « accès absents ou qui se recouvrent »). Le fichier `esra` n'en parle pas. SRA ne peut donc pas expliquer la différence entre `P_none` et `P_all`, ce qui correspond au tableau 3.8. L'hypothèse posée avant E1b (rôle de SRA) est *écartée*.

Les tableaux sont présents dans les deux constructions. **[F]** Le code GIMPLE optimisé déclare les mêmes tableaux locaux dans les deux cas (7 `__m256i temp[36]`, 2 `__m256i temp[24]`, 2 `__m128i tt[6]`).

Ce qui diffère : les écritures dans la boucle. **[F]** Dans le GIMPLE optimisé, 8 blocs de travail contiennent des écritures `temp[i] = ...` (ou `tt[i] = ...`) *dans le corps de la boucle* pour

`P_none`, contre 3 pour `P_all`. Dans ces blocs, le nombre d'écritures par tour est égal au nombre d'accumulateurs (24 + 6 pour A2, 36 pour l'officiel). La boucle ne *lit jamais* `temp[]`, et les accumulateurs passent d'un tour à l'autre par des variables PHI (`prephitmp_N`). Dans `P_all`, 9 blocs utilisent des variables `temp_I_lsm_N` sans écrire le tableau (144 variables `temp_I_lsm` sont déclarées dans `P_none`, 300 dans `P_all`).

Lien avec l'assembleur. [F] La boucle A2 de `P1_times_Vt` compte 52 écritures dans `P_none` (42 de 32 octets et 10 de 16 octets) et 22 dans `P_all` (18 et 4). Les 22 écritures de `P_all` visent exactement les mêmes emplacements de pile que 22 des 52 de `P_none`. Les 30 autres sont 24 écritures de 32 octets (pas de `0x20`) et 6 de 16 octets (pas de `0x10`), ce qui correspond à la forme de `temp[24]` et de `tt[6]`. [I] Nous identifions ces 30 écritures en plus aux écritures de `temp[]`/`tt[]` du GIMPLE à partir de ce décompte et de la forme des tableaux, et non d'une correspondance entre noms de symboles.

3.9.4 Interprétation autorisée

[I] Dans la construction sans toutes les sondes, GCC remplace les lectures de `temp[]` dans la boucle par des valeurs qui passent d'un tour à l'autre dans des PHI, mais il *ne retire pas les écritures de la boucle*. Le tableau, dont l'adresse est utilisée après la boucle, est donc mis à jour à chaque tour, en plus des emplacements où les accumulateurs débordent en mémoire. Dans la construction avec toutes les sondes, les accumulateurs sont traités comme des variables simples (suffixe `_lsm`) et le tableau n'est écrit qu'à la sortie de la boucle. [I] Cela correspond à l'hypothèse de départ (`temp[]` gardé en mémoire en plus des variables), mais *sans que SRA y soit pour quelque chose*.

[F] Le suffixe `_lsm` correspond au déplacement des écritures hors des boucles. Les fichiers d'analyse de la passe `lim2` (option `-fdump-tree-lim2-details`, produits avec GCC 16.2.0) le confirment : on y lit « Executing store motion of `temp[`i`]` from loop n ».

3.9.5 Les fichiers d'analyse de lim2 et de pre

Ces fichiers ont été produits en compilation seule, pour MAYO-5, avec la commande de E1 plus une option de sortie. Ils ne changent pas le code : pour `P_none`, `P_all` et `TAB`, le nombre d'instructions de `sign_signature_internal` et la distance des mnémoniques sont identiques à la variante `base` (distance 0).

Passe `pre`. [F] Aucun déplacement ne vise `temp[]` ou `tt[]` (0 dans les trois constructions).

Passe `lim2`, dans `sign_signature_internal`. [F] Dans `P_none`, la sortie des écritures a lieu pour 4 boucles, toutes de la forme `temp[0..35]` (celle du noyau officiel) ; aucune boucle de la forme A2 (`temp[0..23]` et `tt[0..5]`) n'est concernée. Dans `P_all`, elle a lieu pour 6 boucles

de la forme `temp[0..35]` et pour 2 boucles de la forme A2, avec les 24 emplacements de `temp` et les 6 de `tt`, soit les 30 accumulateurs du noyau A2.

Lien avec l'assembleur. [F] Les boucles A2 ont 52 écritures de 32 octets dans `P_none` et 22 dans `P_all` ; la boucle officielle `P1t_times_Vt` en a 65 dans `P_none` et 29 dans `P_all`. Ces écarts vont dans le même sens que les boucles que `lim2` traite ou non.

Ce que dit lim2 sur le refus, pour `tt[0]`. [F] Dans `P_none`, pour la boucle 58, lim2 note : « SM WAR dependencies of ref 384 in loop 58 : dependent » (ref 384 = `tt[0]`). Les références jugées « dependent » avec elle sont deux lectures `MEM[(const __m256i_u * \{ref-all\})]` faites par des pointeurs, `a0` et `v`. Dans `P_all`, pour la boucle 46, la même question donne « independent ». Dans `P_none`, 4 références passent par un pointeur nommé `a0` et 4 par `a1` ; dans `P_all`, on n'en trouve aucune de ce type. Nous lisons ici la réponse de GCC, pas sa raison : nous n'avons pas établi pourquoi ces pointeurs ne peuvent pas être séparés de `tt` dans un cas et le peuvent dans l'autre.

Ce qui reste non établi. Nous ne savons pas *pourquoi* `lim2` traite les boucles A2 dans `P_all` et pas dans `P_none`. Les sondes sont une différence possible, mais pas la seule : la structure du code change aussi (par exemple, `P_all` a des fonctions `compute_A` et `expand_P1_P2` séparées que `P_none` n'a pas, ce qui indique des choix d'inlining différents). Les fichiers montrent des dépendances différentes pour `tt[0]` dans les deux cas, mais nous n'en tirons pas de cause. La causalité reste donc **ouverte**, et le lien avec la perte de gain mesurée reste une corrélation.

3.10 Comparaison avec la VM

[F] Dans la VM (Xeon virtualisé, GCC 13.3.0), le gain de `mayo_sign` pour MAYO-5 était de 7,1 %, proche des 7,3 % mesurés avec toutes les sondes sur la machine physique. [H] L'écart avec les 1,3 % sans sonde mélange l'effet de la machine et celui du compilateur ; il n'est pas expliqué. Nous le mentionnons sans nous en servir.

3.11 Discussion critique

Cette section sépare ce qui est établi, ce qui est déduit et ce qui reste hypothèse, puis discute les limites et les pistes abandonnées.

3.11.1 Bilan : établi, déduit, supposé

3.11.2 Pourquoi le gain se dilue

Le gain d'un noyau ne se retrouve sur la signature qu'à proportion de son poids. Pour MAYO-5 sans sondes, `m_calculate` pèse environ 931 453/1 504 741 ≈ 62 % de la signature, et son gain dans

TABLEAU 3.9 : Ce qui est établi, déduit ou supposé.

Statut	Énoncé	Appui
[F]	Nombre de `vpshufb` par entrée : 20→15 (MAYO-1), 36→30 (MAYO-5)	assembleur, preuve (prop. 2.5)
[F]	Gains de noyaux 12–29 % ; `mayo_sign` 10,3 % (MAYO-1), 1,3 % (MAYO-5) sur cette machine et ce compilateur	S1, S2, PH1, CT1
[F]	Le reste de la signature est identique entre bras ; la différence est dans `m_calculate` en contexte	mesure directe (`mcalc`)
[F]	Avec toutes les sondes (binaire instrumenté), le gain de MAYO-5 est 7,2 % ; sans, 1,3 % (résultat principal)	CT1
[F]	GCC 16.2.0 ajoute une écriture par accumulateur (32 octets pour un ymm, 16 pour un xmm) quand toutes les sondes ne sont pas actives, et pas de la même façon dans les deux versions	désassemblage E1
[I]	Les écritures en plus, inégales entre les deux versions (+60 pour A2, +36 pour l'officiel sur P1+P1t), expliquent l'essentiel de la perte de gain de MAYO-5 dans la signature	accord entre le code et les mesures
[H]	Le coût vient du nombre d'écritures que le processeur peut faire par cycle, plutôt que des dépendances entre écriture et lecture ou des conflits d'adresses	non testé
[F]	Dans le GIMPLE optimisé, sans toutes les sondes, des écritures `temp[i]=/tt[i]=` restent dans 8 blocs de travail (3 avec toutes les sondes) ; SRA ne touche à `temp` dans aucune construction	E1b (fichiers d'analyse `sra`, `optimized`)
[I]	Les 30 (A2) ou 36 (officiel) écritures supplémentaires de l'assembleur sont ces écritures	décompte et forme des tableaux
[H]	Une passe du compilateur qui déplace les écritures (`lim`) décide de les sortir de la boucle ou non ; elle les sort seulement quand toutes les sondes sont actives	fichiers d'analyse non produits
[H]	Les sondes changent la structure du code, et donc la décision de GCC	cohérent avec E1 et E1b, non démontré

la signature est de 18 406 ticks, soit 1,2 % de la signature : c'est, à la précision des mesures, le gain de 1,3 % observé. Pour MAYO-1, le rapport entre le gain de la signature (10,3 %) et celui de `m_calculate` (18,9 %) donne un poids d'environ 54 % [I]. La vraie question n'est donc pas la part de la signature occupée par `m_calculate`, mais pourquoi son gain tombe de 11,7 % (isolé) à 2,0 % (dans la signature, sans sondes) pour MAYO-5, alors qu'il reste de 18,9 % pour MAYO-1 [F].

3.11.3 Ce que la concordance statique/dynamique prouve et ne prouve pas

Le désassemblage donne un fait solide : le nombre d'instructions de la boucle change selon la construction, et pas de la même façon dans les deux versions. Les temps varient dans le même sens. Il faut cependant trois précautions.

1. **Un modèle à un seul paramètre qui exagère le gain de 40 %** ne peut pas servir de

validation. Le fait que le coût d'une écriture soit proche dans les deux versions (0,358 contre 0,346) est un contrôle de cohérence, pas une preuve.

2. **Corrélation, pas cause** : les constructions qui diffèrent par l'écriture en double diffèrent aussi par la présence de `lfence` entre les sous-étapes, par la taille de la fonction et par la répartition de la pile. Seule une expérience qui supprime l'écriture en double *sans autre changement du code* permettrait de conclure (voir la conclusion générale).

3. **Ticks et non cycles** : un « coût par écriture » de 0,35 tick correspond à un ordre de grandeur de 0,5 à 0,65 cycle si la fréquence réelle est de 3,5 à 4,5 GHz (non mesurée) pour un TSC à 2,4 GHz. Que cela soit compatible avec une écriture par cycle est une remarque **[H]**, pas un résultat.

Plusieurs raisons peuvent expliquer que le modèle exagère le gain (toutes **[H]**) : le coût des instructions n'est pas le même pour toutes (un `vpshufb` et une écriture ne coûtent pas pareil) ; d'autres boucles que P1/P1t ont aussi une écriture en double (`Ut_times_O`, `m_lambda_times_O`), sans que l'on connaisse ici leur nombre de tours ; et des chargements sur la même ligne de cache peuvent se recouvrir.

3.11.4 Pourquoi MAYO-1 et MAYO-5 se comportent différemment

[I] Pour MAYO-1, l'écriture en double ne touche que `P1t_times_Vt`, dans les deux versions et à un rapport voisin (71/91 = 0,78 contre 56/71 = 0,79 dans le banc). L'avantage calculé sur le code est gardé (0,770 sur le banc, 0,774 dans le contexte), ce qui va avec un gain de signature qui ne s'effondre pas. Pour MAYO-5, l'écriture en double touche les deux noyaux d'A2 mais un seul noyau de l'officiel. Cette lecture est compatible avec les faits, mais elle reste une déduction, pour la raison donnée plus haut.

3.11.5 Pistes explorées puis écartées

Nous avons essayé les pistes suivantes dans la VM avant de passer à la machine physique. Nous les gardons ici comme des hypothèses éliminées, avec leurs limites.

Fusion de `P1_times_Vt` et `P1t_times_Vt` en un seul parcours. Idée : lire $\mathbf{P}_1$ une seule fois au lieu de deux. Le nombre de lectures d'entrées est bien divisé par deux (MAYO-1 : 6 480 → 3 240 ; MAYO-5 : 20 306 → 10 153). Mais, avec des accumulateurs simultanés (option 1), le temps de `mayo_sign` augmente de 4,9 à 10,4 % (MAYO-1) et de 5,3 à 7,0 % (MAYO-5). Avec fermeture immédiate (option 2), il augmente de 67 à 78 % (MAYO-1, MAYO-3, MAYO-5). L'option 1 demande $V_{\max}$ accumulateurs, soit de +49 Kio (MAYO-1) à +159 Kio (MAYO-5) de pile. La correction est vérifiée (KAT et 5 000 essais indépendants par paramètre, sans écart). Interprétation : le travail SIMD ne change pas, et la pression sur la mémoire due aux accumulateurs dispersés l'emporte sur la baisse du nombre de lectures. Piste close ; mesures faites dans la VM seulement.

Transposée de $\mathbf{P}_1$. Un diagnostic sur la représentation de $\mathbf{P}_1$ et sur la façon d'y accéder conclut que la disposition en mémoire joue un rôle important dans le gain théorique maximal de 29 à 31 % observé sur MAYO-3 et MAYO-5. Nous ne pouvons pas dire quelle part vient du cache (pas de compteurs matériels). Le coût de construction de la transposée l'emporte sur le gain. Piste close.

Traitement par lots dans EF. Piste fermée ; le patch, le banc et les données sont archivés. Nous avons relevé une anomalie sur MAYO-2 : le rapport mesuré à un seul niveau change d'amplitude selon l'état du processus, ce qui impose de lire les rapports en tenant compte de la dérive de la phase concernée.

3.11.6 Limites de la validation

Une seule machine ; un seul compilateur sur la machine physique ; trois démarrages ; S2 sur cinq processus ; un gain de MAYO-5 proche du critère de 1 % ; des sondes complètes qui changent le comportement du code ; pas de cycles d'horloge ; pas de compteurs matériels ; Clang non examiné ; aucune preuve formelle de temps constant ; des propriétés de bourrage qui reposent sur une liste des utilisations de `Pv`. De plus, les mesures de l'article de référence ont été faites avec `clang-14` sur Skylake et Ice Lake, avec les paramètres du tour 1 : elles ne sont pas directement comparables aux nôtres (GCC 16.2.0, i9-9880H, paramètres du tour 2).

3.11.7 Sur l'intégration dans MAYO-C

Nous ne prenons ici aucune décision d'intégration. Voici ce qu'il faut peser : le gain de 10,3 % pour MAYO-1 (solide sur deux démarrages) ; le gain de 1,3 % pour MAYO-5 (proche du critère et dépendant du code produit par le compilateur) ; les conditions à respecter (deux voies égales, $m \in \{80, 142\}$) ; l'état de la preuve de temps constant (des indices seulement) ; et la dépendance au compilateur. Pour décider avec plus de solidité, il faudrait au moins une deuxième machine ou un deuxième compilateur.

CONCLUSION GÉNÉRALE ET PERSPECTIVES

Contributions effectives

Notre travail a étudié, dans MAYO-C, la façon dont les noyaux AVX2 `P1_times_Vt` et `P1t_times_Vt` traitent la fin des m-vecteurs. Les contributions établies sont les suivantes.

1. Un **comptage rigoureux** dans un modèle précis : le minimum est $\lceil m/32 \rceil$ instructions `vpshufb`, le schéma officiel en utilise $2\lceil m/64 \rceil$, et un gain n'est possible que si $0 < m \bmod 64 \leq 32$. MAYO-1 et MAYO-5 en profitent, pas MAYO-2 ni MAYO-3.

2. Une **mise en œuvre** (A2) pour une fin de vecteur d'au plus 16 éléments, avec une variante SAFE qui ne lit rien au-delà de l'entrée de P_1. Les propriétés de lecture, d'écriture, de tables et de bourrage sont justifiées par le code et vérifiées par des tests.

3. Des **gains mesurés** sur une machine physique, avec une unité statistique précisée : 12 à 29 % sur les noyaux, 10,3 % sur `mayo_sign` pour MAYO-1 et 1,3 % pour MAYO-5, retrouvés lors de deux démarrages de la machine.

4. La **localisation** de la perte de gain de MAYO-5 dans `m_calculate`, quand on mesure la signature complète. On observe aussi, dans le code produit par GCC 16.2.0, des écritures mémoire en plus et inégales entre les deux versions, qui correspondent à cette perte. L'analyse E1b montre que ce sont des stockages de tableaux d'accumulateurs refaits à chaque tour de boucle, et écarte la passe SRA comme cause.

5. Un **dépôt reproductible** (scripts, données brutes, empreintes) avec lequel les neuf analyses donnent exactement les mêmes résultats.

Limites

Le gain sur la signature de MAYO-5 est faible et dépend du choix du compilateur. Le lien de cause à effet entre les écritures en plus et la perte de gain n'est pas démontré, et le modèle chiffré exagère l'effet d'environ 40 %. Les mesures viennent d'une seule machine et d'un seul compilateur, en ticks TSC. Le temps d'exécution indépendant du secret n'est appuyé que par des indices, pas par une preuve. La variante pour une fin de vecteur de 17 à 32 éléments n'est pas réalisée. Nous ne prétendons à aucune nouveauté absolue.

Questions ouvertes et perspectives

Q1 – Le choix du compilateur. Comprendre pourquoi GCC 16.2.0 laisse ces écritures dans la boucle quand toutes les sondes ne sont pas actives, et les sort de la boucle quand elles le sont. L'analyse E1b a écarté SRA, DSE, PRE et IPA-SRA comme causes uniques (en les désactivant une à une, le doublon reste) et ne permet pas d'accuser FRE (le code change aussi ailleurs). Les fichiers d'analyse de `lim2` et de `pre` ont été produits (section 3.9.5) : `pre` ne touche pas à `temp[]` ni à `tt[]`, et `lim2` sort les écritures des boucles A2 dans `P_all` mais pas dans `P_none`. La cause de cette différence n'est pas établie : elle reste à chercher.

Q2 – La cause. Si l'on trouve un moyen de supprimer le doublon sans toucher à autre chose dans le code, comparer alors les temps avec et sans doublon, sur le même code. Les campagnes E2 à E5 prévues au départ sont réservées aux perspectives.

Q2bis – Comparaison ciblée. Faite (section 3.4) : 10,1 % pour MAYO-1 et 1,5 % pour MAYO-5, avec des compilations séparées et sans sondes, sur une machine et une session. Reste à la répéter sur d'autres machines et d'autres sessions.

Q3 – La solidité des résultats. Refaire l'analyse avec Clang et d'autres versions de GCC, et mesurer sur une deuxième machine, si possible avec les compteurs du processeur et en cycles d'horloge (hypothèse de performance H-P).

Q4 – L'extension. Réaliser et valider la variante pour $16 < r \leq 32$ (autres tailles de fin de vecteur) ; examiner d'autres paramètres que ceux étudiés ici. Cette extension est réservée aux perspectives.

Q5 – La garantie. Vérifier, de façon formelle ou avec un outil, que le programme compilé s'exécute en temps indépendant du secret.

Q6 – L'intégration. Décider, en fonction des réponses précédentes et des critères de l'encadrant.

RÉFÉRENCES

[1] MAYO Team, *MAYO : spécification du troisième tour* (algorithmes 7 et 8), https://pqmayo.org/assets/specs/mayo-round3-white-back.pdf, consulté en novembre 2026.

[2] R. Lidl, H. Niederreiter, *Finite Fields*, 2e éd., Cambridge University Press, 1997.

[3] R. A. Horn, C. R. Johnson, *Matrix Analysis*, 2e éd., Cambridge University Press, 2013.

[4] S. Lang, *Algebra*, 3e éd., Graduate Texts in Mathematics 211, Springer, 2002.

[5] A. J. Menezes, P. C. van Oorschot, S. A. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.

[6] P. W. Shor, *Algorithms for quantum computation : discrete logarithms and factoring*, FOCS 1994.

[7] J. Patarin, *The oil and vinegar signature scheme*, Dagstuhl Workshop on Cryptography, 1997.

[8] A. Kipnis, A. Shamir, *Cryptanalysis of the Oil and Vinegar Signature Scheme*, CRYPTO 1998.

[9] A. Kipnis, J. Patarin, L. Goubin, *Unbalanced Oil and Vinegar Signature Schemes*, EUROCRYPT 1999.

[10] W. Beullens, *Breaking Rainbow Takes a Weekend on a Laptop*, CRYPTO 2022.

[11] W. Beullens, *MAYO : Practical Post-Quantum Signatures from Oil-and-Vinegar Maps*, IACR ePrint 2021/1144 (SAC 2021).

[12] W. Beullens, F. Campos, S. Celi, B. Hess, M. J. Kannwischer, *Nibbling MAYO : Optimized Implementations for AVX2 and Cortex-M4*, IACR ePrint 2023/1683 (version consultée).

[13] NIST, *FIPS 203, 204, 205 : normes de chiffrement et de signature post-quantiques*, août 2024.

[14] NIST, *Call for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process*, 2022.

[15] P. Kocher, *Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems*, CRYPTO 1996.

[16] D. J. Bernstein, *Cache-timing attacks on AES*, 2005.

[17] V. Arlazarov, E. Dinic, M. Kronrod, I. Faradzev, *On economical construction of the transitive closure of an oriented graph*, Soviet Math. Doklady 11, 1970.

[18] M. R. Albrecht, G. V. Bard, C. Pernet, *Efficient Dense Gaussian Elimination over the Finite Field with Two Elements*, arXiv :1111.6549, 2011.

[19] W. Beullens, M.-S. Chen, S.-H. Hung, M. J. Kannwischer, B.-Y. Peng, C.-J. Shih, B.-Y. Yang, *Oil and Vinegar : modern parameters and implementations*, IACR TCHES 2023(3).

[20] Dépôt officiel `PQCMayo/MAYO-C`, commit `74de637a7c4a7f9fafa888183bc05f5cf920cd2d`.

ANNEXE A

INVENTAIRE DES ARCHIVES ET REPRODUCTION

État au 22 octobre 2026 (dernier inventaire vérifié) ; les ajouts postérieurs (E1b, identification des noyaux) sont dans le dépôt `mayo-a2-thesis`. Toutes les empreintes sont des SHA-256.

A.1 Versions définitives à citer

Fichier	SHA-256	Contenu
`A2_partie1.tar.gz`	`cbc0cc297ee0a0790b3cb7513d0384c6c323cfe26b6180b93fc7bd1b115477a6`	notes, code, scripts, analyses, brut S2
`A2_partie2.tar.gz`	`d8dbf1b169f249c720a6e74eb0b9c6e6c9fe44eae6087ef6fe188aa21ab0fcab`	brut PH1 et CT1
`A2_partie3.tar.gz`	`00d12e885de273e7851b270f8b53b780eb52fd5bbbe969b1dfe73402b687ac33`	brut S1
patch 0001	`e6c03362cbf974cf750f340453de874d667b5670a7e60eddafa54b35e29e0cea`	prototype A2
patch 0002	`751ec21fbd059e51a4b54f193f3e346a889d4b361cfdf78a3ab77c3e891b6280`	sondes
patch 0003	`b590a81d479fd4a32883a60efe4bb5525fb9f45bf92a1acd70f60b265596112d`	contrôle des sondes
patch 0004	`9c460d0de2193805f3b0c1716f62bf1390ca543778ccfb0022d995ef254a10c6`	analyse statique E1
`MAYO-C-A2_source_e72d407.tar.gz`	`e2252a0b51ab827bd097e5a0df73dcbb7ca13539af82b1e728737c58800f6833`	arbre S1/S2
`MAYO-C-A2-phases-v2_source.tar.gz`	`ecb0941b086affb7189151af4b58355acb39edd4de8d7089b34ac7328cf19ef3`	arbre PH1/CT1

Fichier	SHA-256	Contenu
E1_kit.tar.gz	efe0b71ca5dc1553b0c0a841dc59096216a214080d988ba32731cbe2fcde18f3	run_E1.sh, e1_analyze.py

Archive E1b (sorties de compilation, dumps GCC, désassemblages) : e1b_gcc16.tar.gz, SHA-256 49452120b1b787ad2aed0f30cbcece4f7584a6063768757d92214e264dcd3faa.

Archives brutes internes : S1 ebc01dfe…, S2 b1a69769…c723, PH1 e1a88aec…98cc, CT1 0506c2b5…92de. Les archives de la phase VM (A2_donnees_et_journaux, tail_aware_etude, archive_piste_batch_EF, etc.) sont de l'*historique* et ne doivent jamais être présentées comme performances matérielles.

A.2 Identification exacte du code

Le code est déterminé par l'arbre git (indépendant de l'horodatage des commits). Base : PQCMayo/MAYO-C commit 74de637a7c4a7f9fafa888183bc05f5cf920cd2d, puis git am des patchs dans l'ordre. Arbres attendus : après 0001 87d1729c…(S1, S2) ; après 0002 c8c37c97… ; après 0003 92bcede1…(PH1, CT1) ; après 0004 bf6955da….

```
git clone https://github.com/PQCMayo/MAYO-C && cd MAYO-C
git checkout 74de637a7c4a7f9fafa888183bc05f5cf920cd2d
git am 0001-*.patch 0002-*.patch 0003-*.patch 0004-*.patch
git rev-parse HEAD^{tree}   # attendu : bf6955dae657abb087b58e13e314b946e0d283fe
```

A.3 Reproduire les analyses

Prérequis : Linux, python3 avec numpy, tar, sha256sum. Les trois archives dans un dossier *D* :

```
sh reproduire_analyses.sh D travail_neuf
```

Le script vérifie les empreintes, relance les neuf analyses et affiche OK si la sortie est identique (résultat obtenu : 9 sur 9). Les sorties couvrent la synthèse S1/S2 (synthese_sessions.py), la comparaison (compare_sessions.py), la décomposition (phases_analyze.py), le contraste avec/sans sondes (phases_vs_ref.py) et le contrôle CT1 (controle_analyze.py).

A.4 Refaire les mesures

Conditions : Linux, AVX2, GCC ≥ 13 (mesures : 16.2.0), cmake, python3+numpy, gouverneur performance (sudo cpupower frequency-set -g performance), cœur 1 dédié, aucun autre travail pendant l'exécution.

Étape	Commande (racine de l'arbre correspondant)
S1, S2	`SESSION=<Ã¯tiquette> sh run_session_independante.sh`
PH1	`SESSION=<Ã¯tiquette> sh test/run_phases.sh`
CT1	`SESSION=<Ã¯tiquette> sh test/run_controle.sh`
E1	`SESSION=<Ã¯tiquette> sh test/run_E1.sh`
E1b	`python3 -I e1b.py <dossier_E1> <dossier_neuf>`

Chaque `machine.txt` enregistre date UTC, noyau, compilateur, microcode, gouverneur, `boot_id` et alimentation. La reproduction des *mesures* donnera des valeurs voisines et non identiques.

ANNEXE B

IDENTIFICATION DES NOYAUX DANS LE CODE GÉNÉRÉ (GCC 16.2.0, MAYO-5)

Identification par le pas d'adresse sur P_1 (section 2.3). « Écritures » : écritures dans le corps de boucle, de 32 octets (ymm) ou de 16 octets (xmm).

TABLEAU B.1 : Instructions / écritures (32 ou 16 octets) des boucles de travail selon la construction.

Construction	`P1` off.	`P1` A2	`P1t` off.	`P1t` A2
`ctx_official`	123 / 29	–	162 / 65	–
`ctx_ab`, `none`, `esk`, `mcalc`	123 / 29	135 / 52	162 / 65	139 / 52
`ctx_phases` (toutes sondes)	123 / 29	105 / 22	126 / 29	109 / 22

Les différences « sans toutes les sondes » moins « toutes sondes » sont : A2 `P1` +30 (52−22), A2 `P1t` +30, officiel `P1t` +36 (65−29), officiel `P1` 0. Pour la boucle `P1_times_Vt` d'A2, les 52 écritures de `P_none` sont 42 de 32 octets et 10 de 16 octets, et les 22 de `P_all` sont 18 et 4 : les 30 écritures supplémentaires comprennent donc 24 écritures de 32 octets et 6 de 16 octets (comme au chapitre 3). Pour `P1t_times_Vt` officiel, les +36 sont des écritures de 32 octets (36 accumulateurs ymm).

ANNEXE C

EXEMPLE JOUET DE MAYO : ILLUSTRATION SIMPLIFIÉE ET CONTRÔLE

L'exemple complet est donné à la section 1.3.6. Cette annexe le résume et explique comment il a été contrôlé. C'est une *illustration simplifiée* : les paramètres sont minuscules ($v = 2$, $o = 1$, $m = 2$, $k = 2$) et la matrice $Z = \begin{pmatrix} 1 & z \\ z & z^2 \end{pmatrix}$, avec $z^2 = z + 8$, est plus simple que celles de la spécification du troisième tour [1].

TABLEAU C.1 : Valeurs de l'exemple (hexadécimal, $\mathbb{F}_{16}$).

Donnée	Valeur
secret $\mathbf{O}$	$(\mathrm{b},8)^\top$
$\mathbf{P}_1^{(3)}$, $\mathbf{P}_2^{(3)}$	(6), (e)
$\mathbf{\Lambda}$	$\begin{pmatrix} 3 & 1 & 2 \\ 5 & 7 & 1 \end{pmatrix}$
$\mathbf{t}$	$(\mathrm{f},9)$
vinaigres $\mathbf{v}_0$, $\mathbf{v}_1$	$(2,3)$, $(3,\mathrm{a})$
$\mathbf{M}_0$ (avec $\mathbf{\Lambda}$), $\mathbf{M}_1$	$(9,4)^\top$, $(\mathrm{c},\mathrm{a})^\top$
$\mathbf{y}$, $\mathbf{A}$	$(7,7)$, $\begin{pmatrix} 6 & 3 \\ 2 & 4 \end{pmatrix}$ (déterminant d)
$\mathbf{x}$	$(\mathrm{b},9)$
$\mathbf{s}_0$, $\mathbf{s}_1$	$(\mathrm{b},4,\mathrm{b})$, $(\mathrm{f},\mathrm{e},9)$
$\mathscr{P}^\star(\mathbf{S})$, $L(\mathbf{\Lambda},\mathbf{S})$	$(\mathrm{b},2)$, $(4,\mathrm{b})$
$\mathscr{P}^\star(\mathbf{S}) + L(\mathbf{\Lambda},\mathbf{S})$	$(\mathrm{f},9) = \mathbf{t}$

Contrôle. Le programme `exemple_jouet_mayo_tour3.py` (dépôt de scripts) suit les étapes de l'algorithme 7 pour construire $\mathbf{A}$ et $\mathbf{y}$, résout le système, puis vérifie la signature de façon indépendante : il évalue directement la clé publique (matrices $\mathbf{P}_\ell$ complètes) et calcule $\mathscr{P}^\star(\mathbf{S}) +$

$L(\mathbf{\Lambda}, \mathbf{S})$. Il contrôle aussi que les polynômes s'annulent sur l'espace d'huile, et qu'un changement d'un seul élément de la signature donne un résultat différent de $\mathbf{t}$ (ici (c, f)).

Correction d'une erreur de l'ancienne version. Dans une version précédente, l'exemple annonçait la réduction modulo z^2+z+1 alors que les calculs utilisaient $z^2 = z+8$. Avec le polynôme écrit, on retrouvait $\mathscr{P}^\star(\mathbf{S}) = (\mathrm{c}, 5)$ au lieu de (f, 9). Le polynôme est maintenant $z^2 + z + 8$ partout, et l'exemple a été refait de bout en bout, avec en plus le terme linéaire dérivé de $\mathbf{\Lambda}$.

ANNEXE D

COMPARAISON CIBLÉE : MACHINE ET COMMANDES DE COMPILATION

Machine. Intel Core i9-9880H (16 processeurs logiques, 8 cœurs, 2 fils par cœur), Kali Linux, noyau 7.1.5, GCC 16.2.0 (Debian 16.2.0-3), CMake 4.3.4, gouverneur `powersave`, `no_turbo = 0`, processus fixé sur le cœur 1 (`taskset`), 401 paires, 200 signatures par processus. Arbre git du code mesuré : `4b2849969e8d3392150cdf6ce4e69b5bf59e75b0` (les quatre patchs de l'étude plus le banc de comparaison).

Commandes de compilation de `src/mayo.c`. Les quatre commandes ont la même forme : `/usr/bin/cc` avec `-O3 -funroll-loops -march=haswell -maes -fno-strict-aliasing -std=gnu99 -fvisibility=hidden -Wall -Wextra -Werror` (et les avertissements désactivés listés dans le fichier), `-DENABLE_AESNI -DMAYO_AVX -DMAYO_BUILD_TYPE_AVX2 -DTARGET_AMD64 -DTARGET_OS_UNIX`. Elles ne diffèrent que par :

- `-DMAYO_VARIANT=MAYO_1` ou `MAYO_5` ;
- pour A2 SAFE seulement : `-DMAYO_TAIL_XMM -DMAYO_TAIL_XMM_SAFE`.

Le fichier complet `commandes_compilation.txt`, le fichier `iso.csv` (1 604 lignes de mesures), `machine.txt` et `resultats_iso.txt` sont dans le dépôt de scripts et de résultats.

Reproduire. `sh test/run_comparaison_isolee.sh 1 401 200`.